



دانشگاه تهران
دانشکده فنی
دانشکده مهندسی برق و کامپیوتر



پایان نامه برای دریافت درجه کارشناسی
در رشته مهندسی برق، گرایش الکترونیک

طراحی سیستم تأمین داده جهت استنتاج WCET

نگارش: محمد مهدی دوست محمدی

شماره دانشجویی: ۸۱۰۱۰۰۱۴۲

استاد راهنما: دکتر مهدی کارگهی

(دانشیار دانشکده مهندسی برق و کامپیوتر)

محل انجام پروژه: دانشکده مهندسی برق و کامپیوتر دانشگاه تهران

شهریورماه ۱۴۰۵

تعهدنامه اصالت اثر

اینجانب محمد مهدی دوست محمدی تأیید می‌کنم که مطالب مندرج در این پایان‌نامه با عنوان:

«طراحی سیستم تأمین داده جهت استنتاج WCET»

حاصل تلاش اینجانب است و به دستاوردهای پژوهشی دیگران که در این نوشته از آن‌ها استفاده شده است، مطابق مقررات علمی و دانشگاهی ارجاع گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر در هیچ دانشگاه یا مؤسسه‌ای ارائه نشده است.

کلیه حقوق مادی و معنوی این اثر متعلق به دانشکدگان فنی دانشگاه تهران می‌باشد.

نام و نام خانوادگی دانشجو: محمد مهدی دوست محمدی

شماره دانشجویی: ۸۱۰۱۰۰۱۴۲

امضای دانشجو:

Mahdi

تاریخ: شهریورماه ۱۴۰۵

سپاسگزاری و قدردانی

در وهله نخست، مراتب سپاس و قدردانی صمیمانه و قلبی خود را تقدیم استاد راهنمای گرانقدرم، **دکتر مهدی کارگهی**، می‌نمایم. بدون تردید، راهنمایی‌های ارزشمند، نگاه ژرف علمی و حمایت‌های دلسوزانه ایشان در طول انجام این پروژه، چراغ راه و پشتیبان اصلی در عبور از چالش‌های پیچیده سخت‌افزاری و نرم‌افزاری این مسیر بود. از دانش بی‌پایان و اخلاق حرفه‌ای ایشان درس‌های فراوانی آموختم که همواره توشه راه پژوهشی و مهندسی‌ام خواهد بود.

همچنین از دوست و همراه گرامی‌ام، **محمد حسن آملی**، که در پروژه مکمل استنتاج WCET و طراحی سامانه تست خودکار همگام و پشتیبان بود و تبادل نظرات فنی با وی زمینه‌ساز غنای هر دو پروژه گردید، صمیمانه سپاسگزاری می‌کنم.

در پایان، قدردان زحمات بی‌دریغ و حمایت‌های همیشگی **پدر و مادر عزیزم** هستم که آرامش، انگیزه و امکان ادامه این مسیر را با مهر بی‌کران خود برایم به ارمغان آوردند.

چکیده

در سامانه‌های تعبیه‌شده و بی‌درنگ سخت (Hard Real-Time Embedded Systems)، تخمین دقیق و تضمین‌پذیر بیشینه زمان اجرا (WCET) عاملی حیاتی در اثبات ایمنی و زمان‌بندی‌پذیری سامانه به شمار می‌رود. در معماری‌های نوین پردازنده‌ها با خطلوله‌های چندمرحله‌ای، پیش‌بینی انشعاب و حافظه‌های نهان، روش‌های تحلیل ایستا به دلیل تخمین‌های بیش‌ازحد بدبینانه ناتوان بوده و در نقطه مقابل، رویکردهای سنتی ابزار دقیق نرم‌افزاری (Software Instrumentation) با تحمیل سربار اجرایی و اعوجاج زمانی (پدیده «اثر پروب»، رفتار اصیل پردازنده را دگرگون می‌سازند. این رساله به طراحی، پیاده‌سازی عملی و اعتبارسنجی جامع یک خطلوله غیرتهاجمی (Non-intrusive) جهت تأمین داده‌های ردیابی اجرای برنامه و استنتاج دقیق WCET در مقیاس چرخه ساعت پردازنده می‌پردازد.

سامانه ارائه‌شده با تمرکز بر مازول ردیابی سخت‌افزاری ETMv4 در زیرساخت دیباگ ARM CoreSight بر روی میکروکنترلر پیشرفته (ARM Cortex-M7) STM32H750 توسعه یافته است. در این فرآیند، چالش‌های پیچیده پیکربندی ثباتی نظیر آزادسازی قفل‌های سخت‌افزاری، توزیع توان و کلاک و مغایرت‌های راهنمای مرجع تراشه با استاندارد معماری برطرف گردید. داده‌های فشرده ردیابی دستورات از طریق گذرگاه موازی ۴ بیتی درگاه TPIU بدون تحمیل کوچک‌ترین سربار پردازشی صادر شده و توسط لاجیک آنالایزر دیجیتال با راهبرد اورسمپلینگ ۱۰ برابری ثبت می‌شوند. اسکرپت تخصصی TraceStreamProcessor.py با ترازسازی فریم‌ها و جبران اعوجاج‌های فیزیکی، ساختار اسنپ‌شات استاندارد صنعتی را تولید کرده و با ادغام با موتور دکودر مرجع OpenCSD و باینری ELF، جریان تفکیک‌شده دستورات اسمبلی و برچسب‌های زمانی نانو ثانیه‌ای را بازسازی می‌نماید.

به منظور تبدیل این خطلوله به یک ابزار صنعتی و کاربرپسند، افزونه اختصاصی CoreSight Trace Studio برای محیط توسعه VS Code طراحی و پیاده‌سازی شد که در قالبی سه‌مرحله‌ای، متوالی و نیمه‌خودکار، فرآیند تزریق خودکار درایورها، نشانه‌گذاری پنجره‌های پایش در کدهای منبع، تولید متناظر مانیفست‌های اسنپ‌شات و نمایش لاگ‌های روند اجرای برنامه را در یک داشبورد مدرن محقق می‌سازد. در فاز ارزیابی تجربی، عملکرد سامانه تحت سناریوهای چالش‌برانگیز شامل آزمون دوره‌ای تایمر ۱۰۰ میکروثانیه با ثبت انحراف زمانی ناچیز زیر $0.1 \mu s$ (اثبات قطعی ماهیت غیرتهاجمی)، تطبیق زمانی سیگنال‌های فیزیکی، و رهگیری وقایع ترکیبی وقفه‌ها با هندشیک سخت‌افزاری به اثبات رسید. این بستر در هم‌افزایی با سامانه مکمل آزمون خودکار، پلتفرمی اقتصادی، بومی و کارا جهت آزمون برنامه‌های سیستم‌های نهفته و سنجش ایمنی آن‌ها به ارمغان آورده است. چه بسا این سیستم بتواند در آینده در قامت یک رقیب برای ابزارهای آزمون نرم‌افزار تجاری بین‌المللی واقع شود.

واژگان کلیدی: بیشینه زمان اجرا (WCET)، ردیابی غیرتهاجمی دستورات (Instruction Trace)، مازول ETMv4، معماری ARM CoreSight، پردازنده Cortex-M7، میکروکنترلر STM32H7، دکودر OpenCSD، افزونه CoreSight Trace Studio، سامانه‌های بی‌درنگ سخت.

فهرست مطالب

فهرست جداول

فهرست تصاویر

فصل ۱

مقدمه و بیان مسئله

۱.۱ مقدمه و ضرورت پژوهش

در دهه‌های اخیر، سامانه‌های تعبیه‌شده (Embedded Systems) به ستون فقرات زیرساخت‌های حیاتی دنیای مدرن بدل شده‌اند؛ از سیستم‌های اویونیک و هدایت ناوبری در صنایع هوافضا و خودروهای هوشمند خودران گرفته تا تجهیزات کنترل علائم حیاتی در پزشکی و بازوهای رباتیک در خطوط اتوماسیون صنعتی. ویژگی مشترک و وجه تمایز بنیادین این رده از کاربردها، ماهیت «بی‌درنگ سخت» (Hard Real-Time) آن‌ها است. در سامانه‌های بی‌درنگ سخت، درستی و صحت کل سامانه صرفاً به درستی محاسبات منطقی و ریاضی محدود نمی‌شود، بلکه عامل «زمان» به عنوان یک بُعد تعیین‌کننده و غیرقابل‌مذاکره وارد عمل می‌گردد. به بیان دقیق‌تر، تولید یک پاسخ کاملاً صحیح از نظر منطقی، چنانچه با حتی کسری از میلی‌ثانیه تأخیر پس از موعد مقرر (Deadline) تحویل داده شود، در حکم خطای بحرانی سامانه بوده و می‌تواند فجایع جبران‌ناپذیر انسانی یا خسارات سنگین مالی به همراه داشته باشد.

برای تحقق ضمانت‌های قطعی در زمان‌بندی‌پذیری (Schedulability)، مهندسان سیستم‌های بی‌درنگ نیازمند دانستن کران بالای زمان اجرای تک‌تک وظایف محاسباتی هستند. این کران بیشینه، در متون علمی تحت عنوان «بیشینه زمان اجرا» (Worst-Case Execution Time - WCET) شناخته می‌شود [۱]. استخراج دقیق و قابل‌اطمینان WCET سنگ‌بنای طراحی سامانه‌های با قابلیت اطمینان بالا (High-Reliability) و تخصیص منابع پردازشی است [۲]. با این وجود، به دلیل جهش‌های فنی گسترده در معماری پردازنده‌های تعبیه‌شده در سال‌های اخیر، محاسبه و استنتاج این کمیت به یکی از پیچیده‌ترین چالش‌های مهندسی الکترونیک و کامپیوتر مبدل گشته است.

در سطح صنایع پیشرفته جهانی، خودکارسازی فرآیند آزمون محصول و استنتاج غیرتهاجمی زمان‌بندی توسط پلتفرم‌های نرم‌افزاری انحصاری و بسیار پرهزینه نظیر سامانه‌های شرکت Rapita Systems (مجموعه RVS / RapiTime)، بسترهای آزمون در حلقه سخت‌افزار (HIL) شرکت Speedgoat و ابزارهای مشابه انجام می‌پذیرد. هزینه لایسنس و به‌کارگیری این راهکارهای تجاری بالغ بر ده‌ها تا صدها هزار دلار (بیش از ۱۰ تا ۱۰۰ هزار دلار) بوده و دستیابی به آن‌ها نیازمند بودجه‌های سنگین است. از این رو، طراحی و توسعه یک پلتفرم جامع آزمون خودکار محصول و پایش غیرتهاجمی با حداقل هزینه ممکن، اهمیتی دوچندان و

توجیه اقتصادی و کاربردی بسیار بالایی پیدا می‌کند.

۲.۱ مسئله استنتاج WCET و چالش‌های معماری پردازنده‌های مدرن

در نسل‌های نخستین میکروکنترلرها (نظیر پردازنده‌های ۸ بیتی و ریزکنترلرهای اولیه)، خط سیر اجرای دستورات به شدت تعیینی (Deterministic) بود؛ به طوری که هر دستورالعمل تعداد چرخه‌های کلاک مشخص و ثابتی را مصرف می‌کرد و طراح سیستم می‌توانست با جمع جبری زمان دستورات و بازرسی چشمی کد اسمبلی، بدترین زمان اجرای یک تابع را با دقت بالا تخمین بزند. اما در پردازنده‌های امروزی تعبیه‌شده با کارایی بالا (نظیر هسته‌های ARM Cortex-M7)، مجموعه‌ای از نوآوری‌های سخت‌افزاری جهت افزایش توان محاسباتی به کار گرفته شده است که رفتار زمانی پردازنده را به شدت پویا، غیرخطی و وابسته به کانتکست تاریخی اجرا می‌نماید:

۱. **خط‌لوله‌های چندمرحله‌ای و سوپراسکالر (Superscalar Pipelines):** اجرای هم‌زمان بیش از یک دستور در چرخه کلاک و وابستگی‌های ساختاری داده‌ها (Data Hazards).

۲. **حافظه‌های نهان چندسطحی (Instruction & Data Caches):** وابستگی زمان واکنشی دستور به رخداد اصابت (Cache Hit) یا عدم اصابت (Cache Miss)، که موجب نوسان زمان دسترسی از چند نانو ثانیه تا ده‌ها چرخه کلاک می‌گردد.

۳. **واحدهای پیش‌بینی انشعاب (Branch Predictors):** حدس زدن نتیجه شرط‌ها پیش از حل شدن آن‌ها و نیاز به پاک‌سازی خط‌لوله (Pipeline Flush) در صورت پیش‌بینی نادرست.

۴. **تداخل گذرگاه‌ها و ساختارهای دسترسی مستقیم به حافظه (DMA):** رقابت بر سر پهنای باند حافظه مشترک میان هسته و تجهیزات جانبی.

این پدیده‌ها مجموعاً موجب بروز اثراتی موسوم به «ناهنجاری‌های زمانی» (Timing Anomalies) می‌گردند؛ وضعیتی که در آن یک رخداد با فرض بهینه‌سازی محلی (مانند اصابت به حافظه نهان)، در مقیاس کلی به دلیل تغییر ترتیب تخصیص منابع منجر به تأخیر در اجرای کلی برنامه می‌شود [۱]. در چنین محیط‌های پویایی، استخراج کران بالای زمان اجرا صرفاً با استفاده از مدل‌های نظری ریاضی، یا به ارقام غیرواقعی و بیش‌ازحد محافظه‌کارانه (Over-conservative) می‌انجامد که موجب اتلاف عظیم سخت‌افزار می‌شود، و یا به دلیل خطاهای تقریب مدل سخت‌افزاری، کران نامعتبری تولید می‌کند که نقض‌کننده ایمنی سامانه است.

۳.۱ معضل ابزار دقیق نرم‌افزاری و پدیده «اثر پروب»

برای رفع محدودیت‌های روش‌های صرفاً تحلیلی، مهندسان به روش‌های مبتنی بر اندازه‌گیری تجربی (Measurement-Based Timing Analysis - MBTA) روی آورده‌اند [۲]. در این رویکرد، کد برنامه

بر روی بستر سخت‌افزاری واقعی بارگذاری شده و با ورودی‌های گوناگون تحریک می‌گردد تا زمان‌های اجرای آن مستقیماً ثبت شود.

با این حال، بزرگ‌ترین مانع در پیاده‌سازی این روش، نحوه پایش و ثبت زمان‌ها و شناسایی مسیر پیموده‌شده است. روش سنتی مهندسی نرم‌افزار، استفاده از «ابزار دقیق نرم‌افزاری» (Software Instrumentation) است که در آن دستورات اضافه‌ای به سورس‌کد تزریق می‌شود؛ نظیر تغییر وضعیت پین‌های ورودی/خروجی عمومی (GPIO Toggle) در شروع و پایان هر تابع یا ذخیره مقادیر تایمرها در آرایه‌های حافظه. این دستکاری به شدت رفتار زمانی را آلوده می‌کند. این معضل مهندسی در متون علمی با عنوان **اثر پروب (Probe Effect)** شناخته می‌شود:

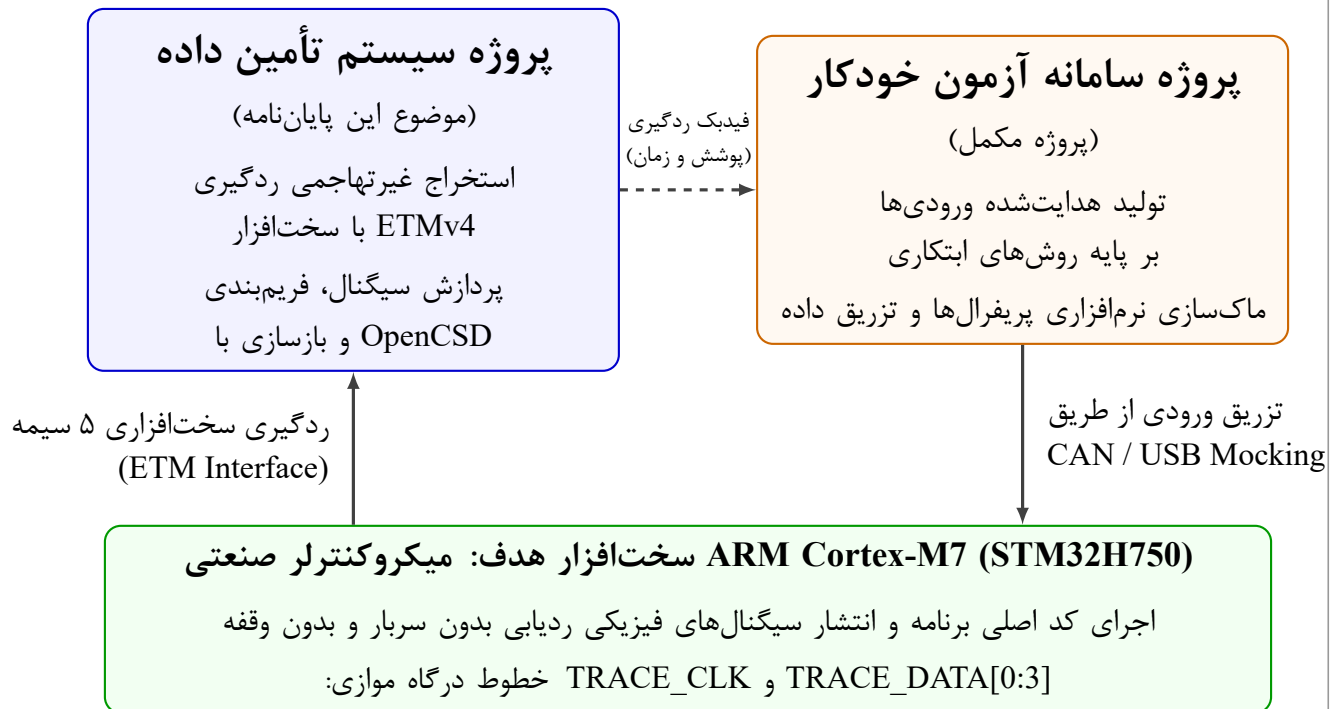
- افزودن دستورات پروب، خط‌لوله دستورات را مخدوش می‌کند.
 - کدهای پروب ساختار محلی حافظه نهان (Cache Locality) را به هم ریخته و باعث بیرون رانده شدن ناخواسته خطوط کش برنامه اصلی می‌شوند.
 - تأخیر زمانی تحمیل‌شده توسط این ابزارها ممکن است توالی دقیق وقفه‌ها و تداخل‌های بی‌درنگ را تغییر داده و رفتاری متناقض با عملکرد واقعی برنامه در محیط عملیاتی به نمایش بگذارد.
- بنابراین، برای استنتاج علمی، دقیق و قابل‌اتکای WCET، وجود یک سازوکار پایش کاملاً غیرتهاجمی (Non-intrusive) که بدون تغییر حتی یک بایت از کدهای اجرایی و بدون ایجاد کوچک‌ترین تغییر در رفتار چرخه‌ای پردازنده، جریان دقیق اجرای دستورات را استخراج نماید، ضرورتی حیاتی به شمار می‌رود.

۴.۱ معماری کلان پژوهش و مدل همکاری دوگانه

پژوهش حاضر در قالب یک معماری یکپارچه و در تعامل با یک سامانه آزمون خودکار مکمل طراحی شده است تا چرخه کامل استنتاج WCET را بر روی سخت‌افزار واقعی شکل دهد. این بستر جامع از دو پروژه مستقل و هماهنگ تشکیل شده است:

این دو پروژه عبارتند از:

۱. پروژه سیستم تأمین داده (موضوع پایان‌نامه حاضر): طراحی و استقرار «سامانه تأمین داده‌های ردیابی دقیق و غیرتهاجمی (Data Provider System)». رسالت این بخش، تسخیر امواج فیزیکی ردگیری از پایه‌های میکروکنترلر، نمونه‌برداری با فرکانس بالا، بازسازی جفت‌نیبل‌ها، فیلتر بسته‌ها، حل چالش‌های همگام‌سازی و تغذیه خودکار موتور تجزیه و تحلیل OpenCSD جهت استخراج دنباله دقیق دستورات اجرا شده و برچسب‌های زمانی با خطای رمزگشایی صفر و جیتر صفر است.
۲. پروژه سامانه آزمون خودکار (پروژه مکمل): طراحی و پیاده‌سازی «سامانه آزمون خودکار جهت استنتاج WCET (Automated Testing System)». این بخش به دنبال هدایت اجرای برنامه به سوی بدترین و طولانی‌ترین مسیرهای منطقی بر اساس الگوریتم‌های هوشمند جستجو و تحلیل خروجی‌های زمان‌بندی تأمین‌شده توسط پروژه اول است.



شکل ۱.۱: مدل مفهومی معماری یکپارچه دوگانه سامانه استنتاج WCET بر بستر سخت‌افزار واقعی

به منظور شفاف‌سازی کامل سهم نوآوری‌های نگارنده در این کار مشترک و تبیین مرزهای ارزیابی در جلسه دفاع، جدول؟؟ تفکیک دقیق وظایف و خروجی‌های هر یک از این دو پروژه را نشان می‌دهد.

۵.۱ اهداف تفصیلی پژوهش

با تکیه بر چارچوب کلان مسئله، اهداف مشخص این پایان‌نامه عبارتند از:

- پیکربندی ثبات‌های نگاشت‌شده در حافظه برای واحدهای ETMv4، CSTF و TPIU با استفاده از اسکریپت‌های کنترل دیباگر GDB و Firmware سیستم.
- استخراج فیزیکی سیگنال‌های ۵ سیمه Trace بر روی بستر سخت‌افزاری دست‌ساز با حفظ سلامت سیگنال.
- توسعه مبدل نرم‌افزاری پیشرفته جهت تبدیل داده‌های ناهمگام و خام لاجیک‌آنالایزر به فریم‌های ساختاریافته OpenCSD.
- بازسازی پیوسته بیش از دویست هزار دستور اجرایی با نرخ خطای صفر و استخراج تفکیک‌شده مدت زمان توابع و وقفه‌ها.
- اثبات عملیاتی بودن یک روش کاملاً اقتصادی و مبتنی بر سخت‌افزارهای تجاری در دسترس به عنوان جایگزین پروب‌های فوق‌گران‌قیمت صنعتی.

جدول ۱.۱: تفکیک وظایف، مرزبندی معماری و سهم نوآوری‌های نگارنده در مقایسه با پروژه مکمل

مؤلفه و حوزه فنی	سهم و نوآوری این رساله (پروژه تأمین داده)	سهم پروژه مکمل (سامانه آزمون)
سخت‌افزار و ردیابی بی‌درنگ	مهندسی معکوس و راه‌اندازی ثبات‌های ETMv4 و TPIU در STM32H7، حل مغایرت‌های راهنمای مرجع، تثبیت مسیرکشی پایه‌ها و نمونه‌برداری $10\times$ با لاجیک آنالایزر	تعامل با درگاه‌های پروگرامینگ جهت تزریق مقادیر اولیه و پیکربندی سناریوهای آزمون
پردازش سیگنال و داده‌های خام	توسعه اسکریپت <code>TraceStreamProcessor.py</code> ، آشکارسازی لبه‌ها، فیلترینگ گلیچ، تراز فریم‌های TPIU با الگوهای FSYNC و تولید اسنپ‌شات استاندارد	الگوریتم‌های فازینگ و تولید هوشمند داده‌های ورودی برای پیمایش مسیرهای طولانی
بازسازی رفتار برنامه و استخراج زمان	یکپارچه‌سازی با کتابخانه مرجع <code>OpenCSD</code> ، انطباق با باینری <code>ELF</code> ، بازسازی جریان دستورات و استخراج برچسب‌های زمانی نانو ثانیه‌ای و شمارش چرخه‌ها (CCI)	دریافت ماتریس داده‌های ردیابی و به‌کارگیری مدل‌های تحلیلی جهت استنتاج ریاضی WCET
ابزار توسعه و اتوماسیون (IDE)	طراحی و پیاده‌سازی افزونه اختصاصی <code>Core-Sight Trace Studio</code> در محیط <code>VS Code</code> جهت تزریق خودکار درایور و پنجره‌ها و مصورسازی نتایج	توسعه فریم‌ورک ماک‌سازی توابع در لایه میانی HAL جهت ایزوله‌سازی آزمون

۶.۱ نوآوری‌ها و دستاوردهای شاخص

دستاوردهای کلیدی حاصل از این پژوهش به شرح زیر خلاصه می‌شود:

- حذف کامل اثر پروب و دستیابی به جیتر صفر (Zero Jitter): اثبات عملیاتی این حقیقت که ردگیری سخت‌افزاری کدهای برنامه، حتی در تکرار ۴۹۲ بار اجرای روتین وقفه `SysTick`، همواره طول اجرای دقیقاً یکسان (۸ دستورالعمل) را ثبت می‌کند که نشانه انطباق مطلق با الزامات سامانه‌های بی‌درنگ سخت است.
- حل چالش بحرانی بسته‌های همگام‌سازی (A-Sync Issue): ابداع رویکرد سوئیچینگ کنترل شده بیت فعال‌سازی ETM (`TRCPRGCTLR.EN`) در مبادی پنجره‌های `Trace` کوچک، جهت تولید اجباری بسته‌های همگام‌سازی اولیه و نجات جریان داده از حالت غیرقابل‌رمزگشایی (`NO_SYNC`).
- بومی‌سازی و ساخت سخت‌افزار سفارشی در شرایط کمبود قطعات: طراحی و مونتاژ موفق هدربرد اختصاصی `STM32H750` در بازه زمانی فشرده یک هفته‌ای.
- طراحی خط لوله نرم‌افزاری خودکار و اتصال به `OpenCSD`: ایجاد پیوند میان خروجی‌های خام لاجیک آنالایزرهای اقتصادی و کتابخانه مرجع صنعتی رمزگشایی `Trace`.

۷.۱ ساختار فصول پایان نامه

این رساله در هشت فصل مدون شده است:

- **فصل دوم (مفاهیم پایه و پیشینه پژوهش):** کالبدشکافی تئوری‌های WCET، ساختار زیرسیستم عیب‌یابی و ردگیری ARM CoreSight و مقایسه تطبیقی رویکردهای صنعتی.
- **فصل سوم (معماری کلان سامانه):** تشریح مدل مفهومی سامانه، لایه‌های شش‌گانه جریان داده و نحوه تعامل با سامانه آزمون خودکار.
- **فصل چهارم (طراحی و پیاده‌سازی سخت‌افزار):** مشخصات برد سفارشی، اتصالات و سیم‌کشی پایه‌ها، ملاحظات فرکانس بالا و راهبرد نمونه‌برداری.
- **فصل پنجم (پیکربندی ETMv4 و Firmware):** پروتکل فشرده‌سازی ETMv4، راه‌اندازی ثباتی از طریق اسکریپت‌های GDB و حل چالش A-Sync.
- **فصل ششم (پردازش و رمزگشایی Trace):** معماری اسکریپت TraceStreamProcessor، ساختار بندی اسنپ‌شات‌ها و رمزگشایی دستور به دستور با OpenCSD.
- **فصل هفتم (ارزیابی تجربی و نتایج):** تحلیل عمیق نتایج آزمون‌های TimerTest، Routine_Irq و FreqTest و ارزیابی اقتصادی و مقایسه‌ای بسترها.
- **فصل هشتم (نتیجه‌گیری و کارهای آینده):** مرور دستاوردها، درس‌آموخته‌های مهندسی، جمع‌بندی اقتصادی پلتفرم و چشم‌انداز پیاده‌سازی FPGA.

فصل ۲

مفاهیم پایه و پیشینه پژوهش

۱.۲ مقدمه

ارزیابی و تضمین رفتارهای زمانی در سامانه‌های تعبیه‌شده بی‌درنگ، نیازمند فهم هم‌زمان مبانی ریاضی نظریه زمان‌بندی، ویژگی‌های ریزمعماری سخت‌افزار و استانداردهای صنعتی حاکم بر ردگیری دستورات است. در این فصل، ابتدا فرمول‌بندی ریاضی و مفاهیم بنیادین «پیشینه زمان اجرا» (WCET) و طبقه‌بندی رویکردهای استنتاج آن مورد بررسی قرار می‌گیرد. سپس معماری عیب‌یابی و پایش بی‌درنگ ARM CoreSight به عنوان استاندارد پیشرو در پردازنده‌های نوین تشریح شده و نقش هر یک از اجزای داخلی آن در زنجیره انتقال داده تبیین می‌گردد. در پایان، رویکرد این رساله در چارچوب پلتفرم‌های تجاری آزمون محصول و پیشینه پژوهش‌های بین‌المللی به صورت تطبیقی مقایسه خواهد شد.

۲.۲ مفهوم فرمالیزه‌شده پیشینه زمان اجرا (WCET)

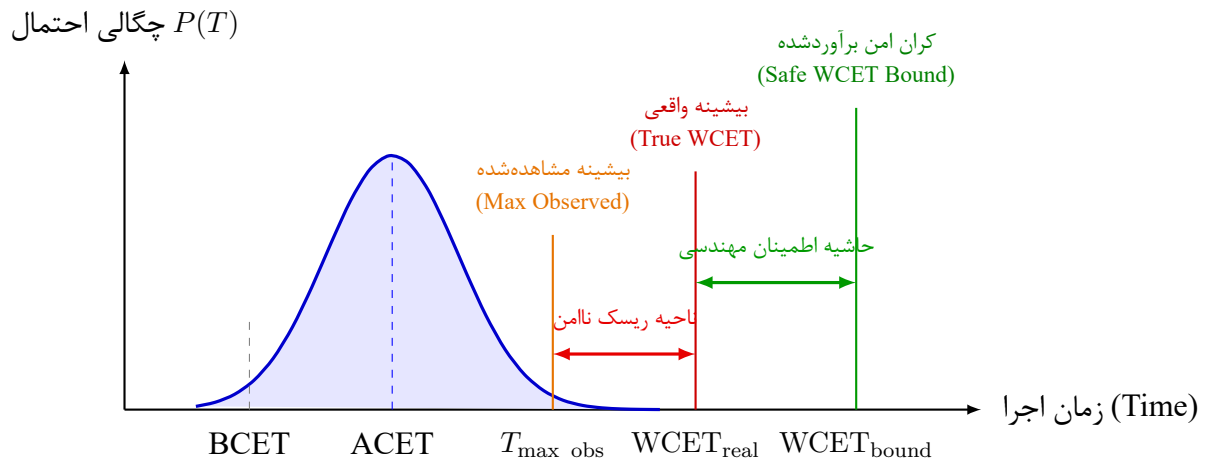
در یک سامانه تعبیه‌شده بی‌درنگ، هر وظیفه محاسباتی (Task) باید پیش از رسیدن به ضرب‌الاجل زمانی خود (Deadline) خاتمه یابد. زمان اجرای یک وظیفه، کمیت ثابتی نیست بلکه تابعی از ورودی‌های متغیر محیطی و حالات داخلی سخت‌افزار است.

به صورت ریاضی، اگر فضای ورودی‌های معتبر برنامه را با $\mathcal{P}$ و فضای تمامی حالات ممکن سخت‌افزار در آغاز اجرای وظیفه (شامل وضعیت خط‌لوله، حافظه نهان و صف‌های انتقال) را با $\mathcal{S}_0$ نمایش دهیم، زمان واقعی اجرای برنامه به صورت یک تابع غیرخطی به فرم زیر بیان می‌شود:

$$T = f(p, s_0) \quad \text{که در آن} \quad p \in \mathcal{P}, s_0 \in \mathcal{S}_0 \quad (۱.۲)$$

بر این اساس، پیشینه زمان اجرا (WCET) عبارت است از سوپرمم (بیشترین مقدار) این تابع بر روی تمامی ترکیبات ممکن ورودی‌ها و حالات اولیه سخت‌افزار:

$$\text{WCET} \triangleq \max_{p \in \mathcal{P}, s_0 \in \mathcal{S}_0} f(p, s_0) \quad (۲.۲)$$



شکل ۱.۲: فضای توزیع زمان اجرای برنامه و موقعیت پیشینه زمان اجرا نسبت به زمان‌های مشاهده شده و کران امن

همان‌طور که در شکل ۱.۲ نشان داده شده است، در عمل یافتن دقیق نقطه $WCET_{real}$ به دلیل بی‌شمار بودن فضای حالات ورودی و سخت‌افزار بسیار دشوار است. بنابراین، مهندسی سامانه‌های بی‌درنگ به دنبال یافتن یک کران بالای امن (Safe Upper Bound) است به طوری که:

$$WCET_{bound} \geq WCET_{real} \quad (۳.۲)$$

در صورتی که کران برآورد شده کمتر از مقدار واقعی باشد ($WCET_{bound} < WCET_{real}$)، با یک برآورد نامعتبر و ناامن (Unsafe Estimation) روبه‌رو هستیم که می‌تواند موجب شکست سیستم در سناریوهای بحرانی شود. از طرف دیگر، اگر این کران بیش‌از حد محافظه‌کارانه و بزرگ تعیین شود (Over-estimation)، مهندس طراح مجبور به انتخاب پردازنده‌های گران‌تر و منابع سخت‌افزاری مازاد بر نیاز خواهد شد.

۳.۲ طبقه‌بندی متدولوژی‌های استخراج WCET

رویکردهای تعیین پیشینه زمان اجرا در ادبیات علمی به سه شاخه بنیادین تقسیم می‌شوند [۱]:

۱.۳.۲ روش‌های تحلیل ایستا (Static WCET Analysis)

در روش‌های ایستا، کد باینری کامپایل شده بدون اجرای واقعی بر روی سخت‌افزار، مورد پردازش و تحلیل صوری ریاضی قرار می‌گیرد. این متدولوژی شامل سه گام کلیدی است:

۱. استخراج گراف جریان کنترل (Control Flow Graph - CFG): بازسازی ساختار شاخه‌ها، حلقه‌ها و توابع از کد ماشین.

۲. مدل‌سازی سخت‌افزار (Hardware Modeling): پیش‌بینی رفتار چرخه‌ای پردازنده (تخمین رفتار خطلوله و نرخ اصابت کش با استفاده از تئوری تفسیر انتزاعی یا Abstract Interpretation).

۳. تکنیک شمارش ضمنی مسیرها (IPET - Implicit Path Enumeration): فرمول‌بندی مسئله یافتن طولانی‌ترین مسیر در قالب یک مسئله برنامه‌ریزی خطی عدد صحیح (Integer Linear Programming - ILP). اگر x_i بیانگر تعداد دفعات اجرای بلوک پایه i و c_i کران بالای زمان اجرای آن بلوک باشد، مسئله به فرم زیر بهینه‌سازی می‌شود:

$$WCET = \max \sum_{i=1}^N c_i \cdot x_i \quad (4.2)$$

با قید برقراری معادلات پیوستگی جریان ورودی و خروجی در گره‌های گراف جریان برنامه.

نقاط ضعف روش‌های ایستا: با ظهور پردازنده‌های مدرن با هسته‌های پیچیده، مدل‌سازی رفتار چرخه به چرخه ریزمعماری به دلیل پدیده‌هایی نظیر عدم‌تعیین حافظه‌های نهان اشتراکی، تداخل گذرگاه‌ها و اجرای خارج از نوبت (Out-of-Order Execution) عملاً ناممکن گشته یا به تخمین‌هایی با خطای بالای ۵۰ تا ۱۰۰ درصد محافظه‌کاری منجر می‌شود.

۲.۳.۲ روش‌های مبتنی بر اندازه‌گیری تجربی (MBTA)

در رویکرد MBTA، برنامه با مجموعه‌ای از سناریوهای آزمون بر روی سخت‌افزار فیزیکی اجرا شده و زمان‌های پاسخ مستقیماً ثبت می‌گردند [۹]. چالش‌های اصلی این متدولوژی عبارتند از:

- پوشش ورودی‌های بدترین شرایط (Worst-Case Input Space): تضمین اینکه آزمون‌ها توانسته‌اند مسیر منجر به بیشینه زمان را فعال سازند بسیار دشوار است (نقشی که در پروژه حاضر به سامانه تست خودکار آقای حسن آملی واگذار شده است).
- اثر تهاجمی ابزار دقیق (Probe Effect): برای آگاهی از مسیر پیموده‌شده و زمان قطعات کد، نباید ساختار کد تغییر کند؛ معضلی که تنها با ردگیری سخت‌افزاری قابل حل است.

۳.۳.۲ روش‌های ترکیبی (Hybrid WCET Analysis)

این روش‌ها از تلفیق نقاط قوت دو رویکرد پیشین پدید آمده‌اند [۹]. در رویکرد ترکیبی، سنجش زمان اجرای قطعات پایه برنامه‌ریزی‌شده (Basic Blocks) بر عهده ردگیری سخت‌افزاری بر روی پردازنده واقعی گذاشته می‌شود تا رفتار پیچیده خطلوله و کش مستقیماً از واقعیت استخراج گردد؛ سپس با تزریق این زمان‌های تجربی دقیق به عنوان ضرایب c_i در مدل‌های ایستا (IPET)، بیشینه زمان اجرای کل ساختار برنامه به صورت ریاضی محاسبه می‌گردد.

۴.۲ معماری عیب‌یابی و ردگیری ARM CoreSight

برای پاسخ به نیاز صنعت در زمینه پایش و صحت‌گذاری سامانه‌های بی‌درنگ بدون متوقف کردن هسته پردازنده یا تحمیل اثر پروب، شرکت ARM معماری جامع CoreSight را به عنوان استاندارد سخت‌افزاری درون‌تراشه‌ای پایه‌گذاری نمود [۹].

معماری CoreSight یک زیرسیستم سخت‌افزاری کاملاً مجزا از هسته محاسباتی است که به صورت موازی در کنار هسته اجرا می‌شود. در تراشه‌های سری STM32H750 (مبتنی بر هسته Cortex-M7)، این زیرسیستم شامل زنجیره‌ای از واحدهای تخصصی است که در شکل ۲-۴ نشان داده شده‌اند.

۱.۴.۲ اجزای زیرسیستم CoreSight در STM32H750

۱. واحد پورت دسترسی به عیب‌یابی (DAP - Debug Access Port): درگاه اصلی اتصال ابزارهای دیباگ خارجی از طریق رابط‌های دو سیمه SWD یا چندسیمه JTAG جهت دسترسی به ثبات‌های کنترلی و فضای آدرس حافظه بدون مداخله هسته.

۲. گذرگاه پیشرفته ردگیری (ATB - Advanced Trace Bus): پروتکل گذرگاه داده همگام و پرسرعت درون‌تراشه‌ای که بسته‌های ردگیری تولیدشده توسط ماژول‌های پایش را میان اجزای گوناگون منتقل می‌سازد.

۳. واحد نگهبان داده و ردگیری (DWT - Data Watchpoint and Trace): ماژولی حیاتی که وظایف متعددی نظیر شمارش چرخه‌های زمانی (CYCCNT) و مقایسه آدرس‌های حافظه را انجام می‌دهد. ثبات‌های مقایسه‌گر DWT_COMP0 و DWT_COMP1 نقشی محوری در تعریف محدوده پنجره‌های پایش دارند؛ به طوری که با رسیدن آدرس اشاره‌گر دستورات به یک برچسب خاص، سیگنال شروع یا پایان ردگیری به ETM صادر می‌گردد.

۴. کیف ادغام ردگیری (CSTF - CoreSight Trace Funnel): ماژولی چندورودی-تک‌خروجی که وظیفه تلفیق جریان داده‌های ردگیری از منابع مختلف درون‌تراشه‌ای (نظیر هسته‌های چندگانه، ETM و ITM) به یک گذرگاه واحد ATB را با شناسه مبدأ (Trace Source ID) مشخص عهده‌دار است.

۵. بافر میانی جریان داده (ETF - Embedded Trace FIFO): یک حافظه SRAM حلقوی درون‌تراشه‌ای جهت جذب انفجارهای ناگهانی داده‌های Trace (Data Bursts)؛ در زمان‌هایی که انشعاب‌های شرطی مکرر موجب افزایش نرخ تولید بسته شده و پهنای باند پورت خروجی پاسخگو نیست.

۶. واحد واسط پورت ردیابی (TPIU - Trace Port Interface Unit): آخرین حلقه زنجیره درون‌تراشه‌ای که فریم‌های پروتکل ATB را دریافت کرده، آن‌ها را با الگوهای همگام‌سازی فریم (بایت 0x7F) بسته‌بندی نموده و به صورت جفت‌نیل‌های موازی روی کلاک خروجی و پایه‌های خارجی تراشه منتشر می‌سازد.

۵.۲ پیشینه پژوهش و مقایسه تطبیقی رویکردها

رویکردهای پایش، استخراج مسیر و ارزیابی زمان در سامانه‌های نهفته در جدول ۲-۴ مورد مقایسه قرار گرفته‌اند.

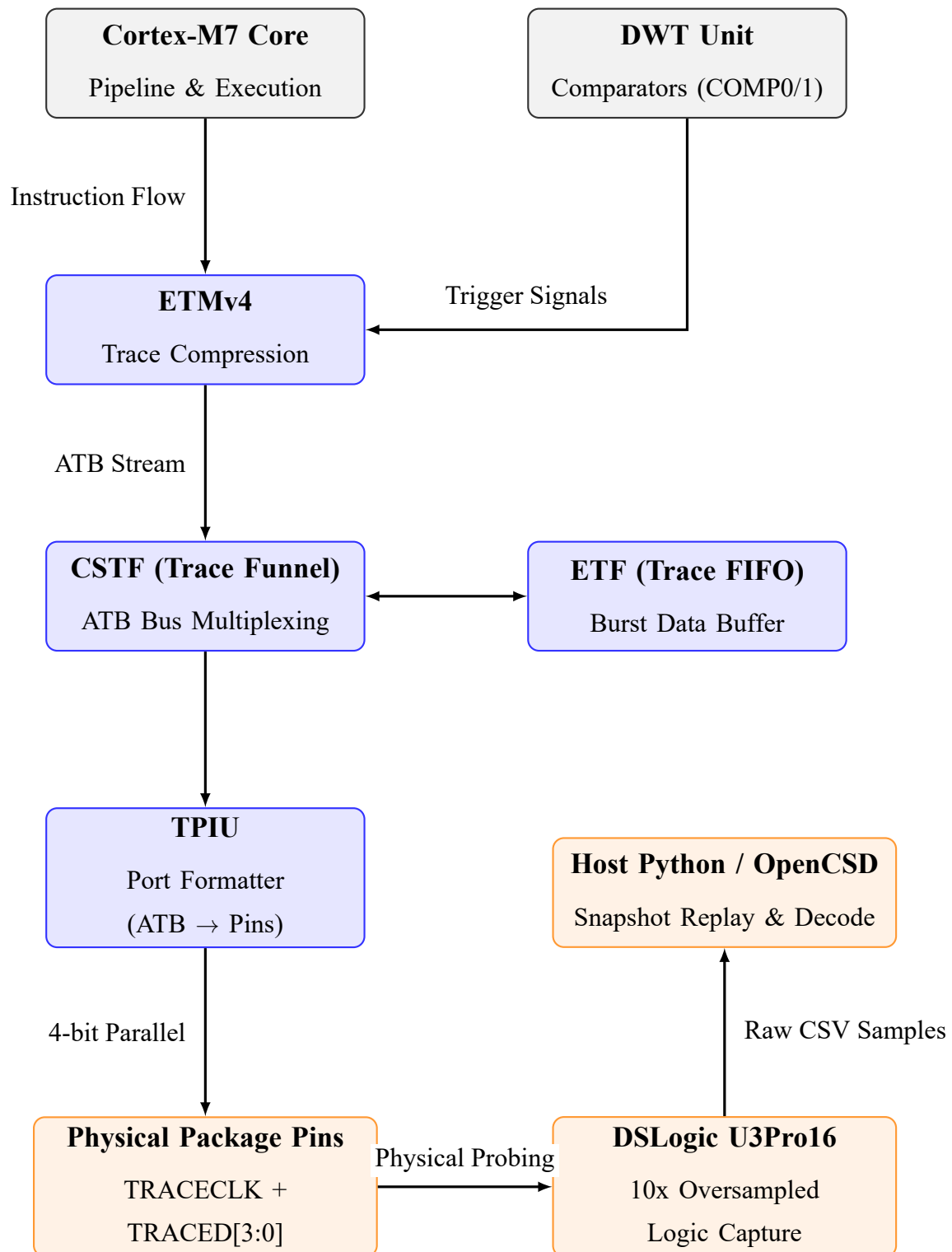
جدول ۱.۲: مقایسه تطبیقی پلتفرم‌های آزمون و ردگیری اجرای برنامه در سیستم‌های نهفته

معیار مقایسه	ابزار دقیق نرم‌افزاری	شبیه‌ساز نرم‌افزاری	پلتفرم تجاری (Rapita/Speedgoat)	پلتفرم این رساله
سر بار زمانی (Probe Effect)	بالا ($> 15\%$)	صفر	صفر (غیرتهاجمی)	صفر (غیرتهاجمی)
تغییر کد منبع (Source Intrusion)	نیاز دارد	بی‌نیاز	بی‌نیاز	کاملاً بی‌نیاز
دقت زمانی و رفتار چرخه‌ای	مخدوش و دارای جیتر	تقریبی	چرخه دقیق	چرخه دقیق (جیتر صفر)
هزینه لایسنس و سامانه آزمون	ناچیز	ناچیز	بسیار بالا ($> \$10,000$)	حداقل هزینه (اقتصادی)
وابستگی به برنامه کامپایل شده	بالا	کامل	فقط فایل باینری	فقط باینری (.elf)
پشتیبانی از پایش وقفه‌ها	نامطمئن	وابسته به مدل	کامل	کامل و تفکیک شده

همان‌گونه که از جدول ۱.۲ برمی‌آید، در صنایع هوافضا، خودروسازی و اتوماسیون پیشرفته، پلتفرم‌های تجاری آزمون خودکار محصول و تحلیل زمان‌بندی (نظیر محصولات Rapita Systems و بسترهای Speedgoat) استاندارد طلایی به شمار می‌روند؛ اما استقرار این پلتفرم‌ها نیازمند هزینه‌های بسیار سنگین (ده‌ها هزار دلار) و تجهیزات انحصاری است. یکی از مهم‌ترین اهداف و دستاوردهای محصول مشترک این پژوهش و پروژه همگام (سامانه آزمون خودکار)، تحقق یک پلتفرم آزمون محصول کارا و ارزان‌قیمت است که توانسته فرآیند آزمون نرم‌افزار، ماک‌سازی پریفرال‌ها و استخراج مسیر اجرا را با حداقل هزینه محقق سازد. همچنین در سطح سخت‌افزار دریافت سیگنال، اگرچه پروب‌های سخت‌افزاری تجاری نظیر SEGGER J-Trace به ثبت مستقیم در فرکانس‌های بالا می‌پردازند، اما درون آن‌ها از تراشه‌های اختصاصی FPGA و حافظه‌های سریع DRAM استفاده شده است. آزمون‌های تجربی این پروژه نشان داد که لاجیک‌آنالایزرهای عمومی اقتصادی نهایتاً برای فرکانس‌های بسیار بالا ایده‌آل نبوده و بهترین انتخاب سخت‌افزاری، یک برد سفارشی مبتنی بر FPGA است؛ هرچند هزینه پیاده‌سازی اختصاصی آن با هزینه خرید پروب‌های تجاری تقریباً برابری خواهد کرد. با این حال، ارزش افزوده اصلی این رساله، توسعه کل چرخه نرم‌افزاری استخراج و دی‌کودینگ داده‌ها با هزینه بسیار پایین است.

۶.۲ جمع‌بندی فصل

در این فصل پایه‌های تئوریک محاسبه WCET و ساختار لایه‌ای معماری ARM CoreSight تشریح گردید. همچنین ضرورت اجتناب از ابزار دقیق نرم‌افزاری و مزایای اقتصادی و فنی پلتفرم توسعه داده‌شده نسبت به راهکارهای تجاری گران‌قیمت بازار مورد ارزیابی قرار گرفت. با استناد به این مبانی، در فصل سوم به طراحی معماری کلان سامانه و پیاده‌سازی گام‌به‌گام خط لوله تأمین داده پرداخته خواهد شد.



شکل ۲.۲: زنجیره کامل جریان سیگنال‌ها در معماری ARM CoreSight از هسته پردازنده تا ابزار تحلیل

فصل ۳

معماری کلان سامانه

۱.۳ مقدمه

طراحی و پیاده‌سازی یک خطلوله داده جهت استنتاج بدترین حالت زمان اجرا (WCET) در سامانه‌های نهفته بی‌درنگ، نیازمند اتخاذ تصمیمات بنیادین معماری، تفکیک دقیق وظایف میان زیرسیستم‌های سخت‌افزاری و نرم‌افزاری، و ایجاد یک بستر عملیاتی بدون سربر است. در این فصل، ابتدا مسیر تکامل پژوهش و دلایل فنی گذار از بردهای مبتنی بر RP2350 به میکروکنترلر صنعتی STM32H750 تبیین می‌گردد. سپس نیازمندی‌های کارکردی و کیفی سامانه استخراج شده و معماری کلان سامانه به همراه نحوه تعامل و تبادل داده با سامانه آزمون خودکار (پروژه مکمل) با جزئیات فنی تشریح می‌شود.

۲.۳ مسیر تکامل سخت‌افزار: از RP2350 (Pico 2) تا STM32H750

در فاز مطالعاتی اولیه این پژوهش، تمرکز بر بهره‌گیری از میکروکنترلر نوین Raspberry Pi Pico 2 (مبتنی بر تراشه RP2350) قرار داشت. تراشه RP2350 با ساختار دوهسته‌ای ترکیبی (ARM Cortex-M33 و Hazard3 RISC-V) گزینه‌ای جذاب و اقتصادی به نظر می‌رسید. هدف اولیه، راه‌اندازی واحد ETM موجود در هسته Cortex-M33 و خواندن داده‌های ردگیری از طریق یک لایه میان‌افزار (Middleware) یا اسکریپت‌های کامپایلر بود؛ به نحوی که کد اصلی کاربر به زبان C کاملاً دست‌نخورده باقی بماند. با این حال، در جریان پیشبرد عملیاتی این راهکار، موانع فنی بازدارنده‌ای شناسایی گردید:

۱. **نقص مستندات و ناپختگی تراشه RP2350:** با توجه به عرضه بسیار جدید تراشه RP2350، مستندات رسمی مربوط به ثبات‌های نگاشت‌شده درگاه‌های عیب‌یابی CoreSight، ماتریس اتصال سویچ‌های ATB، و ساختار داخلی ETM در زمان اجرای پروژه بسیار محدود، ناقص و همراه با اراتاهای سخت‌افزاری متعدد بود.

۲. **عدم دسترسی به پایه‌های فیزیکی Trace در بردهای تجاری:** بزرگ‌ترین چالش فیزیکی، عدم هدایت پایه‌های موازی TPIU به پین‌هدرهای خروجی برد تجاری Pico 2 بود. بردهای توسعه تجاری

تنها پایه‌های دو سیمه SWD و پورت سریال UART را به بیرون هدایت کرده بودند؛ در حالی که پورت ردیابی پیوسته نیازمند دسترسی بدون واسطه به حداقل ۵ پایه سخت‌افزاری Trace (یک کلاک و ۴ خط داده) بود. دسترسی به این پایه‌ها نیازمند لحیم‌کاری‌های میکرومتری روی پکیج QFN بود که ریسک نویز، عدم تطبیق امپدانس و ناپایداری سیگنال را به شدت افزایش می‌داد.

۳. **محدودیت شدید پهنای باند و بافر ذخیره‌سازی در RP2350:** در تراشه رزبری پای، پهنای باند و حافظه بافر اختصاصی سخت‌افزاری برای ذخیره‌سازی و خروجی دادن داده‌های Trace بسیار محدود است. این گلوگاه موجب می‌شد که استخراج داده‌های ردیابی مستلزم مداخله نرم‌افزاری در روند اجرای برنامه و درگیر کردن یکی از هسته‌های پردازنده به عنوان واسط ذخیره‌سازی یا ارسال داده باشد. این درگیرسازی نرم‌افزاری و اشغال هسته، شرط بنیادین عدم مداخله و اصل غیرتهاجمی بودن (Non-intrusive) را کاملاً نقض می‌کرد و رفتار زمانی سیستم را به شکل نامطلوبی تغییر می‌داد.

۴. **جامعیت و بلوغ صنعتی خانواده STM32:** در سوی مقابل، شرکت STMicroelectronics در خانواده میکروکنترلرهای مبتنی بر ARM Cortex-M7 (به‌ویژه سری STM32H750) ساختار استاندارد، بالغ و کاملاً مستندشده CoreSight ETMv4 را ارائه می‌دهد. در این پردازنده‌ها، پایه‌های پورت موازی TPIU به عنوان توابع جانبی جایگزین (Alternate Function 0) بر روی پورت E (پایه‌های PE2 الی PE6) تعبیه شده‌اند و دسترسی سخت‌افزاری کاملاً پایدار را ممکن می‌سازند.

بنابراین، تصمیم راهبردی معماری بر گذار کامل به میکروکنترلر STM32H750VBT6 اتخاذ گردید تا تمرکز پژوهش بر هسته علمی استنتاج غیرتهاجمی و پردازش سیگنال‌های Trace متمرکز بماند.

۳.۳ تحلیل جامع نیازمندی‌های سامانه

برای تضمین صحت عملکرد در محیط‌های بی‌درنگ سخت، نیازمندی‌های سامانه در دو دسته کارکردی و کیفی تدوین گردید:

۱.۳.۳ نیازمندی‌های کارکردی (Functional Requirements)

۱. **پیکربندی غیرتهاجمی زیرسیستم CoreSight:** فعال‌سازی، تنظیم ثبات‌های کنترلی، باز کردن قفل دسترسی (LAR) و تعیین فیلترهای رویداد بدون تزریق حتی یک خط کد اضافه به باینری نرم‌افزار هدف.

۲. **پیکربندی جامع و مناسب تمامی بلوک‌های زنجیره Trace:** مقداردهی هماهنگ و سازگار کلیه واحدهای سخت‌افزاری فعال در خط‌لوله شامل DBGMCU، TPIU، قیف ادغام داده‌ها (CSTF) و هسته ETMv4 بدون ایجاد تناقض یا بن‌بست در جریان داده‌ها.

۳. **قابلیت تنظیم محل و طول پنجره ردیابی توسط کاربر:** فراهم ساختن امکان تعیین دقیق نقاط شروع و پایان فرآیند ردگیری (تعیین پنجره زمانی یا بازه آدرس‌های اجرایی هدف) توسط کاربر، جهت متمرکز شدن بر بخش‌های بحرانی کد و بهینه‌سازی حجم داده‌های ثبتی.

۴. هدایت موازی جریان Trace از پورت فیزیکی TPIU: ارسال بایت‌های فشرده‌شده داده از طریق رابط ۵ سیمه سنکرون (TRACECLK و TRACED[0:3]).
۵. بازسازی دقیق جریان بیت‌ها به بایت‌های پروتکل: نمونه‌برداری با فرکانس متناسب، آشکارسازی لبه‌ها و بازیابی جفت‌نیل‌های فریم‌بندی‌شده.
۶. همگام‌سازی اجباری در پنجره‌های کوتاه آزمون: حل چالش عدم همگام‌سازی (A-Sync) در روتین‌های وقفه و توابع میکروثانیه‌ای.
۷. بازسازی جریان کامل دستورات با موتور استاندارد: ادغام با کتابخانه صنعتی OpenCSD، نگاشت نمادهای فایل باینری (ELF)، و استخراج گام‌به‌گام دستورات اسمبلی.
۸. شمارش دقیق چرخه‌های پردازنده (Cycle-Accurate Timing): انتساب برچسب زمانی و چرخه‌های کلاک سپری‌شده به هر بلوک محاسباتی و روتین‌های وقفه.

۲.۳.۳ نیازمندی‌های کیفی و غیرکارکردی (Non-Functional Requirements)

- **عدم قطعیت و جیتر صفر (Zero Jitter):** سیستم مانیتورینگ نباید رفتار زمانی پردازنده را متغیر سازد؛ زمان‌های اندازه‌گیری‌شده در شرایط ورودی یکسان باید در سطح چرخه ماشین (Cycle-deterministic) کاملاً تکرارپذیر باشند.
- **نرخ خطای صفر در بازسازی (Zero Decode Error):** از دست رفتن حتی یک بایت یا خطا در تصمیم یک انشعاب شرطی، کل مسیر اجرای پس از آن را نامعتبر می‌سازد؛ از این رو نرخ خطای رمزگشایی در سناریوهای طولانی باید اکیداً صفر باشد.
- **قابلیت اتکا و کاهش هزینه سخت‌افزاری:** پرهیز از پروب‌های صنعتی انحصاری با قیمت چند هزار دلاری و جایگزینی آن با سخت‌افزارهای اقتصادی در دسترس.

۴.۳ معماری کلان و لایه‌های شش‌گانه سامانه

سامانه تأمین داده‌های ردیابی در قالب یک معماری شش‌لایه‌ای سلسله‌مراتبی مهندسی شده است. شکل ۱.۳؟ جریان تبدیل داده را از نانوثنیه‌های پالس‌های فیزیکی سیلیکون تا ماتریس‌های ساخت‌یافته تحلیل WCET به تصویر می‌کشد.



شکل ۱.۳: معماری شش‌لایه‌ای سامانه تأمین داده از هسته سخت‌افزار تا موتور رمزگشایی

۵.۳ مرزبندی معماری و تعامل با سامانه آزمون خودکار (پروژه مکمل)

پروژه حاضر (سامانه تأمین داده‌های ردیابی) و پروژه مکمل (سامانه آزمون خودکار) به گونه‌ای طراحی شده‌اند که دو سامانه کاملاً ماژولار، تفکیک‌شده و با مرزبندی صریح معماری (Separation of Concerns) باشند.

۱.۵.۳ حفظ اصالت کدهای کاربر و بازنگاشت در لایه میانی

یکی از شروط بنیادین در ارزیابی رفتارهای زمانی سیستم‌های نهفته، اجتناب از دستکاری کد اصلی برنامه کاربر است. در پروژه مکمل، شبیه‌سازی و تحریک مقادیر پریفرال‌های خارجی بدون تغییر در کدهای اصلی کاربر و صرفاً از طریق بازتعریف توابع لایه میانی HAL صورت گرفته است؛ به نحوی که ثبات‌های پریفرال به بخش دیگری از حافظه بازنگاشت (Remap) شده و داده‌های آزمون بدون نیاز به تغییر منطق سورس‌کد به برنامه خورنده می‌شوند.

۲.۵.۳ استقلال عملیاتی و پروتکل تبادل داده

معماری و پارتیشن‌بندی میان این دو پروژه به گونه‌ای پیاده‌سازی شده است که کارکرد هر دو سامانه به طور کامل از یکدیگر مستقل باشد و ارتباط آن‌ها صرفاً به تبادل استاندارد داده محدود گردد. سامانه تأمین داده به عنوان یک ناظر سخت‌افزاری کاملاً مستقل عمل می‌کند که هیچ وابستگی به منطق آزمون ندارد؛ وظیفه این سامانه منحصراً ثبت سیگنال‌های فیزیکی، بازسازی جریان دستورات و تحویل ماتریس ساخت‌یافته ردگیری اجرای برنامه (Execution Trace Matrix) است:

$$\mathcal{M}_{\text{trace}} = \{(PC_k, \Delta t_k, \text{BranchTaken}_k, \text{Context}_k)\}_{k=1}^M \quad (۱.۳)$$

در این رابطه، PC_k آدرس ۳۲ بیتی شمارنده برنامه، Δt_k تعداد چرخه‌های کلاک سپری‌شده بین رخدادها، BranchTaken_k وضعیت تحقق یا عدم تحقق انشعاب شرطی، و Context_k شناسه زمینه اجرایی پردازنده است. این ماتریس به عنوان خروجی رسمی سامانه تأمین داده در اختیار سامانه مکمل قرار می‌گیرد تا برای هدایت آزمون‌های بعدی و استنتاج بدترین حالت زمان اجرا (WCET) به کار گرفته شود. مدل مفهومی مرزبندی معماری و نحوه تعامل بدون مداخله میان این دو سامانه در شکل ۳۲ به تصویر کشیده شده است.



شکل ۲.۳: مدل مرزبندی معماری و تبادل داده میان سامانه تأمین داده و سامانه آزمون مکمل

۶.۳ معماری نهایی سامانه

پیرو تحلیل نیازمندی‌ها، تفکیک لایه‌های شش‌گانه پردازش داده و تبیین مدل مرزبندی با سامانه آزمون مکمل، پیکربندی عملیاتی کل سیستم نیازمند یک بلوک‌دیگرام جامع معماری است که مرزهای فیزیکی، جایگاه دقیق دستگاه‌های ابزار دقیق و اندازه‌گیری، و خط‌لوله پردازش نرم‌افزاری میزبان را به وضوح نشان دهد. ساختار عملیاتی سامانه در سه بخش بنیادین (سخت‌افزار هدف، لایه اندازه‌گیری و ابزار دقیق، و سامانه نرم‌افزاری میزبان) سازمان‌دهی شده است:

۱. **بخش اول: سخت‌افزار هدف تحت آزمون (Target Hardware / DUT):** میکروکنترلر صنعتی STM32H750VBT6 که مجهز به هسته محاسباتی پیشرفته ARM Cortex-M7 است. کدهای کاربر بدون نیاز به هیچ‌گونه ابزار دقیق نرم‌افزاری یا تزریق پروب‌های آلوده‌کننده زمان، با رفتار اصیل خود اجرا می‌شوند. مازول آن چیپ ETMv4 با پایش بی‌درنگ خط‌لوله دستورات، اطلاعات شاخه‌ها (Atom Packets)، شمارنده چرخه‌ها (CCI) و برجسب‌های زمانی سراسری (TS) را تولید کرده و از طریق گذرگاه ۳۲ بیتی ATB به سمت قیف ادغام داده‌ها (CSTF) و بافر حلقوی ETF گسیل می‌دارد. در نهایت، واحد واسط پورت ردیابی (TPIU) این بسته‌ها را در فریم‌های ۱۶ بیتی استاندارد سازمان‌دهی نموده و به صورت یک گذرگاه موازی ۴ بیتی سنکرون (شامل پایه PE2 به عنوان کلاک خروجی TRACECLK و پایه‌های PE3..PE6 به عنوان خطوط موازی داده TRACED[0:3]) بدون تحمیل کوچک‌ترین سربار محاسباتی یا توقف هسته به بیرون صادر می‌کند.

۲. **بخش دوم: تجهیزات اندازه‌گیری و ثبت فیزیکی سیگنال‌ها (Measurement & Acquisition Instrumentation):** این بخش به عنوان پل واسط فیزیکی میان میکروکنترلر هدف و رایانه میزبان عمل می‌نماید و شامل مؤلفه‌های سخت‌افزاری زیر است:

• **پروب‌های پرسرعت با طول کم:** کابل‌های رابط شیلددار با طول کمتر از ۱۰ سانتی‌متر جهت جلوگیری از نوسانات پله‌ای (Ringing) و انطباق امپدانس به پایه‌های پورت E میکروکنترلر و پین‌های زمین مجاور متصل شده‌اند.

• **دستگاه لاجیک آنالایزر سخت‌افزاری (DSLogic U3Pro16):** ثبت الکتریکی سیگنال‌های فرکانس بالا بر عهده این ابزار صنعتی ۱۶ کاناله قرار دارد. این دستگاه با بهره‌گیری از یک بافر آن‌برد ۲ گیگابیتی DDR3 DRAM و استراتژی نمونه‌برداری ناهمگام ۱۰ برابری ($10\times$) Asynchronous Oversampling، پالس‌های ساعت و داده را با دقت بسیار بالا ثبت کرده و اثرات جیت و لغزش فاز را در مبدأ خنثی می‌سازد.

• **درگاه پرسرعت USB 3.0:** داده‌های نمونه‌برداری شده با پهنای باند بالای ۵ گیگابیت بر ثانیه و در مد استریم پیوسته و بدون سرریز، از لاجیک آنالایزر به رایانه میزبان منتقل می‌شوند.

۳. **بخش سوم: سامانه نرم‌افزاری میزبان و رپر یکپارچه‌ساز (Host Processing System & Studio Wrapper):** در رایانه میزبان (Host PC)، افزونه اختصاصی CoreSight Trace Studio به عنوان یک محیط یکپارچه توسعه و رپر جامع (Toolchain Wrapper) در محیط VS Code عمل می‌کند. این ابزار به جای قرارگیری صرف در انتهای خطلوله، به عنوان یک چتر نرم‌افزاری دربرگیرنده کل مؤلفه‌های پردازشی عمل نموده و خطلوله را مدیریت می‌نماید:

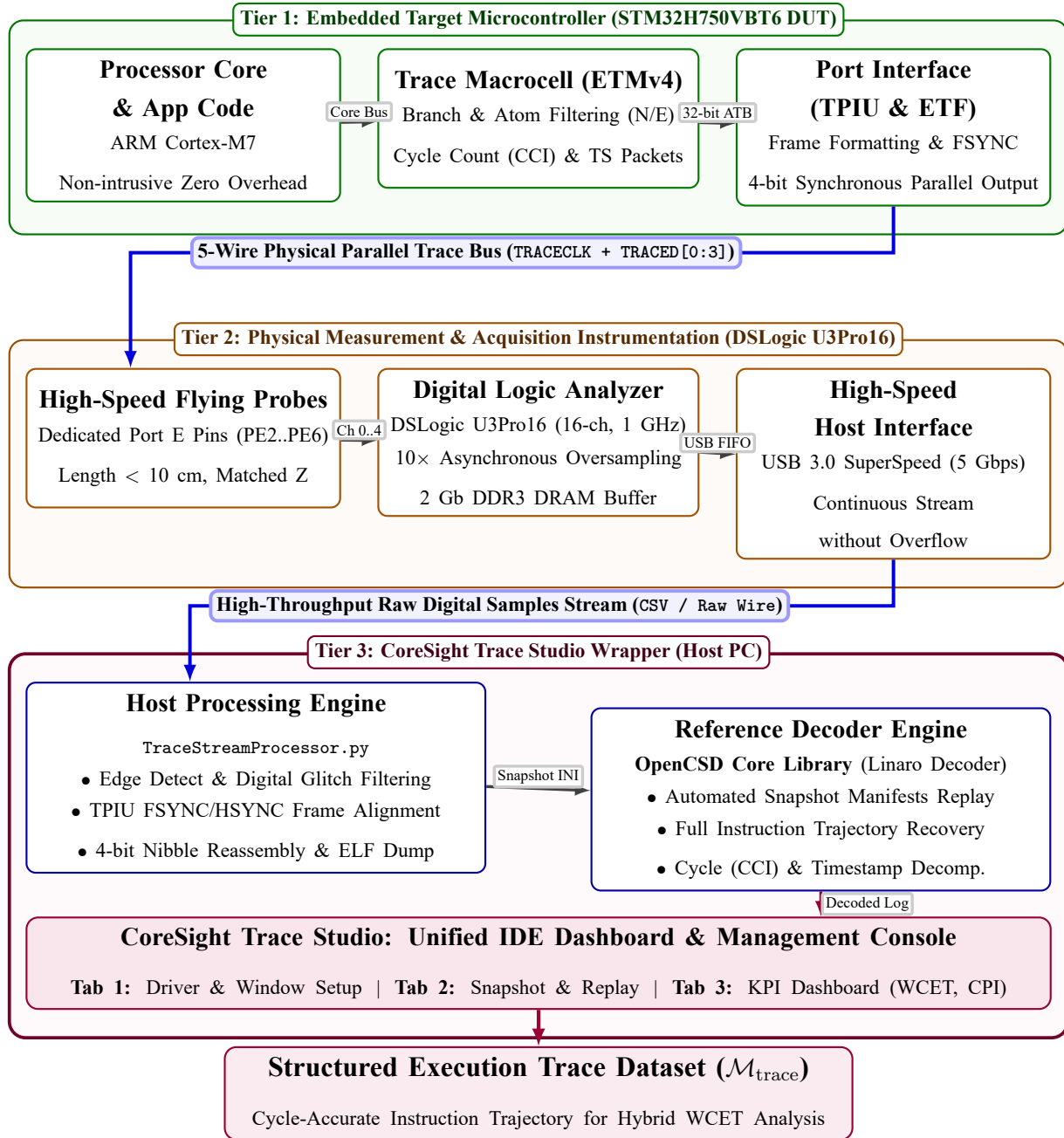
• **موتور پیش‌پردازش پایتون (TraceStreamProcessor.py):** فایل نمونه‌های خام لاجیک آنالایزر را دریافت و پردازش نموده، لبه‌های معتبر کلاک را با فیلتر هوشمند گلیچ استخراج می‌کند، نیبل‌های ۴ بیتی را به بایت‌های پروتکل تجمیع می‌نماید، فریم‌ها را بر پایه الگوهای FSYNC/HSYNC تراز فاز می‌کند و با تفکیک سگمنت‌های بارگذاری از باینری ELF، بسته اسنپ‌شات استاندارد OpenCSD را تولید می‌نماید.

• **موتور رمزگشای مرجع (OpenCSD Library):** با فراخوانی مانیفست‌های اسنپ‌شات و اجرای ابزار استاندارد trc_pkt_lister، جریان بسته‌های ردیابی و باینری برنامه را ادغام کرده و دنباله دقیق دستورات اسمبلی، برچسب‌های زمانی و شمارش چرخه‌ها را بازسازی می‌کند.

• **کنسول کنترلی و داشبورد استودیو (Trace Studio Dashboard):** این پوسته یکپارچه، فرآیند تزریق خودکار درایورها به کد منبع، نشانه‌گذاری پنجره‌های پایش، هماهنگی اجرای خطلوله و مصورسازی زنده شاخص‌های کلیدی کارایی (KPI) نظیر تعداد دستورات، مجموع چرخه‌ها، میانگین CPI و بیشینه زمان اجرا را در قالبی تعاملی محقق می‌سازد.

خروجی نهایی این معماری، ماتریس جامع ردگیری اجرای برنامه (M_{trace}) است که به سامانه آزمون خودکار تحویل داده می‌شود تا به عنوان سنگ‌بنای ریاضی جهت استنتاج نهایی WCET مورد بهره‌برداری قرار گیرد.

شکل ۳-۱؟ معماری نهایی و انتها-به-انتها پلتفرم تأمین داده را با نمایش رپر یکپارچه‌ساز CoreSight Trace Studio و تعامل لایه‌ها به تصویر می‌کشد.



شکل ۳.۳: معماری نهایی و جامع فیزیکی و نرم‌افزاری سامانه با تبیین دقیق جایگاه سخت‌افزار هدف، ابزار دقیق اندازه‌گیری و رپر خط‌لوله میزبان

۷.۳ جمع‌بندی فصل

در این فصل ریشه‌های فنی گذار از پردازنده‌های با دسترسی محدود به میکروکنترلر قدرتمند STM32H750 تبیین گردید. نیازمندی‌های دقیق سامانه استخراج شد و لایه‌های شش‌گانه تبدیل داده مورد تدقیق قرار گرفت. همچنین سازوکار مرزبندی معماری با سامانه آزمون مکمل بر پایه استقلال کارکردی و تبادل استاندارد ماتریس ردگیری پی‌ریزی شد و در بخش پایانی، معماری نهایی سامانه با تبیین دقیق جایگاه سخت‌افزار هدف، ابزار دقیق اندازه‌گیری و خط‌لوله پردازش میزبان مستندسازی گردید. در فصول بعدی، به پیاده‌سازی سخت‌افزاری و ثبت الکتریکی سیگنال‌ها پرداخته خواهد شد.

فصل ۴

طراحی بستر سخت‌افزاری و ثبت سیگنال‌های فیزیکی Trace

۱.۴ مقدمه

سیگنال‌های درگاه ردیابی موازی (Parallel Trace Port) از حیث ماهیت الکتریکی، در رده سیگنال‌های دیجیتال فرکانس بالا قرار می‌گیرند. در این فرکانس‌ها، فرضیات ایده‌آل مدارهای متمرکز جای خود را به پدیده‌های خطوط انتقال، اعوجاج شکل‌موج، خازن‌ها و سلف‌های پارازیتی و تداخل متقابل سیگنال‌ها (Crosstalk) می‌دهند. از سوی دیگر، شرایط تحریم و کمبود تجهیزات آزمایشگاهی پیشرفته در کشور، مانع بزرگی بر سر دستیابی به بردهای ارزیابی گران‌قیمت صنعتی به شمار می‌رفت.

در این پژوهش، مسیر توسعه بستر سخت‌افزاری شامل دو فاز عملیاتی بود: در فاز نخست، به دلیل نبود گزینه‌های تجاری مناسب، یک هدربرد دست‌ساز با سیم‌کشی دستی بر روی برد هزارسوراخ توسعه داده شد که با چالش‌های متعدد نویز، ناپایداری در پروگرم کردن و شکنندگی اتصالات فیزیکی روبرو بود. در فاز دوم و با گذر زمان، یک هدربرد استاندارد تجاری از میکروکنترلر STM32H750VBT6 در بازار داخلی در دسترس قرار گرفت و با جایگزینی آن، تمامی موانع فیزیکی و سیگنالی برطرف گردید.

در این فصل، چالش‌های ساخت هدربرد دست‌ساز، مشخصات فنی و مدار هدربرد تجاری، تمهیدات یکپارچگی سیگنال (Signal Integrity)، معماری سمپلینگ ۱۰ برابری با لاجیک آنالایزر DSLogic U3Pro16، مشاهدات تجربی تغییر دیوتی‌سایکل و توجیه تئوری آن بر پایه خطای کوانتیزاسیون دیجیتال، و در نهایت دلایل بنیادین رد استفاده از درگاه تک‌سیمه Serial Wire جهت دریافت Trace تشریح می‌گردد.

۲.۴ چالش دسترسی سخت‌افزاری و ساخت هدربرد دست‌ساز اولیه

در آغاز پروژه، بررسی گزینه‌های سخت‌افزاری موجود برای میکروکنترلرهای سری Cortex-M7 شرکت STMicroelectronics نتایج مایوس‌کننده‌ای به همراه داشت:

۱. بردهای ارزیابی رسمی (STM32H743I-EVAL و Keil MCBSTM32H7): این بردها مجهز

به کانکتورهای اختصاصی ۲۰ پین و ۳۸ پین ردیابی (MICTOR / CoreSight 20) هستند؛ اما قیمت آن‌ها بیش از ۵۰۰ تا ۱۰۰۰ دلار بوده و به دلیل محدودیت‌های وارداتی و تحریمی، امکان تهیه آن‌ها برای یک پروژه دانشگاهی در بازه زمانی معقول وجود نداشت.

۲. **بردهای عمومی دیسکآوری و نوکلئو (NUCLEO-144):** اگرچه بردهایی نظیر NUCLEO-H743ZI در بازار یافت می‌شدند، اما پایه‌های درگاه TPIU (به‌ویژه پایه‌های پورت E یعنی PE2 الی PE6) بر روی این بردها به صورت مشترک با مدارهای کنترلر فیزیکی اترنت (Ethernet PHY)، کریستال‌های فرعی یا سوکت‌های هدر Arduino متصل شده بودند. اتصال این پایه‌ها از مسیرهای طولانی، بدون محافظ و با خازن پارازیتی بالا بر روی برد، شکل موج کلاک ردیابی را کاملاً دفرمه می‌کرد و مانع ثبت پایدار داده‌ها می‌شد.

با توجه به این بن‌بست، در فاز نخست پروژه تصمیم گرفته شد یک هدربرد اولیه به صورت کاملاً دست‌ساز بر روی برد هزارسوراخ (Perfboard) با استفاده از یک برد مبدل تبدیل پکیج LQFP100 به پایه‌های دیپ (DIP) ساخته و سیم‌کشی گردد. در این مدار، پایه‌های تغذیه، خازن‌های دی‌کوپلینگ، کریستال خارجی، مدار ریست و اتصالات پین‌هدرها با سیم‌بندی دستی نقطه‌به‌نقطه اجرا شد. همچنین یک ماژول تغذیه کاهنده دی‌سی به دی‌سی (Buck Converter) با پورت ورودی USB برای تأمین ولتاژ پایدار ۳.۳ ولت بر روی برد تعبیه گردید.

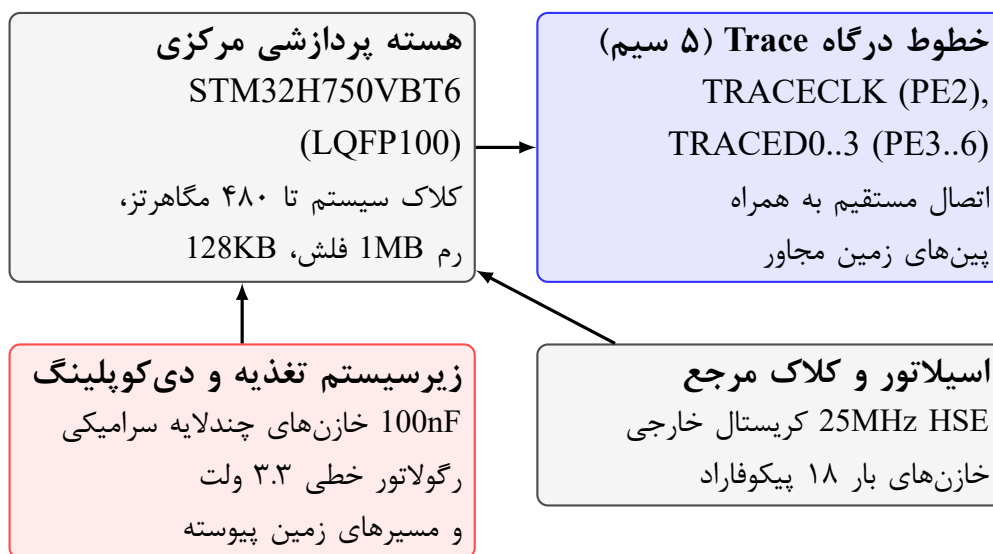
اگرچه این هدربرد دست‌ساز امکان راه‌اندازی اولیه و اجرای نخستین آزمایش‌های کلاک را فراهم ساخت، اما در عمل با چالش‌های فنی شدیدی مواجه شد:

- **نویز شدید بر روی خطوط Trace:** به دلیل فقدان صفحه زمین (Ground Plane) یکپارچه و وجود سلف‌ها و خازن‌های پارازیتی ناشی از سیم‌های معلق، شکل موج کلاک Trace دچار جهش‌های ولتاژی، نوسانات میرای فرکانس بالا (Ringing) و جهش زمین (Ground Bounce) می‌شد که منجر به خطا در ثبت بسته‌ها می‌گردید.

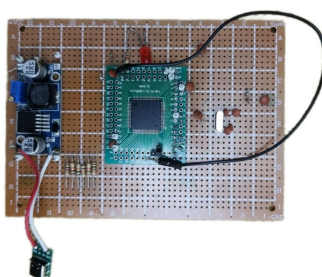
- **مشکلات و بی‌ثباتی در پروگرام کردن میکروکنترلر:** رابط برنامه‌ریزی SWD با نویزهای القایی محیطی تداخل پیدا می‌کرد و پروگرامر ST-Link مکرراً ارتباط خود را با پردازنده از دست می‌داد یا در شناسایی شناسه تراشه با شکست مواجه می‌شد.

- **سختی اتصالات فیزیکی و شکنندگی مکانیکی:** سیم‌کشی‌های متعدد بر روی برد هزارسوراخ در اثر کوچک‌ترین جابجایی یا اتصال پروب‌های تست دچار قطعی، شکستگی یا اتصالی ناخواسته می‌شدند و زمان زیادی صرف عیب‌یابی مداوم اتصالات می‌گردید.

ساختار کلی بلوک‌های سخت‌افزاری مورد نیاز و ارتباطات بخش‌های مختلف هدربرد در شکل ۴-۱ نشان داده شده است. همچنین تصویر هدربرد دست‌ساز اولیه پیاده‌سازی شده بر روی برد هزارسوراخ در شکل ۴-۲ به تصویر کشیده شده است.



شکل ۱.۴: بلوک دیاگرام معماری سخت‌افزار هدربرد و اتصالات درگاه Trace

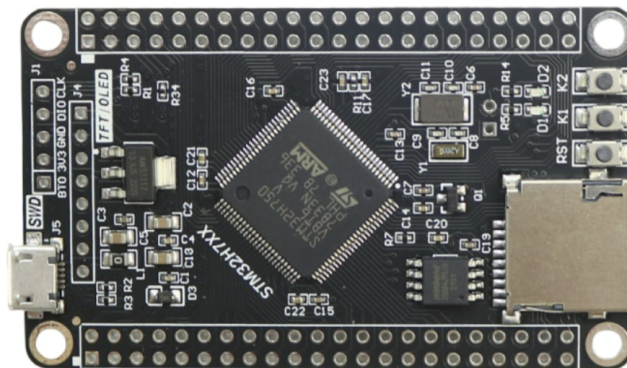


شکل ۲.۴: هدربرد دست‌ساز اولیه پیاده‌سازی شده بر روی برد هزارسوراخ

۳.۴ گذار به هدربرد تجاری STM32H750VBT6 و مشخصات فنی آن

با گذر زمان و پیگیری‌های مستمر، یک هدربرد ساده و استاندارد تجاری از میکروکنترلر STM32H750VBT6 در بازار داخلی موجود شد و بلافاصله خریداری گردید. از آن پس، تمامی فرآیندهای توسعه نرم‌افزاری و سخت‌افزاری بر روی این هدربرد هدایت شد.

ورود هدربرد تجاری، جهش بزرگی در کیفیت و پایداری آزمایش‌ها ایجاد کرد؛ به‌طوری‌که چالش‌های پروگرم کردن تراشه، نویز خطوط Trace، سختی اتصالات و شکنندگی مکانیکی کاملاً برطرف شده و دیگر چالشی در مسیر پروژه محسوب نمی‌شدند. شکل ۳.۴؟ هدربرد تجاری تهیه‌شده را نشان می‌دهد که دارای پکیج ۱۰۰ پایه LQFP و جانمایی استاندارد است.



شکل ۳.۴: هدربرد استاندارد تجاری STM32H750VBT6

۱.۳.۴ تخصیص پایه‌ها و مسیرکشی اختصاصی Trace

پایه‌های ردیابی موازی ۴ بیتی در تراشه STM32H750 بر روی پورت E قرار دارند. جدول ۱.۴؟ مشخصات سیگنال‌ها و جانمایی پایه‌ها را به همراه خط‌کشی کامل نمایش می‌دهد:

جدول ۱.۴: تخصیص پایه‌های فیزیکی میکروکنترلر STM32H750VBT6 به درگاه TPIU

پایه تراشه	شماره پین	سیگنال TPIU	تابع جایگزین	پیکربندی درگاه
PE2	پین ۱	TRACECLK	AF0	Very High Speed (100MHz+)
PE3	پین ۲	TRACED0	AF0	Very High Speed (100MHz+)
PE4	پین ۳	TRACED1	AF0	Very High Speed (100MHz+)
PE5	پین ۴	TRACED2	AF0	Very High Speed (100MHz+)
PE6	پین ۵	TRACED3	AF0	Very High Speed (100MHz+)

یکی از مزایای مهم فیزیکی پکیج LQFP100 در این تراشه، مجاورت پایه‌های ۱ تا ۵ با یکدیگر در گوشه سمت چپ بالای آی‌سی است. این امر باعث می‌شود که خطوط کلاک و داده ردیابی در کمترین فاصله ممکن به پین‌هدرهای لبه برد برسند و تطبیق طول مسیرها با کمترین اختلاف فاز ممکن میسر گردد.

۲.۳.۴ ملاحظات دی‌کوپلینگ و پایداری تغذیه

تغییر وضعیت همزمان ۵ پین دیجیتال در فرکانس بالا با نرخ تغییر ولتاژ حداکثری، جریانی لحظه‌ای به ریل‌های تغذیه وارد می‌کند. در هدربرد تجاری خریداری‌شده، وجود صفحه زمین پیوسته و خازن‌های سرامیکی 100nF چندلایه (MLCC) در نزدیکی پایه‌های VDD/VSS، پدیده جهش زمین را به حداقل رسانده و مسیر برگشت جریان را با کمترین اندوکتانس حلقه مسدود ساخته است.

۴.۴ یکپارچگی سیگنال در کابل‌های رابط و پروب‌ها

برای انتقال سیگنال‌ها از هدربرد به ابزار دریافت لاجیک، اصول یکپارچگی سیگنال مد نظر قرار گرفت:

۱. **طول کابل‌های اتصال:** استفاده از کابل‌های بلند رابط به دلیل ایجاد اندوکتانس پارازیتی موجب نوسانات پله‌ای (Ringing) و خوانش اشتباه سطوح منطقی می‌گردد. بنابراین، طول کابل‌های پروب به کمتر از ۱۰ سانتی‌متر محدود گردید.

۲. **تنظیم ثبات نرخ خیز خروجی (OSPEEDR):** ثبات GPIOE_OSPEEDR برای تمام پایه‌های PE6. PE2 بر روی وضعیت «سرعت بسیار بالا» (0b11 - Very High Speed) تنظیم شد تا زمان خیز و افت پالس‌ها به حداقل رسیده و لبه‌های کلاک به صورت کاملاً شارپ شکل گیرند.

۵.۴ معماری سمپلینگ با فرکانس متناسب (سمپلینگ ۱۰ برابری)

در سامانه‌های صنعتی مجهز به پروب‌های سخت‌افزاری پیشرفته، دیباگر مستقیماً با کلاک سنکرون تراشه در فرکانس‌های ۱۰۰ مگاهرتز به بالا هماهنگ می‌شود. اما در سناریوی استفاده از لاجیک آنالایزرهای عمومی اقتصادی، این رویکرد به دلیل تغییرات فاز کلاک و عدم تطبیق زمان برپایی و نگهداری (Setup & Hold Time) منجر به پرش بیت‌ها می‌گردد. برای دستیابی به اطمینان ۱۰۰ درصدی، استراتژی سمپلینگ ناهمگام با نرخ متناسب ۱۰ برابری اتخاذ گردید.

۱.۵.۴ مشخصات لاجیک آنالایزر DSLogic U3Pro16

برای ثبت سیگنال‌های درگاه ردیابی، از لاجیک آنالایزر پرسرعت DSLogic U3Pro16 ساخت شرکت DreamSourceLab استفاده شد. این دستگاه دارای ۱۶ کانال ورودی دیجیتال، ارتباط پرسرعت USB 3.0 و حافظه داخلی 2Gb DDR3 DRAM است که امکان نمونه‌برداری با فرکانس‌های بالا را بدون افت فریم در بافر داخلی فراهم می‌کند. شکل؟؟ این دستگاه را نمایش می‌دهد.

۲.۵.۴ استراتژی سمپلینگ ۱۰ برابری و استخراج پایدار داده‌ها

برای تضمین دریافت بیست درصدی داده‌ها و حذف کامل خطای فاز:

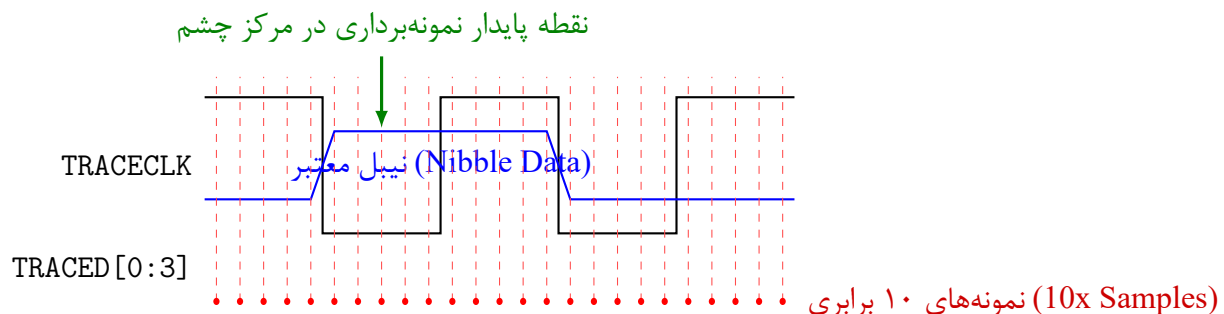
- کلاک پردازنده و کلاک خروجی درگاه ردیابی در یک پله کاری پایدار (مانند ۱۰ مگاهرتز یا کلاک مقیاس‌شده ۱ مگاهرتز) تنظیم می‌گردد.
- لاجیک آنالایزر با نرخ نمونه‌برداری ۱۰ برابری (به عنوان مثال ۱۰۰ مگاهرتز برای کلاک ردیابی ۱۰ مگاهرتز) خطوط TRACECLK و TRACED[0:3] را اسکن می‌نماید.

همان‌طور که در شکل؟؟ مشاهده می‌شود، سمپلینگ با ضریب ۱۰ برابری مزایای بنیادینی به همراه دارد:



شکل ۴.۴: لاجیک‌آنالایزر سخت‌افزاری DSLogic U3Pro16 با رابط پرسرعت USB 3.0 و حافظه داخلی DRAM

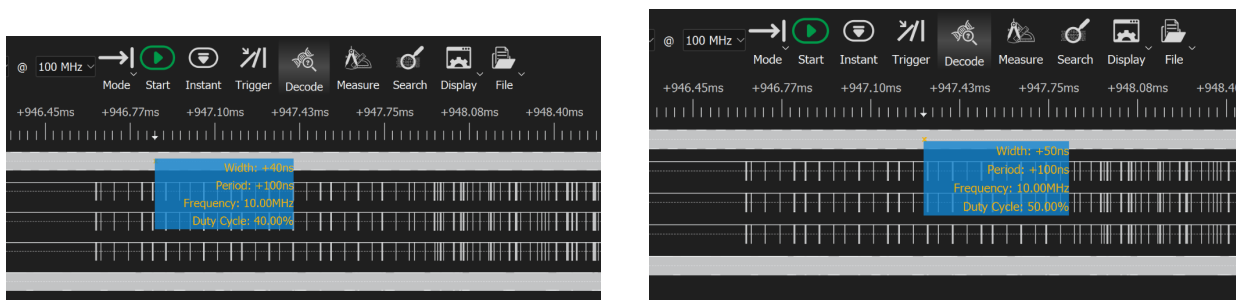
۱. هر نیم‌پریود کلاک شامل حداقل ۵ نمونه دیجیتال است؛ بنابراین نرم‌افزار میزبان لبه‌های بالارونده و پایین‌رونده را بدون تأثیرپذیری از نویزهای ضربه‌ای تک‌نمونه‌ای شناسایی می‌کند.
۲. خطوط داده (TRACED[0:3]) در نمونه میانی (نمونه سوم در پنجره ۵ نمونه‌ای) خوانده می‌شوند؛ یعنی دقیقاً در مرکز چشم‌پالس الکتریکی (Eye Diagram) که سیگنال به بیشترین ثبات و لتازی رسیده و زمان‌های برپایی و نگهداری کاملاً ارضا شده‌اند.



شکل ۵.۴: نمودار زمانی تکنیک سمپلینگ ۱۰ برابری و استخراج پایدار داده‌ها در مرکز چشم الکتریکی

۳.۵.۴ مشاهدات تجربی دیوتی‌سایکل و توجیه تئوری کوانتیزاسیون دیجیتال

در مراحل اولیه آزمایش‌های عملی، پدیده‌ای جالب توجه در ثبت سیگنال کلاک Trace مشاهده شد: هنگامی که یک سیگنال کلاک با فرکانس اسمی ۱۰ مگاهرتز (پریود ۱۰۰ نانوثانیه) با نرخ نمونه‌برداری ۱۰۰ مگاهرتز (۱۰ برابر فرکانس سیگنال) ثبت می‌شد، مقادیر دیوتی‌سایکل اندازه‌گیری‌شده توسط نرم‌افزار لاجیک‌آنالایزر همواره ثابت نبود. در برخی دوره‌ها، دیوتی‌سایکل دقیقاً ۵۰ درصد (50.00%) با عرض پالس مثبت ۵۰ نانوثانیه بود (شکل ۴؟)؛ اما در سایر دوره‌ها یا نمونه‌برداری‌های دیگر، این مقدار به اعدادی نظیر ۴۰ درصد (40.00% در شکل ۴؟)، ۶۰ درصد، ۵۶ درصد یا مقادیر نامتقارن دیگر تغییر می‌کرد.



(آ) نمونه‌برداری با دیوتی‌سایکل متقارن ۵۰ درصد (Width: +50ns)
(ب) نمونه‌برداری با دیوتی‌سایکل نامتقارن ۴۰ درصد (Width: +40ns)

شکل ۴.۴: مشاهدات تجربی اندازه‌گیری کلاک Trace ۱۰ مگاهرتز توسط لاجیک‌آنالایزر در نرخ نمونه‌برداری ۱۰۰ مگاهرتز

با بررسی دقیق فرآیند نمونه‌برداری لاجیک‌آنالایزر و مراجعه به راهنمای کاربری سازنده (Dream-SourceLab DSView User Guide [۴]، مشخص گردید که این نوسان دیوتی‌سایکل ناشی از طبیعت دیجیتال و ناهمگام نمونه‌برداری گسسته در زمان (Asynchronous Discrete-Time Quantization) است و نشان‌دهنده تغییر فیزیکی در سیگنال خروجی میکروکنترلر نیست. دلایل تئوری این رخداد به شرح زیر تبیین می‌گردد:

۱. خطای کوانتیزاسیون زمانی (Timing Quantization Uncertainty): لاجیک‌آنالایزر مجهز

به اسلاتور محلی خود است که مستقل از کلاک میکروکنترلر عمل می‌کند. در نرخ نمونه‌برداری ۱۰۰ مگاهرتز، فاصله زمانی بین دو نمونه متوالی $T_s = 10 \text{ ns}$ است. از آنجا که لبه واقعی پالس کلاک می‌تواند در هر نقطه‌ای میان دو تیک نمونه‌برداری رخ دهد، بر اساس اصل نمونه‌برداری دیجیتال، عدم قطعیت در تشخیص هر لبه معادل $\pm T_s$ یعنی $\pm 10 \text{ ns}$ خواهد بود [۴].

۲. توزیع نمونه‌ها در یک دوره پریودی: در یک دوره تناوب ۱۰۰ نانوثانیه‌ای، دقیقاً ۱۰ نمونه ثبت

می‌شود. اگر فاز کلاک نمونه‌برداری به‌طور متقارن با سیگنال ورودی قفل شده باشد، ۵ نمونه در سطح بالا (High) و ۵ نمونه در سطح پایین (Low) قرار می‌گیرند که دیوتی‌سایکل را ۵۰ درصد نشان می‌دهد ($5 \times 10 \text{ ns} = 50 \text{ ns}$). اما یک جابجایی زمانی ناچیز، لغزش فاز یا جیت به میزان چند نانوثانیه کافی است تا یکی از نمونه‌های نزدیک به لبه تغییر وضعیت داده و تعداد نمونه‌های سطح بالا به ۴ و

سطح پایین به ۶ برسد؛ در نتیجه دیوتی‌سایکل اندازه‌گیری شده برابر ۴۰ درصد ($4 \times 10 \text{ ns} = 40 \text{ ns}$) گزارش می‌شود.

۳. **توجیه نیاز به سمپلینگ ۱۰ برابری بر پایه مستندات سازنده:** در راهنمای رسمی لاجیک‌آنالایزر [؟، بخش ۲.۳.۲ و شکل ۲-۱۰] صراحتاً بیان شده است که با ضریب نمونه‌برداری کمتر (نظیر ۲ یا ۵ برابر)، خطای زمانی به $1/2T$ یا $1/5T$ می‌رسد که می‌تواند به از دست رفتن کامل پالس‌ها (Pulse Loss) یا اعوجاج شدید دیوتی‌سایکل (نظیر نسبت‌های ۳ به ۲ یا ۵ به ۳) منجر شود. در مقابل، سمپلینگ با ضریب ۱۰ برابری خطای فاز را به کمتر از ۱۰ درصد ($\pm 10\%$) محدود کرده و تضمین می‌نماید که تمامی لبه‌ها بدون پرش بیت شناسایی شوند. این تطابق کامل مشاهدات تجربی با روابط تئوری کوانتیزاسیون، مهر تأییدی بر کفایت و ضرورت ضریب ۱۰ برابری در این سامانه بود.

۶.۴ بررسی و دلایل رد درگاه Serial Wire جهت دریافت Trace

در مراحل طراحی معماری سخت‌افزاری سامانه، این پرسش مطرح شد که آیا می‌توان به جای درگاه موازی چندسیمه TPIU، داده‌های ردیابی را بر روی درگاه سریال Serial Wire (SW / SWO) دریافت نمود تا از چالش‌های سیم‌کشی متعدد، ملاحظات فرکانس بالا و نیاز به لاجیک‌آنالایزر چندکاناله رهایی یافت؟ با ارزیابی دقیق معماری پردازنده و الزامات آزمون، این گزینه به دلایل قاطع زیر رد گردید:

۱. پهنای باند بسیار بالای داده‌های Trace دستورات (High Bandwidth of ETM Trace):

ماژول ردیابی دستورات (ETMv4) به ازای هر انشعاب برنامه، استثناً یا تغییر مسیر کنترلی، بسته‌های ردیابی اتم (Atom Packets) و نشانی هدف تولید می‌کند. در پردازنده‌های مدرن فرکانس بالا، حجم این داده‌ها به ده‌ها مگابایت بر ثانیه می‌رسد. خط تک‌سیمه SWO که در مدهای آسنکرون UART یا منچستر کار می‌کند، دارای پهنای باند بسیار محدودی (معمولاً در محدوده چند مگابایت بر ثانیه) است و به سرعت دچار سرریز بافر و افت بسته‌ها (Packet Loss) می‌گردد.

۲. عدم وجود یا محدودیت بافر اختصاصی Trace در تراشه (Lack of Trace Buffer):

از میکروکنترلرهای سری استاندارد (از جمله STM32H750) فاقد بافرهای حجیم آن‌چپ نظیر Embedded Trace Buffer (ETB) یا Embedded Trace FIFO (ETF) هستند تا بتوانند جریان کامل Trace را در حافظه درونی متوقف و سپس به صورت قطره‌چکانی تخلیه کنند. بنابراین، داده‌های Trace باید به صورت کاملاً بی‌درنگ و با کمترین تأخیر به دنیای خارج هدایت شوند که تنها از عهده درگاه موازی پرسرعت برمی‌آید.

۳. اشغال انحصاری درگاه SW توسط پروژه مکمل برای تبادل ورودی/خروجی: حیات‌ترین دلیل

رد این گزینه، وابستگی معماری کلان سامانه آزمون به درگاه SWD بود. در این ساختار مشترک، پورت SW (شامل پایه‌های SWDIO و SWCLK) به‌طور مداوم توسط پروژه مکمل جهت تزریق تحریک‌های تست (Test Stimuli) به داخل ثبات‌ها و خواندن مقادیر ورودی/خروجی و پاسخ‌های سخت‌افزار تحت آزمون استفاده می‌شد. اشغال این درگاه برای داده‌های حجیم ردیابی، به مسدود شدن کانال آزمون و بن‌بست ارتباطی منجر می‌گردید.

۴. فقدان کلاک فیزیکی سنکرون و ناسازگاری با استاندارد Trace دستورات: پایه SWO فاقد کلاک سنکرون فیزیکی (TRACECLK) است و در سطح سخت‌افزار تنها برای ارسال بسته‌های رویدادی ماژول‌های سبک‌تر نظیر ITM (Instrumentation Trace) و DWT طراحی شده است؛ در حالی که ردیابی چرخه‌دقیق دستورات نیازمند گذرگاه همگام چندبیتی TPIU با تضمین هم‌زمانی کلاک و داده است.

بنابراین، پیاده‌سازی سخت‌افزاری مبتنی بر درگاه ۴ بیتی موازی TPIU به همراه کلاک اختصاصی، تنها رویکرد ممکن و استاندارد برای دستیابی به ردیابی کامل و بی‌درنگ دستورات بدون تداخل با فرآیند آزمون نرم‌افزار ارزیابی گردید.

۷.۴ جمع‌بندی فصل

در این فصل، مسیر تکامل بستر سخت‌افزاری سامانه از هدربرد دست‌ساز اولیه ساخته‌شده بر روی برد هزارسوراخ تا گذار موفق به هدربرد استاندارد تجاری STM32H750VBT6 تبیین گردید. تمهیدات یکپارچگی سیگنال شامل کوتاه نگاه داشتن طول پروب‌ها و تنظیم خروجی‌های سرعت بسیار بالا شرح داده شد. سپس معماری سمپلینگ ۱۰ برابری با استفاده از لاجیک‌آنالایزر DSLogic U3Pro16 معرفی شد و پدیده تجربی تغییر دیوتی‌سایکل بر پایه تئوری کوانتیزاسیون دیجیتال ناهمگام و استانداردهای سازنده لاجیک‌آنالایزر توجیه گردید. در نهایت، دلایل فنی و عملیاتی رد درگاه Serial Wire به نفع درگاه موازی TPIU بررسی شد. در فصل پنجم، پیکربندی سطح پایین ثبات‌های درونی میکروکنترلر و Firmware مورد نیاز جهت فعال‌سازی این بستر سخت‌افزاری تشریح خواهد شد.

فصل ۵

پیکربندی ETMv4 و پیاده‌سازی Firmware

۱.۵ مقدمه

زیرسیستم ردیابی و عیب‌یابی CoreSight در میکروکنترلرهای پیشرفته مبتنی بر معماری ARM Cortex-M7 (نظیر STM32H750)، مجموعه‌ای بسیار غنی از ماژول‌های مستقل سیلیکونی است که در فضای نگاشت‌شده حافظه (Memory-Mapped I/O) قرار دارند. این ماژول‌ها پس از روشن شدن اولیه تراشه، به منظور جلوگیری از مصرف بیهوده توان و ممانعت از ایجاد تداخل‌های ناخواسته بر خطوط کلاک و پین‌های خروجی، در وضعیت کاملاً قفل‌شده و خاموش قرار دارند. راه‌اندازی این زیرسیستم بدون نقض شرط بنیادین «غیرتهاجمی بودن» (Non-Intrusiveness) و بدون اعمال هرگونه اثر جانبی بر فرکانس و رفتار زمانی برنامه‌های کاربر، نیازمند مهندسی دقیق ترتیب پیکربندی ثبات‌ها، مدیریت دامنه‌های توان و کلاک، و حل چالش‌های پنهان تطبیق معماری است.

در این فصل، ابتدا پروتکل فشرده‌سازی جریان دستورات در ETMv4 تبیین می‌گردد. سپس ساختار معماری زیرساخت دیباگ، دامنه‌های توان و توزیع کلاک در STM32H750 به همراه بلوک‌دیagramهای مربوطه از راهنمای مرجع شرکت سازنده تشریح می‌شود. در ادامه، نگاشت حافظه و مراحل گام‌به‌گام پیکربندی ثباتی با تأکید بر ثبات حیاتی TRCCONFIGR بررسی شده و چالش بنیادین مغایرت راهنمای مرجع آی‌سی‌ها با استاندارد معماری ARM تبیین می‌گردد. در بخش‌های بعدی، کالبدشکافی فایل‌های درایور مستقل ETMv4.h و ETMv4.c و روش به‌کارگیری محصول و تزریق به کدهای کاربری ارائه شده و در نهایت، ریشه‌یابی چالش بسته‌های همگام‌سازی (A-Sync) و آسیب‌شناسی روش سوئیچینگ متناوب مستندسازی خواهد شد.

۲.۵ پروتکل فشرده‌سازی جریان دستورات در ETMv4

پردازنده Cortex-M7 در فرکانس‌های کاری چندصد مگاهرتزی، در هر ثانیه صدها میلیون دستورالعمل را اجرا می‌کند. ارسال بی‌درنگ و کامل آدرس ۳۲ بیتی هر دستورالعمل از طریق پایه‌های فیزیکی میکروکنترلر به پهنای باندی در مقیاس چند گیگابایت بر ثانیه نیاز دارد؛ در حالی که در عمل تنها یک گذرگاه موازی

بسیار محدود (شامل یک خط کلاک و ۴ خط داده) در دسترس قرار دارد. برای غلبه بر این محدودیت فیزیکی، استاندارد معماری ETMV4 شرکت ARM از پروتکل فشرده‌سازی پیشرفته‌ای بر مبنای تفکیک رفتارهای خطی و غیرخطی پردازنده بهره می‌برد [۹]:

۱. **ردیابی خطی و حذف آدرس‌ها:** تا زمانی که برنامه به صورت ترتیبی و بدون انشعاب دستورات متوالی حافظه را اجرا می‌کند، هیچ داده‌ای روی خط ردیابی ارسال نمی‌شود. دلیل این امر آن است که موتور دکودر نرم‌افزاری بر روی کامپیوتر میزبان، نسخه کامپایل شده باینری برنامه (.elf) را در اختیار دارد؛ بنابراین با در دست داشتن آدرس اولیه، می‌تواند با دیس‌اسمبل کردن کد، نشانی دستورهای متوالی بعدی را خودش به صورت آفلاین محاسبه نماید.

۲. **بسته‌های انشعاب شرطی (Atom Packets):** هنگامی که پردازنده به یک انشعاب شرطی (مانند دستورات BEQ، BNE یا عبارات if-else) می‌رسد، ماژول ETM تنها یک بیت وضعیت تولید می‌کند:

- کد E (Executed / Taken): نشان‌دهنده تحقق شرط و پرش به نشانی مقصد انشعاب.
- کد N (Not-Executed / Not-Taken): نشان‌دهنده عدم تحقق شرط و ادامه مسیر خطی برنامه.

سخت‌افزار ETM جهت پیشینه‌سازی فشرده‌سازی، چندین اتم متوالی را در قالب یک بایت تجمیع کرده و به صورت بسته‌های تک‌بایتی به خروجی می‌فرستد.

۳. **بسته‌های همگام‌سازی (A-Sync و I_SYNC):** به منظور بازیابی نقطه شروع اجرای برنامه یا هدایت دکودر در هنگام پرش‌های غیرمستقیم (نظیر فراخوانی توابع از طریق اشاره‌گرها، بازگشت از توابع یا ورود به توابع سرویس وقفه)، بسته‌های همگام‌سازی اجباری حاوی نشانی ۳۲ بیتی کامل ثبات PC، اطلاعات کانتکست و وضعیت کاری پردازنده ارسال می‌شوند.

۴. **برچسب‌های زمانی سخت‌افزاری (Timestamping):** برای ارزیابی دقیق زمان سپری شده و همبسته‌سازی زمانی رویدادها، ماژول ETM قادر است بسته‌های برچسب زمانی ۶۴ بیتی سراسری تولید کند.

۵. **شمارش چرخه‌های پردازنده (Cycle Counting - CCI):** در کاربردهای تخمین بدترین زمان اجرا (WCET)، ثبت دقیق سیکل‌های سپری شده پردازنده بین دستورات حیاتی است. قابلیت CCI با درج دلتای سیکل‌ها در جریان دستورات پایه، اندازه‌گیری چرخه-دقیق را بدون هیچ‌گونه سربار نرم‌افزاری فراهم می‌سازد.

۳.۵ معماری زیرساخت دیباگ، دامنه‌های توان و توزیع کلاک در STM32H750

میکروکنترلر STM32H750 دارای زیرساخت عیب‌یابی و ردیابی بسیار پیچیده‌ای است که در راهنمای مرجع RM0433 شرکت STMicroelectronics به تفصیل مستند شده است [۹]. درک ساختار ارتباطی

اجزای این زیرسیستم، تفکیک دامنه‌های توان و مدیریت کلاک‌های آنها پیش‌نیاز قطعی پیکربندی موفق Firmware محسوب می‌شود.

۱.۳.۵ بلوک‌دیگرام زیرساخت دیباگ و مسیر سیگنال‌های ردیابی

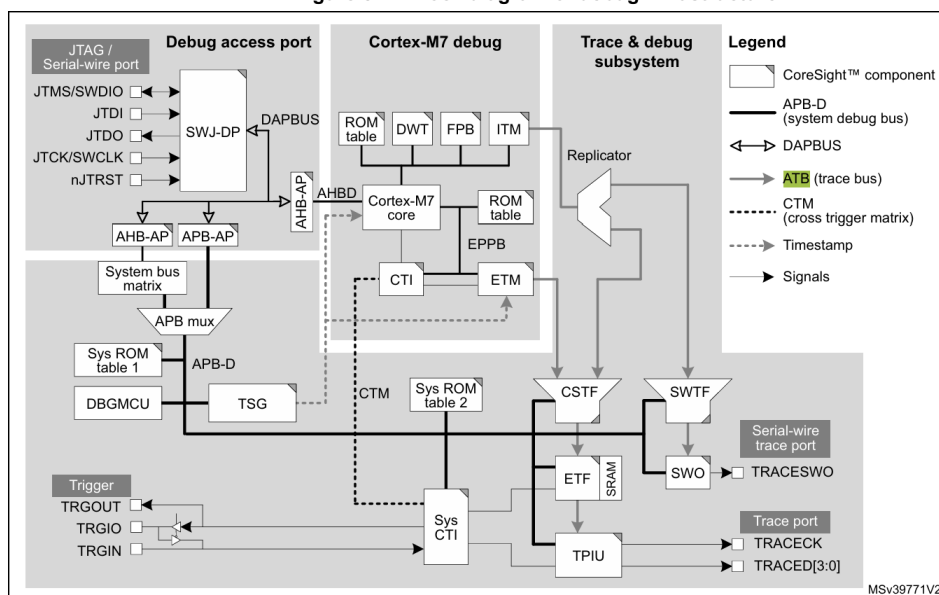
شکل ۹۹؟ (برگرفته از شکل ۸۲۷ راهنمای مرجع RM0433)، معماری اتصالات داخلی اجزای دیباگ را نمایش می‌دهد.

60.3 Debug infrastructure functional description

60.3.1 Debug infrastructure block diagram

The block diagram shows the logical partitioning of the debug infrastructure.

Figure 827. Block diagram of debug infrastructure



شکل ۱.۵: بلوک‌دیگرام زیرساخت دیباگ و ردیابی در میکروکنترلر STM32H750 (شکل ۸۲۷ راهنمای مرجع RM0433)

همان‌گونه که در شکل ۹۹؟ مشاهده می‌شود:

- هسته پردازنده Cortex-M7 دربرگیرنده بلوک‌های ETMV4، DWT و ITM است. ماژول ETMV4 پایش مستقیم گذرگاه دستورات پردازنده را بر عهده دارد.
- داده‌های ردیابی تولیدشده توسط ETM و ITM بر روی گذرگاه داخلی ۳۲ بیتی ATB (Advanced Trace Bus) به سمت قیف ادغام ردیابی CSTF (CoreSight Trace Funnel) هدایت می‌شوند.
- خروجی CSTF وارد بافر سخت‌افزاری ETF (Embedded Trace FIFO) می‌گردد. این بافر حافظه واسط، نوسانات شدید نرخ تولید بایت‌های Trace در جهش‌های ناگهانی برنامه را مهار کرده و از سرریز داده‌ها جلوگیری می‌کند.

- در نهایت، جریان داده‌ها از خروجی ETF وارد واحد فرمت‌بندی پورت ردیابی TPIU (Trace Port Interface Unit) شده و این واحد، بایت‌ها را در قالب فریم‌های منظم سازمان‌دهی کرده و روی پورت موازی ۴ بیتی خارجی منتشر می‌سازد.

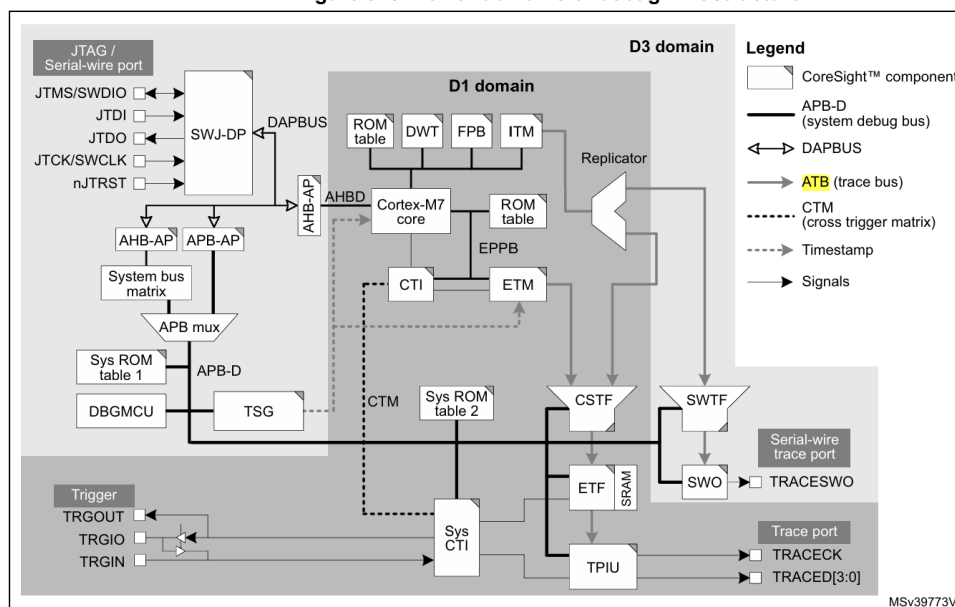
۲.۳.۵ دامنه‌های توان زیرساخت دیباگ (Power Domains)

یکی از مهم‌ترین دلایل شکست تلاش‌های اولیه در راه‌اندازی ردیابی، عدم آگاهی از تفکیک دامنه‌های توان این ماژول‌ها در سیلیکون است. شکل؟؟ (برگرفته از شکل ۸۲۸ راهنمای مرجع RM0433) دامنه‌های توان ماژول‌های دیباگ را تبیین می‌کند.

60.3.3 Debug infrastructure powering, clocking and reset

Power domains

Figure 828. Power domains of debug infrastructure



شکل ۲.۵: دامنه‌های توان زیرساخت دیباگ در میکروکنترلر STM32H750 (شکل ۸۲۸ راهنمای مرجع RM0433)

اجزای زیرسیستم ردیابی در دو دامنه توان متمایز قرار گرفته‌اند:

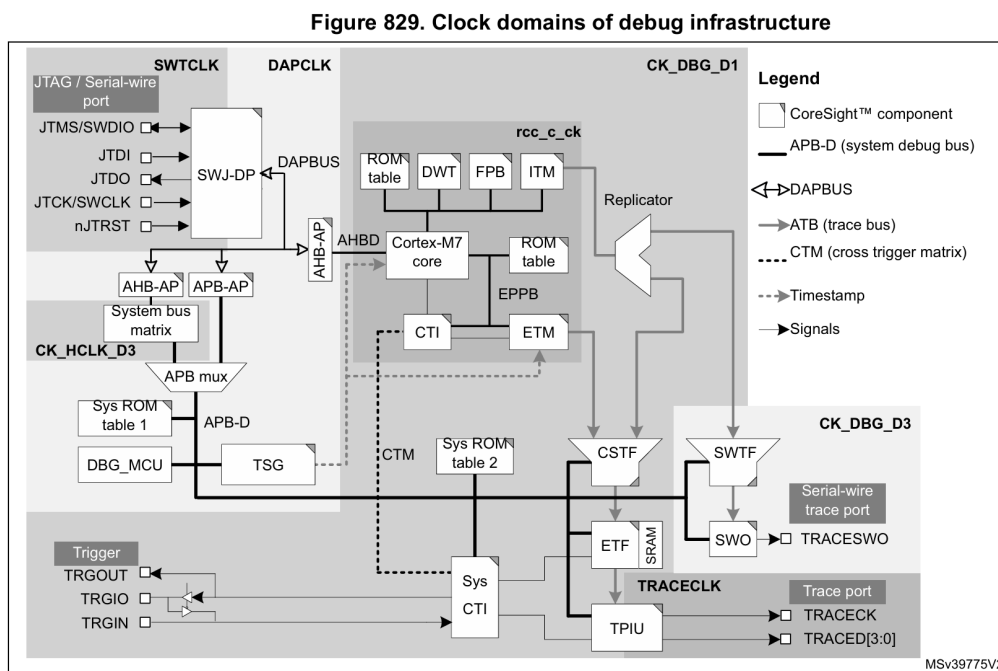
۱. **دامنه توان هسته (D1 Power Domain):** هسته پردازنده Cortex-M7 و ماژول‌های محلی آن شامل ETMV4، DWT و ITM در دامنه D1 قرار دارند. اگر هسته به مدهای کم‌توان (Sleep / Deep-Sleep) وارد شود، برق این بخش قطع یا توان آن کاهش می‌یابد.

۲. **دامنه توان سیستم (D3 Power Domain):** ماژول‌های زیرساختی شامل CSTF، ETF، TPIU، DBGMCU و درگاه SWO در دامنه D3 قرار گرفته‌اند.

بنابراین، Firmware باید تضمین کند که هر دو دامنه D1 و D3 در وضعیت فعال کامل نگه داشته شوند و هیچ‌یک از اجزای مسیر ردیابی در حالت تعلیق توان قرار نگیرند.

۳.۳.۵ دامنه‌های کلاک زیرساخت دیباگ و اقدامات نرم‌افزاری

شکل ۴۴؟ (برگرفته از شکل ۸۲۹ راهنمای مرجع RM0433) مسیرهای توزیع کلاک را در زیرسیستم ردیابی نمایش می‌دهد.



شکل ۳.۵: دامنه‌های کلاک زیرساخت دیباگ در میکروکنترلر STM32H750 (شکل ۸۲۹ راهنمای مرجع RM0433)

در این ساختار، سه شبکه کلاک اساسی وجود دارد:

- **کلاک هسته (CPU_CLK):** کلاک اصلی پردازنده که ماژول‌های ETMV4 و DWT مستقیماً با آن کار می‌کنند. این موضوع تضمین می‌کند که ردیابی رویدادها دقیقاً با سرعت اجرای دستورات انجام می‌پذیرد.

- **کلاک خروجی ردیابی (TRACECLK / TRACECK):** کلاکی که واحد TPIU از آن برای کلاک کردن پین خروجی TRACECLK (پایه PE2) و تغییر وضعیت پایه‌های داده ردیابی استفاده می‌کند.

- **کلاک‌های گذرگاه دیباگ (DBG_CKD1 و DBG_CKD3):** کلاک‌های فعال‌کننده دسترسی ثباتی به فضای CoreSight در دامنه‌های D1 و D3.

اقدامات انجام‌شده در Firmware جهت روشن کردن دامنه‌های کلاک و توان: برای فعال‌سازی کامل این مسیرها، کدهای راهنما باید مراحل زیر را در ثبات‌های سیستمی اعمال نمایند:

```
1 /* 1. Enable GPIOE peripheral clock for trace output pins */
2 RCC->AHB4ENR |= RCC_AHB4ENR_GPIOEEN;
```

```

3
4 /* 2. Enable trace and debug monitor in CoreDebug */
5 CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; // Bit 24 (TRCENA = 1)
6
7 /* 3. Enable Trace clock, D1 debug clock, and D3 debug clock in DBGMCU */
8 DBGMCU->CR |= DBGMCU_CR_DBG_TRACECKEN |
9               DBGMCU_CR_DBG_CKD1EN      |
10              DBGMCU_CR_DBG_CKD3EN;

```

Listing 5.1: RCC و DBGMCU کد راه‌اندازی دامنه‌های توان و کلاک ردیابی در

بدون ست کردن همزمان بیت‌های فوق در ثبات DBGMCU_CR، گذرگاه‌های ATB و پورت TPIU کلاک دریافت نکرده و کل سخت‌افزار ردیابی در وضعیت خاموش باقی خواهد ماند.

۴.۵ نگاشت حافظه ثبات‌های CoreSight در STM32H750

در میکروکنترلر STM32H750، هر یک از اجزای زیرسیستم ردیابی دارای یک بلوک آدرس ۴ کیلوبایتی در فضای دسترسی خصوصی پردازنده (Private Peripheral Bus - PPB) یا گذرگاه AHB هستند. جدول ۱.۵ آدرس‌های پایه، دامنه‌های کاری و نقش هر مؤلفه را خلاصه نموده است [۹]:

جدول ۱.۵: آدرس‌های پایه ماژول‌های CoreSight در حافظه نگاشت‌شده STM32H750

نام ماژول	آدرس پایه فیزیکی	دامنه	وظیفه معماری
ETMv4	0xE0041000	PPB / D1	موتور اصلی پایش دستورات، فشردن سازی شاخه‌ها و برچسب زمانی
DWT	0xE0001000	PPB / D1	مقایسه‌گرهای آدرس، تریگرهای شروع/پایان و شمارنده چرخه
DBGMCU	0x5C001000	APB4 / D3	کنترل کلاک‌های دیباگ، مدهای توان و فعال‌سازی Trace
TSG	0x5C005000	AHB / D3	تولیدکننده برچسب زمانی ۶۴ بیتی سراسری سیستم
CSTF	0x5C013000	AHB / D3	قیف ادغام داده‌های Trace و تخصیص شناسه جریان (Source ID)
ETF	0x5C014000	AHB / D3	بافر سخت‌افزاری FIFO جهت جذب نوسانات ترافیک داده
TPIU	0x5C015000	AHB / D3	فرمت‌بندی پورت موازی خارجی، تنظیم پهنای باند و فریم‌ها

۵.۵ مراحل گام‌به‌گام راه‌اندازی ثباتی

۱.۵.۵ گام اول: آزادسازی قفل دسترسی نرم‌افزاری (Software Lock Removal)

مطابق با استاندارد معماری ARM CoreSight، کلیه ثبات‌های کنترلی و پیکربندی ماژول‌های ردیابی توسط مکانیزم قفل دسترسی نرم‌افزاری محافظت می‌شوند. تا پیش از باز شدن این قفل، هرگونه عملیات نوشتن در این ثبات‌ها توسط سخت‌افزار نادیده گرفته شده و وضعیت مدار تغییر نمی‌کند. برای باز کردن قفل هر ماژول، باید «کلید Lock Access Register» (مقدار هگزادسیمال 0xC5ACCE55) در ثبات LAR (Lock Access Register) در آفست 0xFB0 ماژول مربوطه نوشته شود. پس از نوشتن این کلید، بیت ۱ در ثبات وضعیت قفل (LSR - Lock Status Register) در آفست 0xFB4 پاک شده و مقدار صفر (نشان‌دهنده باز بودن قفل) را گزارش می‌دهد:

```

1 #define CORESIGHT_UNLOCK_KEY  0xC5ACCE55UL
2 #define CORESIGHT_LAR_OFFSET  0xFB0UL
3 #define CORESIGHT_LSR_OFFSET  0xFB4UL
4
5 void CoreSight_Unlock_Component(uint32_t base_addr) {
6     /* Write LAR unlock key to permit register write access */
7     *((volatile uint32_t *) (base_addr + CORESIGHT_LAR_OFFSET)) =
8     CORESIGHT_UNLOCK_KEY;
9     __DSB(); // Ensure memory write completion
10
11     /* Optional: poll LSR bit 1 until lock status indicates unlocked (0) */
12     while (*((volatile uint32_t *) (base_addr + CORESIGHT_LSR_OFFSET)) & (1UL
    << 1));
13 }
```

Listing 5.2: LAR با استفاده از کلید CoreSight باز کردن قفل دسترسی نرم‌افزاری ماژول‌های

۲.۵.۵ گام دوم: پیکربندی پین‌های خروجی ردیابی (GPIOE Configuration)

پورت موازی ردیابی در میکروکنترلر STM32H750 بر روی پین‌های درگاه GPIOE نگاشت شده است. این پایه‌ها باید روی حالت عملکرد دوم (Alternate Function 0 - AF0) با بالاترین سرعت سوئیچینگ خروجی (Very High Speed) تنظیم شوند:

- پایه PE2: سیگنال کلاک خروجی Trace (TRACECLK)
- پایه‌های PE3 تا PE6: خطوط ۴ بیتی داده ردیابی موازی (TRACED0 تا TRACED3)

۳.۵.۵ گام سوم: پیکربندی خط لوله ردیابی (ETP, CSTF و TPIU)

مسیر عبور داده‌ها پس از تولید در هسته باید از میان ماژول‌های واسط هدایت گردد:

۱. پیکربندی کیف ردیابی (CSTF): در ثبات CTRL ماژول CSTF، پورت‌های متصل به هسته فعال شده و حداقل زمان نگهداری سوئیچینگ (Minimum Hold Time) تنظیم می‌شود:

```
1 CSTF->LAR = CORESIGHT_UNLOCK_KEY;
2 CSTF->CTRL = (0x3U << 8) | 0x1U; // Enable slave port with hold time
3 = 3
```

۲. پیکربندی بافر سخت‌افزاری ETF: ماژول ETF در مد Hardware FIFO (MODE = 0x02) پیکربندی می‌شود تا داده‌های ورودی از ATB را مستقیماً در بافر رم داخلی قرار داده و با فعال بودن قالب‌بندی (FFCR = 0x01)، به سمت TPIU پمپاژ نماید.

۳. پیکربندی واحد پورت خارجی TPIU: اندازه پورت خروجی در ثبات CURPSIZE بر روی مقدار ۸ (معادل پورت ۴ بیتی) تنظیم گشته و ثبات FFCR با مقدار 0x00000002 پر می‌شود که نشان‌دهنده فعال‌سازی حالت قالب‌بندی پیوسته (Continuous Formatting Mode) است.

۴.۵.۵ گام چهارم: پیکربندی موتور ETMv4 و تشریح دقیق ثبات TRCCONFIGR

کلیدی‌ترین و سرنوشت‌سازترین ثبات در کل فرآیند راه‌اندازی سخت‌افزار، ثبات پیکربندی اصلی ردیابی ETM موسوم به TRCCONFIGR (یا M7_ETM_CONFIG) در آفست 0x010 است. این ثبات به طور مستقیم قابلیت‌ها و ماهیت داده‌های خروجی ماژول ETM را معین می‌سازد. شکل ۴۴ ساختار بیت‌های این ثبات را مطابق با راهنمای مرجع RM0433 نمایش می‌دهد.

RM0433

Debug infrastructure

ETM trace configuration register (M7_ETM_CONFIG)

Address offset: 0x010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DV	DA
														rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	RS	TS	COND[2:0]			Res.	Res.	Res.	CCI	BB	INSTP0[1:0]		Res.
			rw	rw	rw	rw	rw				rw	rw	r	r	

Bits 31:18 Reserved, must be kept at reset value.

Bit 17 **DV**: Data value tracing (read-only)

0: Disabled

Bit 16 **DA**: Data address tracing (read-only)

0: Disabled

Bits 15:13 Reserved, must be kept at reset value.

Bit 12 **RS**: Return stack enable

0: Disabled

1: Enabled

Bit 11 **TS**: Global timestamp tracing

0: Disabled

1: Enabled

Bits 10:8 **COND[2:0]**: Conditional instruction tracing

0x0: Conditional instruction tracing disabled

0x1: Conditional load instructions are traced

0x2: Conditional store instructions are traced

0x3: Conditional load and store instructions are traced

0x7: All conditional instructions are traced

Other: Reserved

Bits 7:5 Reserved, must be kept at reset value.

Bit 4 **CCI**: Cycle counting in instruction trace

0: Disabled

1: Enabled

Bit 3 **BB**: Branch broadcast mode

0: Disabled

1: Enabled

Bits 2:1 **INSTP0[1:0]**: Determines which instructions are P0 instructions (read-only)

0x0: Only branches are P0 instructions

Bit 0 Reserved, must be kept at reset value.



شکل ۴.۵: ساختار بیت‌های ثبات پیکربندی ردیابی M7_ETM_CONFIG برگرفته از راهنمای مرجع RM0433

بررسی دقیق بیت‌های این ثبات، مرز میان قابلیت‌های پیاده‌سازی شده در سیلیکون STM32H750 و مشخصات کلی استاندارد ARM را آشکار می‌سازد:

- **بیت ۱۷ (DV - Data Value Tracing):** وضعیت فقط خواندنی (Read-Only). در این پیاده‌سازی سخت‌افزاری، ردیابی مقادیر متغیرهای حافظه پشتیبانی نمی‌شود و همواره مقدار صفر دارد.
 - **بیت ۱۶ (DA - Data Address Tracing):** وضعیت فقط خواندنی (Read-Only). ردیابی نشانی‌های داده پشتیبانی نمی‌شود و مقدار آن صفر است.
 - **بیت ۱۲ (RS - Return Stack Enable):** وضعیت خواندن/نوشتن (RW). فعال کردن این بیت به ماژول ETM اجازه می‌دهد پشته بازگشت توابع را در سخت‌افزار مدل‌سازی کند که منجر به کاهش فوق‌العاده بسته‌های انشعاب در هنگام بازگشت از زیربرنامه‌ها می‌شود.
 - **بیت ۱۱ (TS - Global Timestamp Tracing):** وضعیت خواندن/نوشتن (RW). با فعال‌سازی این بیت، ماژول ETM بسته‌های برچسب زمانی ۶۴ بیتی دریافت شده از مولد TSG را در جریان داده‌ها منتشر می‌سازد که جهت اندازه‌گیری دقیق زمان دیوار ساعت (Wall-Clock Time) ضروری است.
 - **بیت‌های ۱۰ تا ۸ (COND[2:0] - Conditional Instruction Tracing):** تعیین‌کننده نحوه پایش دستورات شرطی. مقدار 0b000 پایش دستورات شرطی را غیرفعال کرده و مقدار 0b111 تمامی دستورات شرطی را پایش می‌کند.
 - **بیت ۴ (CCI - Cycle Counting in Instruction Trace):** حیاتی‌ترین بیت جهت سنجش WCET. فعال کردن این بیت سبب می‌شود ماژول ETM دلتای تعداد چرخه‌های کلاک سپری‌شده هسته پردازنده را در جریان دستورات پایه (P0) درج نماید. بدین ترتیب دکودر می‌تواند طول مدت اجرای هر بلوک را بر حسب چرخه‌های دقیق کلاک استخراج کند.
 - **بیت ۳ (BB - Branch Broadcast Mode):** در صورت فعال بودن، آدرس مقصد تمامی شاخه‌ها حتی شاخه‌های مستقیم را منتشر می‌سازد.
 - **بیت‌های ۲ و ۱ (INSTP0[1:0]):** وضعیت فقط خواندنی. مشخص‌کننده کلاس دستورات پایه (P0) است که در این میکروکنترلر به شاخه‌ها محدود می‌باشد.
- در پروژه حاضر، مقداردهی این ثبات به صورت $ETM \rightarrow CONFIG = (1U \ll 4) \mid (1U \ll 11)$ ؛ انجام گرفت که به طور همزمان قابلیت شمارش دقیق چرخه (CCI) و برچسب زمانی سراسری (TS) را برای سنجش چرخه‌ای WCET فعال می‌سازد.

۵.۵.۵ چالش مغایرت راهنمای مرجع آی‌سی‌ها با مشخصات معماری ARM ETM

یکی از مهم‌ترین یافته‌های تجربی و در عین حال اصلی‌ترین دلایل طولانی شدن فرآیند تحقیق و توسعه و پیکربندی اولیه سخت‌افزار در این پروژه، شکاف عمیق و مغایرت ساختاری میان اسناد مرجع معماری شرکت ARM و راهنمای مرجع سازندگان تراشه (STMicroelectronics) بود.

در ابتدای فاز پیاده‌سازی، رویکرد متداول مهندسی ایجاب می‌کند که از دستورالعمل رسمی شرکت ARM در کتاب استاندارد معماری ETMV4 (ARM IHI 0064D) استفاده شود. شکل ۴-۱۵ جدول معروف Table 4-15 این سند استاندارد را نشان می‌دهد که مراحل و مقادیر پیشنهادی ARM را برای راه‌اندازی اولیه ردیابی دستورات مشخص کرده است [۴].

4.5.1 Enabling the trace unit for program flow trace

To enable the trace unit for basic program flow trace:

1. Program the access control registers so that the trace unit can be accessed and controlled. This might include unlocking the OS Lock and unlocking the Software Lock, if using the memory-mapped interface.
2. Set `TRCPRGCTLR.EN = 0`, to disable the trace unit.
3. Program the registers and values that Table 4-15 shows. The registers are in alphabetical order but any order is permitted.

Table 4-15 Enabling the trace unit for basic program flow trace

Register	Value	Description
TRCONFIGR	0x000018C1	Enable the return stack, global timestamping, Context identifier, and Virtual context identifier tracing.
TRCEVENTCTL0R	0x00000000	Disable all event tracing.
TRCEVENTCTL1R	0x00000000	
TRCSTALLCTLR	0x00000000	Disable stalling, if implemented.
TRCSYNCPR	0x0000000C	Enable trace synchronization every 4096 bytes of trace.
TRCTRACEIDR	Non-zero	Set a value for the trace ID.
TRCTSCTLR	0x00000000	Disable the timestamp event. The trace unit still generates timestamps due to other reasons such as trace synchronization.
TRCVICTLR	0x00000201	Enable ViewInst to trace everything, with the Start/Stop logic started.
TRCVIIECTLR	0x00000000	No address range filtering for ViewInst.
TRCVISSCTLR	0x00000000	No start or stop points for ViewInst.

4. Set `TRCPRGCTLR.EN=1`, to enable the trace unit.

شکل ۵-۵: دستورالعمل پیشنهادی فعال‌سازی ردیابی در استاندارد مرجع معماری ARM ETMV4 (جدول ۴-۱۵ سند ARM IHI 0064)

با این حال، پیاده‌سازی عینی و تلاش برای اجرای مستقیم مقادیر جدول ۴-۱۵ بر روی میکروکنترلر STM32H750 با شکست کامل مواجه شد. کالبدشکافی راهنمای مرجع RM0433 نشان داد که نمونه‌های ارائه‌شده در استاندارد ARM بر روی پردازنده‌های واقعی قابل اجرای کورکورانه نیستند و هر ثبت نیازمند تطبیق موشکافانه با جزئیات پیاده‌سازی سیلیکون است:

۱. **ثبت‌های TRCVISSCTLR و TRCVIIECTLR:** در جدول ۴-۱۵ شرکت ARM، صریحاً دستور داده شده که مقادیر این دو ثبت صفر شوند تا فیلترینگ محدوده آدرس غیرفعال شود. اما در میکروکنترلر STM32H750، ثبت TRCVIIECTLR اصلاً در سیلیکون پیاده‌سازی نشده و در فضای نگاشت حافظه وجود خارجی ندارد! همچنین ثبت TRCVISSCTLR بازطراحی شده و کنترل شروع/توقف به ثبت‌های متفاوتی نظیر VPCSSCTL و VICTL سپرده شده است. تلاش برای نوشتن در این آدرس‌ها موجب خطای دسترسی یا نادیده گرفته شدن فرمان می‌شود.

۲. **ثبت دوره همگام‌سازی (TRCSYNCPR):** در جدول پیشنهادی ARM، نوشتن مقدار 0x0000000C

(صدور همگام‌سازی هر ۴۰۹۶ بایت) توصیه شده است. اما در STM32H750، این ثبات از دید نرم‌افزار کاملاً فقط‌خواندنی (Read-Only) است و مقدار آن روی هگزادسیمال 0x0A ثابت نگه داشته شده است؛ بنابراین هرگونه تلاش نرم‌افزاری جهت نوشتن در این ثبات بی‌اثر است.

۳. مقداردهی ثبات پیکربندی (TRCCONFIGR): شرکت ARM در این جدول نوشتن مقدار 0x000018C1 را پیشنهاد می‌دهد که بیت‌های ردیابی Context ID و Virtual Context ID را فعال می‌سازد. این بیت‌ها مختص پردازنده‌های رده کاربردی (Cortex-A) با واحد مدیریت حافظه پیشرفته (MMU) و هایپروایزر هستند. در پردازنده مایکروکنترلری Cortex-M7 این بیت‌ها یا رزرو هستند یا پیاده‌سازی نشده‌اند و نوشتن چنین مقداری وضعیت ثبات را غیرقابل پیش‌بینی (UNPREDICTABLE) می‌سازد.

این مغایرت‌های بنیادین اثبات نمود که پیکربندی CoreSight نیازمند مهندسی معکوس دقیق و اعتبارسنجی هر آفست حافظه بر اساس راهنمای مرجع اختصاصی تراشه (RM0433) است و نمونه کدهای عمومی قابلیت اطمینان ندارند.

۶.۵ تشریح فایل‌های راهکار نرم‌افزاری: پیاده‌سازی ETMv4.h و ETMv4.c

یکی از نکات حائز اهمیت در سطح معماری نرم‌افزار این است که نه کتابخانه استاندارد هسته CMSIS (core_cm7.h) و نه درایورهای شرکت سازنده (STM32Cube HAL)، هیچ‌گونه استراکت داده یا توابع رابطی برای ثبات‌های ETMv4 در پردازنده Cortex-M7 فراهم نکرده‌اند! شرکت ST تنها ماژول‌های پایه‌ای DBGMCU و درگاه تک‌سیمه SWO/ITM را تعریف نموده است. از این رو، کلیه ساختارهای داده نگاشت حافظه، ماسک‌های بیتی و توابع راه‌اندازی توسط نگارنده با مطالعه سطر به سطر اسناد RM0433 Rev 8 و ARM CoreSight v2.0 طراحی و پیاده‌سازی شد.

۱.۶.۵ ساختارهای داده در ETMv4.h

فایل هدر ETMv4.h حاوی استراکت‌های کاملی از ماژول‌های سخت‌افزاری است:

- ETM_TypeDef: ساختار کامل ثبات‌های ETMv4 در آدرس 0xE0041000 شامل ثبات‌های کنترل برنامه‌ریزی (PRGCTL)، وضعیت (STAT)، پیکربندی (CONFIG)، شمارش چرخه (CCCTL)، شناسه ردیابی (TRACEID)، کنترل فیلترینگ (VICTL و VIPCSSCTL) و ثبات‌های شناسایی سیلیکون (IDR0 تا IDR13).
- TPIU_TypeDef: ساختار ثبات‌های واحد اینترفیس پورت ردیابی در آدرس 0x5C015000 شامل انتخاب اندازه پورت (CURPSIZE) و کنترل قالب‌بندی و تخلیه (FFCR).

- CSTF_TypeDef: ساختار کیف ردیابی در آدرس 0x5C013000 جهت کنترل مالتی پلکسر ورودی‌های ATB.
- ETF_TypeDef: ساختار بافر FIFO سخت‌افزاری در آدرس 0x5C014000 جهت مدیریت جریان داده در مد Hardware FIFO.
- TSG_TypeDef: ساختار مولد برچسب زمانی ۶۴ بیتی در آدرس 0x5C005000.
- Trace_Status_t: ساختار جامع عیب‌یابی که با یک فراخوانی وضعیت تمامی ثبات‌های مسیر ردیابی را در متغیر سراسری g_trace_status کپی می‌کند تا در دیباگر قابل بازبینی باشد.

۲.۶.۵ توابع عملیاتی در ETMv4.c

پیاده‌سازی ارائه‌شده در ETMv4.c شامل توابع کلیدی زیر است:

- Parallel_Trace_configure(): تابع اصلی راه‌اندازی که به ترتیب ماژول‌های پین خروجی، TPIU، ETF، CSTF، TSG، ETM و DWT را پیکربندی کرده و در نهایت ردیابی را فعال می‌سازد.
- Disable_ETM() و Enable_ETM(): وظیفه خاموش و روشن کردن ایمن هسته ETM. تابع غیرفعال‌سازی پس از صفر کردن بیت PRGCTL.EN، در یک حلقه منتظر ست شدن بیت‌های STAT.IDLE و STAT.PMSTABLE می‌ماند تا اطمینان حاصل شود خط لوله داخلی کاملاً تخلیه شده است.
- DWT_Configure(): مقایسه‌گرهای سخت‌افزاری شماره ۰ و ۱ ماژول DWT را روی نشانی‌های شروع و پایان پنجره ردیابی تنظیم می‌کند.
- StartPoint() و StopPoint(): توابع نشانه‌گذار مرزهای اندازه‌گیری.

راهکار ممانعت از بهینه‌سازی کامپایلر در توابع نشانه‌گذار: StartPoint و StopPoint

صرفاً شامل دستورات توخالی یا مشابه‌ای مانند nop باشند، بهینه‌ساز پیشرفته کامپایلر GCC در سطوح بهینه‌سازی بالا (Os یا O2-) به دلیل فعال بودن قابلیت (Identical Code Folding) -fipa-icf، هر دو تابع را در یک نشانی یکسان در حافظه کد تجمیع می‌کند! در این صورت، نشانی تریگر شروع و پایان در DWT برابر شده و هیچ ردیابی صورت نمی‌گیرد. برای شکستن این همسانی و ممانعت از ادغام توابع توسط کامپایلر، صفت ((noinline)) __attribute__ به همراه مقداردهی‌های متمایز به یک متغیر سراسری از جنس فرار (volatile) درون توابع تزریق گردید:

```

1 volatile uint32_t g_gate_marker;
2
3 __attribute__((noinline)) void StartPoint(void){
4     g_gate_marker = 0xA5A50001UL;
5 }
6

```

```

7 __attribute__((noinline)) void StopPoint(void){
8     g_gate_marker = 0x5A5A0002UL;
9 }

```

Listing 5.3: پیاده‌سازی توابع نشانه‌گذار مرز پنجره با ممانعت از ادغام بهینه‌ساز

۷.۵ روش به‌کارگیری محصول و تزریق به کدهای کاربری

طراحی این سامانه با هدف سهولت بهره‌برداری در پروژه‌های صنعتی و دانشگاهی انجام شده است. برای استفاده از این سیستم تأمین داده ردیابی در پروژه‌های مختلف، فرآیند زیر طی می‌شود:

۱. افزودن فایل‌های درایور ETMv4.h و ETMv4.c به ساختار پروژه نرم‌افزاری میکروکنترلر.
۲. فراخوانی تابع راه‌انداز اصلی Parallel_Trace_configure(); در فاز مقداردهی اولیه سیستم (پیش از ورود به حلقه اصلی برنامه).
۳. احاطه کردن قطعه‌کد، تابع، یا حلقه مورد نظر جهت سنجش WCET با دو فراخوانی ساده StartPoint(); و StopPoint();

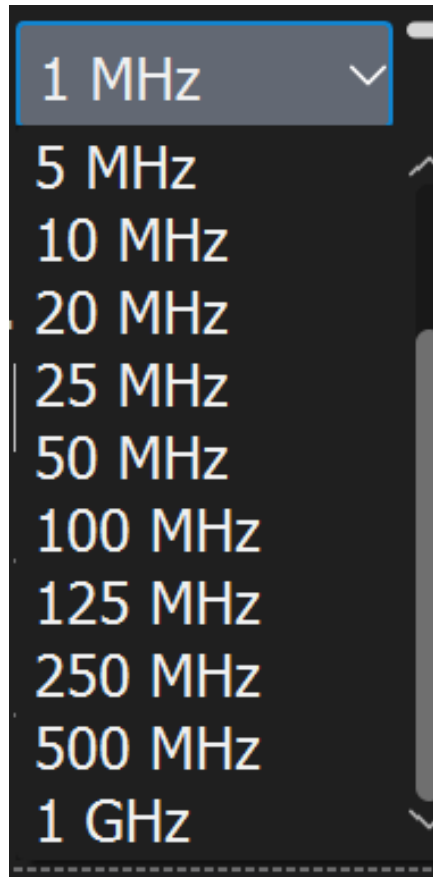
۱.۷.۵ تضمین عدم تداخل و حفظ رفتار غیرتهاجمی

این روش به‌کارگیری، تضمین قطعی می‌دهد که هیچ‌گونه تداخلی در رفتار زمانی، اولویت وقفه‌ها، یا منطق اجرایی سایر بخش‌های برنامه کاربر رخ نخواهد داد. دلایل این قطعیت عبارتند از:

- بلوک ردیابی ETM به صورت مستقل و مستقیم با فرکانس اصلی پردازنده (CPU_CLK) کار می‌کند و هیچ سرباری بر خطوط لوله دستورات هسته تحمیل نمی‌نماید.
- تنظیمات زنجیره کلاک سیستم (PLLها)، مقسم‌های فرکانسی، کنترل‌کننده‌های وقفه (NVIC) و پریفرال‌های کاربر بدون کمترین تغییر دست‌نخورده باقی می‌مانند.
- تنها تغییرات سخت‌افزاری اعمال‌شده، فعال‌سازی ماژول‌های زیرساخت دیباگ (CoreSight) و تغییر پیکربندی پایه‌های اختصاصی ردیابی (PE2 تا PE6) است که در کاربردهای عادی آزاد هستند.

۲.۷.۵ محدودیت نرخ‌های نمونه‌برداری سخت‌افزاری لاجیک آنالایزر

نکته بسیار مهم در استفاده عملیاتی از این فرمور، هماهنگی فرکانسی با ابزار گیرنده خارجی است. لاجیک آنالایزرهای دیجیتال (نظیر DSLogic U3Pro16) از فرکانس‌های نمونه‌برداری گسسته و از پیش تعیین‌شده‌ای پشتیبانی می‌کنند. شکل ۲؟ منوی فرکانس‌های مجاز نمونه‌برداری در نرم‌افزار لاجیک آنالایزر مورد استفاده را نمایش می‌دهد.



شکل ۵.۶: فرکانس‌های مجاز و گسسته نمونه‌برداری در نرم‌افزار لاجیک آنالایزر DSLogic

مطابق با شکل ۵.۶، نرخ‌های نمونه‌برداری دستگاه منحصراً به مقادیر ۱، ۵، ۱۰، ۲۰، ۲۵، ۵۰، ۱۰۰، ۱۲۵، ۲۵۰، ۵۰۰ مگاهرتز و ۱ گیگاهرتز محدود است.

همان‌طور که در فصل چهارم اثبات شد، برای نمونه‌برداری پایدار و بدون خطای جیت از لبه‌های کلاک پورت موازی، حداقل نسبت بیش‌نمونه‌برداری ۱۰ برابری ($10\times$) مورد نیاز است. کلاک پورت ردیابی (TRACECLK) معمولاً نصف کلاک هسته پردازنده است ($f_{\text{trace}} = f_{\text{CPU}}/2$). بنابراین:

- اگر پردازنده در فرکانس ۲۰ مگاهرتز تنظیم شود، کلاک Trace ۱۰ مگاهرتز بوده و با نرخ نمونه‌برداری ۱۰۰ مگاهرتز (۱۰ برابر) یا ۲۵۰ مگاهرتز (۲۵ برابر) لاجیک آنالایزر به طور کامل همخوانی دارد.

- اما اگر پردازنده در یک فرکانس کاری اختیاری غیرمنطبق (مثلاً ۴۸۰ مگاهرتز) تنظیم گردد، کلاک Trace ۲۴۰ مگاهرتز خواهد شد که به نرخ نمونه‌برداری ۴.۲ گیگاهرتز نیاز دارد؛ نرخی که فراتر از سقف ۱ گیگاهرتزی لاجیک آنالایزر بوده و در منوی سخت‌افزار وجود ندارد.

بنابراین، در حال حاضر کاربر برای نمونه‌برداری معتبر سیگنال‌های Trace، باید فرکانس کاری پردازنده را متناسب با نرخ‌های گسسته موجود در ابزار نمونه‌برداری انتخاب نماید.

۸.۵ چالش بسته‌های همگام‌سازی (A-Sync)

و آسیب‌شناسی روش سوئیچینگ

۱.۸.۵ کشف ریشه خطای NO_SYNC

و تصحیح وضعیت ثبات TRCSYNCPR

در فازهای نخست پایش قطعه کدهای بسیار کوتاه (نظیر اجرای یک حلقه یا روتین وقفه با طول زمانی کمتر از ۵۰ میکروثانیه)، خطای عدم تطبیق در موتور دکودر OpenCSD پدیدار می‌شد:

```
1 OpenCSD: Trace un-synced - waiting for sync packet (NO_SYNC)
2 Error: Frame synchronization failed. 0 instructions reconstructed.
```

کالبدشکافی جریان بایت‌ها ریشه این خطا را روشن ساخت:

- دکودر نرم‌افزاری جهت تطبیق دادن بایت‌های فشرده اتم (E/N) با باینری ELF، لزوماً نیازمند دانستن «آدرس مبدأ مطلق» است که توسط بسته همگام‌سازی دستورالعمل (I_SYNC / A-Sync) حمل می‌شود.

- در معماری ETMv4، ارسال بسته‌های I_SYNC بر عهده شمارنده داخلی است که دوره‌ای بودن آن به ثبات TRCSYNCPR منتهی می‌شود.

- نکته حیاتی و تصحیح در گزارش: در میکروکنترلر STM32H750، ثبات TRCSYNCPR در سیلیکون به صورت کاملاً فقط‌خواندنی (Read-Only) پیاده‌سازی شده و مقدار آن به طور سخت‌افزاری بر روی مقدار هگزادسیمال 0x0A قفل است (معادل $2^{10} = 1024$ بایت). در نسخه‌های ابتدایی درایور، کدهایی جهت نوشتن مقادیر کوچکتری نظیر 0x08 در این ثبات وجود داشت؛ اما به دلیل ماهیت فقط‌خواندنی بودن، این عملیات توسط سخت‌افزار نادیده گرفته می‌شد.

- در قطعه کدهای کوتاه‌مدت، حجم کل داده‌های ردیابی تولیدشده تنها در حدود ۲۸ تا ۴۸ بایت بود. از آنجا که حجم داده به سقف سخت‌افزاری ۱۰۲۴ بایت نمی‌رسید، سخت‌افزار هیچ بسته همگام‌سازی صادر نمی‌کرد و دکودر به دلیل فقدان نشانی اولیه قادر به رمزگشایی جریان نبود.

۲.۸.۵ روش ابداعی سوئیچینگ TRCPRGCTLR.EN و آسیب‌شناسی آن

برای حل معضل بسته‌های همگام‌سازی، با اتکا به یک ویژگی معماری کشف‌شده در استاندارد ARM مبنی بر این که:

«تغییر وضعیت بیت برنامه‌ریزی TRCPRGCTLR.EN از صفر به یک، ماشین حالت ETM را ریست کرده و سخت‌افزار را قانوناً ملزم می‌سازد تا در نخستین فریم خروجی، یک بسته همگام‌سازی اجباری اولیه (Initial I_SYNC) منتشر نماید.»

ایده خاموش و روشن کردن پویا و مقطعی ماژول ETM در شروع هر پنجره آزمون پیاده‌سازی گردید. با این حال، ارزیابی‌های دقیق‌تر پرده از نقایص و خطرات این روش برداشت.

مشکلات و آسیب‌شناسی روش خاموش و روشن کردن مکرر ETM:

۱. از بین رفتن پیوستگی و یکپارچگی جریان ردیابی: خاموش و روشن کردن پی‌درپی ETM موجب شکستن پیوستگی زمانی داده‌ها می‌شود. در هر بار خاموش شدن، پایپ‌لاین فشرده‌سازی بازنشانی شده و تاریخچه ثبت شاخه‌ها پاک می‌گردد. در نتیجه، لاگ خروجی حاوی قطعات ناپیوسته و نامفهوم خواهد بود.

۲. عدم تخلیه کامل بافر ردیابی (Buffer Drain Truncation): هنگامی که ETM به صورت ناگهانی غیرفعال (Disable) می‌شود، داده‌های ردیابی مربوط به آخرین دستورالعمل‌های اجرا شده هنوز در بافرهای FIFO داخلی ماژول و رم ETF قرار دارند و فرصت انتقال کامل به پورت TPIU را پیدا نکرده‌اند. در نتیجه، بخش انتهایی قطعه‌کد مورد سنجش دور ریخته شده یا با خطا مواجه می‌گردد.

راهکار مطمئن (Robust) نهایی: Trace ممتد در یک پنجره و فیلترینگ نرم‌افزاری در دکودر: برای عبور بنیادین از این مشکلات، بهترین و قابل‌اعتمادترین رویکرد مهندسی آن است که:

- ماژول ETM در طول اجرای برنامه در یک پنجره زمانی متوالی، پیوسته روشن و فعال بماند تا هیچ بسته‌ای به دلیل قطع و وصل ناگهانی مفقود نگردد و یکپارچگی خط لوله فشرده‌سازی حفظ شود.
- در سمت کامپیوتر میزبان، اسکرپت دکودر پس از رمزگشایی کامل جریان بایت‌ها، در لاگ خروجی به جستجوی رخداد‌های ورود به تابع StartPoint و خروج از StopPoint پردازد.

گرچه این رویکرد به دلیل ثبت پیوسته دستورات حجم بایت‌های بیشتری تولید کرده و بار پردازشی دکودر را افزایش می‌دهد، اما داده‌هایی کاملاً یکپارچه، بدون مفقودی و فوق‌العاده مطمئن (Robust) برای سنجش قطعی WCET فراهم می‌سازد.

۹.۵ جمع‌بندی فصل

در این فصل پروتکل فشرده‌سازی داده‌ها در ETMv4، معماری زیرساخت دیباگ در میکروکنترلر STM32H750 و دامنه‌های توان و کلاک آن تبیین شد. تشریح جامع ثبات پیکربندی TRCCONFIGR، ارائه شواهد مغایرت اسناد ARM با راهنمای مرجع تراشه‌ها، معرفی فایل‌های درایور اختصاصی ETMv4.h و ETMv4.c، بررسی روش‌های به‌کارگیری و تزریق خودکار کد، و در نهایت ریشه‌یابی چالش بسته‌های A-Sync و آسیب‌شناسی روش‌های سوئیچینگ ETM مستند گردید. با تسلط بر این زیرساخت سخت‌افزاری و سیستم نرم‌افزاری روی تراشه، زمینه برای بررسی نحوه دریافت و رمزگشایی بایت‌ها در فصل ششم فراهم گردیده است.

فصل ۶

پردازش و رمزگشایی Trace

۱.۶ مقدمه

داده‌های دیجیتال خام ثبت شده بر روی حافظه لاجیک آنالایزر، دنباله‌ای از نمونه‌های ولتاژ ناهمگام هستند که بدون طی یک خطلوله پردازش نرم‌افزاری، فاقد معنای محاسباتی می‌باشند. پروتکل فشرده‌سازی ETMv4 یک جریان کاملاً غیرخطی از تصمیمات انشعاب و رویدادهای سیستمی است که باید با کدهای باینری فایل کامپایل شده (ELF) تلفیق شود تا به دستورات متوالی زبان اسمبلی و زمان‌بندی دقیق تبدیل گردد. برای تحقق این هدف، اسکریپت پیشرفته پایتون موسوم به `TraceStreamProcessor.py` طراحی و پیاده‌سازی شد. این ابزار به عنوان موتور اصلی پیش‌پردازش، وظیفه استخراج بایت‌های فریم پورت TPIU، اعتبارسنجی یکپارچگی سیگنال‌ها، ساختار بندی اسنپ‌شات استاندارد و ادغام با موتور مرجع صنعتی OpenCSD را بر عهده دارد. در این فصل، کالبدشکافی خطلوله پردازش داده، ساختار فایل‌های اسنپ‌شات، پیوند عمیق پارامترهای دکودر با ثبات‌های سخت‌افزاری پردازنده، و در نهایت طراحی و پیاده‌سازی افزونه اختصاصی محیط توسعه VS Code موسوم به CoreSight Trace Studio جهت اتوماسیون کامل این فرآیندها تشریح می‌گردد.

۲.۶ معماری اسکریپت پردازش داده (TraceStreamProcessor.py)

اسکریپت `TraceStreamProcessor.py` به عنوان پل ارتباطی هوشمند میان لایه سخت‌افزار ثبت سیگنال و موتور نرم‌افزاری رمزگشای مرجع OpenCSD عمل می‌کند. در رویکردهای اولیه، رمزگشایی مستقیم داده‌های خام با خطاهای متعددی نظیر عدم تراز فاز، گم‌شدن مرز فریم‌ها و ناسازگاری قالب مواجه می‌شد. برای غلبه بر این مسائل، خطلوله‌ای هفت‌مرحله‌ای و کاملاً خودکار پیاده‌سازی گردید که وظایف تبدیل، تراز‌سازی، اعتبارسنجی و بسته‌بندی را انجام می‌دهد. شکل ۴۹؟ فلوچارت مراحل اجرایی این اسکریپت را به تصویر کشیده است.



شکل ۱.۶: فلوچارت خط‌لوله پردازش هفت‌مرحله‌ای اسکریپت TraceStreamProcessor.py

۳.۶ کالبدشکافی فنی مراحل پردازش در TraceStreamProcessor

۱.۳.۶ آشکارسازی لبه، حذف گلیچ و استخراج نیبل‌ها

در مد ردیابی موازی ۴ بیتی، هر بایت داده شامل دو نیبل ۴ بیتی است که در لبه‌های کلاک پورت TPIU صادر می‌شوند. سیگنال‌های دریافتی از لاجیک آنالایزر با نرخ اورسмпلینگ ۱۰ برابری ثبت شده‌اند؛ بنابراین هر دوره تناوب کلاک با چندین نمونه دیجیتال نشان داده می‌شود. اسکرپت برای ممانعت از خطاهای ناشی از نویزهای ضربه‌ای و جهش‌های ولتاژی (Glitches)، از یک مکانیزم فیلترینگ پایداری لبه استفاده می‌کند: یک نیبل تنها زمانی معتبر شناخته می‌شود که وضعیت کلاک نسبت به نمونه پیشین تغییر کرده و برای حداقل یک نمونه در وضعیت جدید پایدار بماند. قطعه‌کد؟؟ پیاده‌سازی این منطق را نشان می‌دهد:

```

1 def nibbles_from_csv(path):
2     """
3     Recover TRACEDATA nibbles from DSLogic oversampled CSV export.
4     Rejects single-sample glitches by ensuring the clock state
5     persists after an edge transition.
6     """
7     CLK_COL, DO_COL, D1_COL, D2_COL, D3_COL = 1, 2, 3, 4, 5
8     nibbles = bytearray()
9     prev_raw_clk = None
10
11     with open(path, "r") as f_in:
12         for row in csv.reader(f_in):
13             if not row or row[0].startswith(";") or "Time" in row[0]:
14                 continue
15             try:
16                 raw_clk = int(row[CLK_COL])
17                 nib = ((int(row[D3_COL]) << 3) | (int(row[D2_COL]) << 2) |
18                       (int(row[D1_COL]) << 1) | int(row[DO_COL]))
19             except (ValueError, IndexError):
20                 continue
21
22             if prev_raw_clk is None:
23                 prev_raw_clk = raw_clk
24                 continue
25
26             # Clock edge detected
27             if raw_clk != prev_raw_clk:
28                 nibbles.append(nib)
29                 prev_raw_clk = raw_clk

```

```

30
31 return nibbles

```

Listing 6.1: TraceStreamProcessor.py منطق آشکارسازی لبه کلاک و استخراج نیبل‌ها در

۲.۳.۶ تراسازی فریم‌های پروتکل TPIU بر پایه الگوهای FSYNC

در استاندارد معماری CoreSight، بایت‌های ردیابی مستقیماً به صورت خام بر روی سیم منتشر نمی‌شوند؛ بلکه در قالب فریم‌های ۱۶ بیتی پروتکل واسط پورت TPIU بسته‌بندی می‌گردند. در این فریم‌ها:

- بایت پانزدهم (بایت آخر) به عنوان بایت کمکی (Auxiliary Byte) عمل می‌کند که بیت‌های کم‌ارزش بایت‌های زوج و نشانگرهای تغییر شناسه جریان (ATB ID) را در خود جای داده است.
- برای همگام‌سازی فریم‌ها، الگوهای ویژه FSYNC با توالی هگزادسیمال 0x7FFFFFFF (۴ بایت: FF 7F FF FF) به طور منظم در جریان داده ارسال می‌شوند. همچنین در زمان‌های بیکاری پورت، بایت‌های پرکننده HSYNC با مقدار 0x7FFF (۲ بایت: FF 7F) درج می‌گردند.

یکی از دستاوردهای برجسته اسکریپت TraceStreamProcessor.py، کشف این حقیقت مهندسی بود که بایت‌های سیم نباید پیش از ارسال به OpenCSD به صورت دستی دی‌فریم (De-frame) شوند! در نسخه‌های ابتدایی، اسکریپت سعی در حذف دستی بایت‌های همگام‌سازی و جداسازی بایت‌های داده داشت که به دلیل لغزش فاز موجب خطای غیرقابل‌رفع I_BAD_SEQUENCE در موتور دکودر می‌شد. در نسخه توسعه‌یافته حاضر، بایت‌های کامل سیم همراه با الگوهای همگام‌سازی در قالب فایل باینری tpiu_wire.bin حفظ شده و با تعریف فرمت coresight در فایل پیکربندی، دی‌فریم کردن و تطبیق فاز به دی‌فرمتور داخلی و بهینه‌شده موتور OpenCSD واگذار می‌گردد. اسکریپت با اسکن الگوهای FSYNC، ابتدای بافر را به نخستین فریم معتبر ۱۶ بیتی تراز کرده و قطعات ناقص ضبط اولیه و انتهایی را پیراست (Trim) می‌نماید.

۳.۳.۶ تفکیک خودکار سگمنت‌های اجرایی باینری (ELF Reader)

موتور رمزگشای OpenCSD برای اینکه بتواند بسته‌های ردگیری (شامل تصمیمات انشعاب شرطی) را به کدهای اسمبلی تبدیل کند، نیازمند دسترسی به تصویر کدهای باینری موجود در حافظه فلش میکروکنترلر است. اسکریپت TraceStreamProcessor.py دارای یک ماژول مستقل پارسر فایل‌های باینری ELF ۳۲ بیتی با فرمت لیتل‌اندیان (Little-Endian) است که بدون وابستگی به تولچین‌های سنگین خارجی (نظیر arm-none-eabi-objcopy)، مستقیماً هدرهای برنامه (Program Headers) را کاوش نموده و سگمنت‌های با فلگ PT_LOAD را که حاوی کدهای اجرایی برنامه کاربر هستند استخراج می‌کند. محتوای این سگمنت‌ها به عنوان فایل‌های دامپ حافظه (نظیر mem_08000000.bin) در پوشه اسنپ‌شات ذخیره می‌شوند.

۴.۶ ادغام با موتور رمزگشای مرجع OpenCSD

پس از ایجاد و ساختاربندی پوشه اسنپشات، اسکریپت ابزار خط فرمان استاندارد `trc_pkt_lister` متعلق به بنیاد Linaro را با دستور زیر فراخوانی می‌کند:

```
1 trc_pkt_lister -ss_dir ./snapshot/ -tpiu_hsync -decode -logstdout \
2 -o decoded_trace.txt
```

در این دستور، فلگ `-tpiu_hsync` دی‌فرمتور سخت‌افزاری OpenCSD را فعال می‌سازد تا بسته‌های پرکننده همگام‌سازی را حذف نماید و آرگومان `-decode` دستور می‌دهد که کدهای باینری با جریان ردگیری تلفیق شده و دستور به دستور بازسازی گردند. عملیات دکود در سه لایه پردازشی انجام می‌گیرد:

۱. پردازشگر بسته‌ها (**Packet Processor**): بایت‌های ورودی را به عناصر سازنده پروتکل ETMv4 شامل بسته‌های همگام‌سازی (`I_SYNC`)، بسته‌های اتم (`ATOM`)، برچسب زمانی (`TIMESTAMP`)، شمارش چرخه (`CYCLE_COUNT`) و استثناها (`EXCEPTION`) تجزیه می‌کند.

۲. رمزگشای دستورات (**Instruction Decoder**): با استخراج نشانی شروع برنامه از بسته `I_SYNC`، کدهای ماشین را در سگمنت‌های بارگذاری شده دنبال می‌کند. با برخورد به دستورات انشعاب شرطی، وضعیت شاخه را از بیت‌های بسته اتم خوانده و مسیر واقعی اجرای برنامه را بازسازی می‌نماید.

۳. قالب‌بندی عناصر ردگیری (**Trace Element Formatter**): جریان زمانی دستورات را همراه با نشانی دستور، کد اسمبلی، نام تابع مبدأ، شماره خط در کد مبدأ و شماره چرخه ماشین در قالب فایل متنی ساختاریافته استخراج می‌کند.

۵.۶ کالبدشکافی ساختار فایل‌های اسنپشات OpenCSD

۱.۵.۶ ساختار و محتوای فایل‌های پوشه اسنپشات

پوشه اسنپشات تولیدشده توسط `TraceStreamProcessor.py` یک کپسول کاملاً مستقل و خودکفا از شرایط تست، اتصالات سخت‌افزاری و جریان داده‌هاست. این پوشه شامل پنج جزء کلیدی است:

۱. فایل پیکربندی ریشه (`snapshot.ini`): این فایل به عنوان مانیفست اصلی دکودر عمل کرده و مشخصات نسخه پروتکل، اسامی فایل‌های توصیف تجهیزات و فایل متادیتای بافر را تعیین می‌کند:

```
1 [snapshot]
2 version=1.0
3
4 [device_list]
5 device1=device1.ini
6 device2=device2.ini
7
```

```

8 [trace]
9 metadata=trace.ini

```

Listing 6.2: snapshot.ini محتوای فایل مانیفست ریشه

۲. فایل توصیف هسته پردازنده (device1.ini): این فایل معماری هسته پردازنده، وضعیت اولیه ثبات‌های کنترلی آن، و ارجاع به قطعات استخراج‌شده از حافظه فلش فایل ELF برنامه را تعریف می‌نماید:

```

1 [device]
2 name=Cortex-M7_0
3 class=core
4 type=Cortex-M7
5
6 [regs]
7 PC=0x08000000
8 xPSR=0x01000000
9
10 [dump1]
11 file=mem_08000000.bin
12 address=0x08000000
13 length=0x1E4A0

```

Listing 6.3: device1.ini محتوای فایل توصیف هسته

۳. فایل توصیف منبع ردگیری و ثبات‌های سخت‌افزاری (device2.ini): این فایل هویت منبع ردگیری را با نام CSETM_0 و کلاس trace_source تعیین کرده و حیاتی‌ترین اطلاعات مورد نیاز دکودر یعنی مقادیر ثبات‌های سخت‌افزاری ماژول ETM را در خود جای داده است:

```

1 [device]
2 name=CSETM_0
3 class=trace_source
4 type=ETM4.0
5
6 [regs]
7 TRCCONFIGR(0x4)=0x00000811
8 TRCEVENTCTLOR(0x8)=0x00000000
9 TRCEVENTCTL1R(0x9)=0x00000000
10 TRCSTALLCTLR(0xB)=0x00000000
11 TRCTSCTLR(0xC)=0x00000000
12 TRCSYNCPR(0xD)=0x0000000A
13 TRCCCCTLR(0xE)=0x00000040
14 TRCTRACEIDR(0x10)=0x0000003E

```

```

15 TRCVICTLR(0x20)=0x00000201
16 TRCVIIECTLR(0x21)=0x00000000
17 TRCVISSCTLR(0x22)=0x00000000
18 TRCVIPCSSCTLR(0x23)=0x00020001
19 TRCIDR8(0x60)=0x00000001
20 TRCIDR9(0x61)=0x00000000
21 TRCIDR10(0x62)=0x00000000
22 TRCIDR11(0x63)=0x00000000
23 TRCIDR12(0x64)=0x00000001
24 TRCIDR13(0x65)=0x00000000
25 TRCIDR0(0x78)=0x080006E1
26 TRCIDR1(0x79)=0x4100F401
27 TRCIDR2(0x7A)=0x00000004
28 TRCIDR3(0x7B)=0x07090004
29 TRCIDR4(0x7C)=0x00114000
30 TRCIDR5(0x7D)=0x90C70002
31 TRCAUTHSTATUS(0x3EE)=0x000000C0

```

Listing 6.4: محتوای فایل منبع ردگیری و ثبات‌های سخت‌افزاری device2.ini

۴. فایل متادیتای بافر و توپولوژی ارتباطات (trace.ini): این فایل ساختار ارتباطی گذرگاه‌ها و نحوه هدایت داده‌ها از پورت فیزیکی به موتور نرم‌افزاری را تبیین می‌سازد:

```

1 [trace_buffers]
2 buffers=buffer0
3
4 [buffer0]
5 name=TPIU_WIRE
6 file=tpiu_wire.bin
7 format=coresight
8
9 [core_trace_sources]
10 Cortex-M7_0=CSETM_0
11
12 [source_buffers]
13 CSETM_0=TPIU_WIRE

```

Listing 6.5: محتوای فایل توپولوژی بافر trace.ini

۲.۵.۶ انطباق پارامترهای اسنپ‌شات با ثبات‌های فیزیکی پردازنده

یک نکته بنیادین که در این پژوهش اثبات گردید آن است که: موتور رمزگشای OpenCSD بدون آگاهی دقیق از مقادیر ثبات‌های سخت‌افزاری پردازنده بر روی سیلیکون، به هیچ عنوان قادر به رمزگشایی صحیح داده‌ها نخواهد بود. بخش [regs] در فایل device2.ini دقیقاً حامل مقادیر همان ثبات‌هایی است که در فصل پنجم در فریمورک میکروکنترلر و در تابع ETM_Configure() مقداردهی شده‌اند:

- **ثبات TRCCONFIGR (آفست 0x010، شناسه 0x4):** این ثبات تعیین می‌کند که چه امکاناتی فعال بوده‌اند. در پروژه حاضر، مقدار هگزادسیمال 0x00000811 نشان‌دهنده فعال بودن شمارش دقیق چرخه‌ها (CCI - بیت ۴)، برچسب زمانی سراسری (TS - بیت ۱۱) و فعال بودن موتور ردیابی است. اگر دکودر از فعال بودن این بیت‌ها مطلع نباشد، بایت‌های مربوط به شمارنده‌ها را به عنوان بایت‌های نامعتبر تلقی کرده و خروجی با خطای مهلک مواجه می‌شود.
- **ثبات TRCCCCTLR (آفست 0x038، شناسه 0xE):** آستانه صدور بسته‌های چرخه‌ای را تعیین می‌کند (مقدار 0x40 معادل ۶۴ چرخه). دکودر با دانستن این مقدار، فواصل زمانی میان دستورات متوالی را تخمین می‌زند.
- **ثبات TRCSYNCP (آفست 0x034):** مشخص‌کننده دوره صدور بسته‌های همگام‌سازی اجباری (I_SYNC) است که در سیلیکون STM32H750 روی مقدار ثابت 0x0A ($2^{10} = 1024$ بایت) قفل است.
- **ثبات TRCTRACEIDR (آفست 0x040):** شناسه منبع در گذرگاه ATB (مقدار 0x3E) است. دکودر در لایه فریم‌زدایی TPIU، بسته‌های مربوط به این شناسه را فیلتر کرده و به موتور ETMv4 تحویل می‌دهد.
- **ثبات‌های فیلترینگ (TRCVIPCSSCTLR و TRCVICTLR):** تعیین‌کننده تریگرهای شروع و پایان پنجره ردگیری توسط مقایسه‌گرهای DWT هستند که وضعیت پنجره پایش فعال را به دکودر اعلام می‌نمایند.

۶.۶ مدیریت مرزهای وقفه و کانتکست‌های پردازنده

یکی از توانمندی‌های برجسته این خطلوله، تشخیص تمیز ورود و خروج به روتین‌های وقفه (ISR) است. در پردازنده Cortex-M7، وقوع وقفه موجب درج خودکار فریم پشته بر روی MSP و صدور مقدار بازگشت ویژه EXC_RETURN (نظیر 0xFFFFFFFF9) می‌گردد. خطلوله پایتون با تحلیل عناصر رویداد استثنا (Exception Packets) در موتور OpenCSD و بازگشت با دستور LR BX، زمان سپری‌شده در درون روتین وقفه را به صورت کاملاً ایزوله از کدهای زمینه برنامه اصلی استخراج می‌کند.

۷.۶ ابزار توسعه یکپارچه: افزونه اختصاصی CoreSight Trace Studio

فرآیند ردیابی سخت‌افزاری با پروتکل ETMv4 و استنتاج بیشینه زمان اجرا (WCET)، ذاتاً فرآیندی چندلایه‌ای و حساس است که از سطح ثبات‌های سیلیکونی میکروکنترلر آغاز شده، از خط‌لوله پردازش اسنپ‌شات می‌گذرد و به تحلیل زمان‌بندی دستورات منتهی می‌گردد. انجام دستی این مراحل نیازمند تسلط هم‌زمان بر پیکربندی ثبات‌های C، تطبیق‌عاری از خطای فایل‌های INI اسنپ‌شات OpenCSD، اجرای دستورات پیچیده خط فرمان و محاسبه ریاضی زمان است. هرگونه عدم انطباق جزئی میان ثبات‌های سخت‌افزاری و متادیتای اسنپ‌شات، موجب شکست فرآیند رمزگشایی می‌گردد.

برای رفع کامل این پیچیدگی‌ها، حذف احتمال خطای انسانی و فراهم آوردن یک تجربه کاربری مدرن و صنعتی، یک افزونه اختصاصی تحت عنوان **CoreSight Trace Studio** برای محیط توسعه محبوب Visual Studio Code طراحی و پیاده‌سازی گردید. این افزونه به عنوان یک محیط کاربری یکپارچه (IDE GUI)، تمام ابعاد فرآیند را در سه زبانه متوالی و هدفمند در اختیار توسعه‌دهنده قرار می‌دهد.

۱.۷.۶ تب اول: مدیریت درایور سخت‌افزار و تزریق کدهای ابزار دقیق (Instrumentation & Driver)

تب اول افزونه (شکل ۱.۷.۶) بر آماده‌سازی محیط برنامه میکروکنترلر و پنجره‌گذاری آسان کدهای کاربری تمرکز دارد و شامل دو بخش اصلی است:

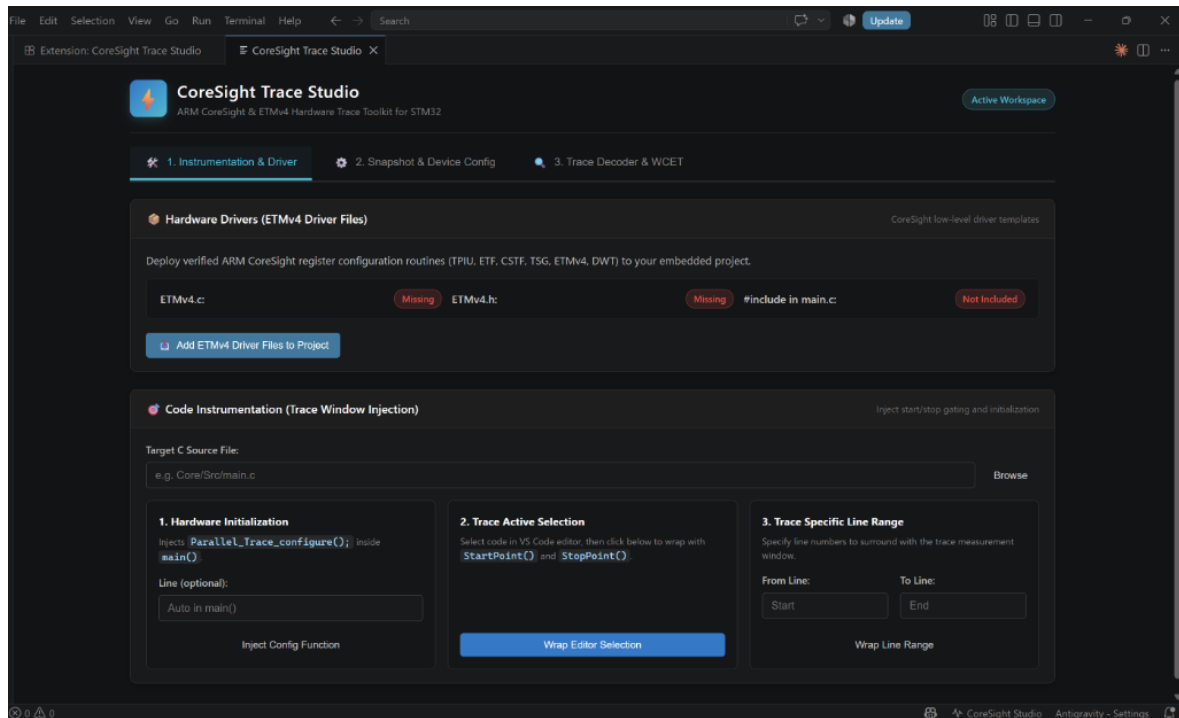
۱. مدیریت فایل‌های راه‌انداز سخت‌افزاری (**Hardware Drivers**): افزونه به طور زنده ساختار دایرکتوری پروژه جاری را بررسی می‌کند. وضعیت وجود فایل‌های درایور تاییدشده (ETMv4.c) و ETMv4.h و شمول هدر در فایل اصلی (#include in main.c) با نشانگرهای گرافیکی مشخص می‌شود. با کلیک بر روی دکمه **Add ETMv4 Driver Files to Project**، کلیه توابع بهینه‌شده راه‌اندازی ثبات‌های CoreSight به صورت خودکار به ساختار پروژه افزوده می‌گردد.

۲. تزریق خودکار پنجره ردیابی (**Code Instrumentation**): این بخش کاربر را از نگارش دستی توابع نشانه‌گذار بی‌نیاز می‌سازد. پس از تعیین فایل منبع هدف (نظیر Core/Src/main.c)، سه عملیات در دسترس است:

- **تزریق تابع پیکربندی سخت‌افزار:** درج خودکار فراخوانی تابع `Parallel_Trace_configure()` در ابتدای تابع `main`.

- **احاطه کردن متن انتخابی در ادیتور (Wrap Editor Selection):** کاربر با نشانگر ماوس در ویرایشگر متن VS Code، هر بخش، حلقه یا تابعی را انتخاب نموده و با فشردن این دکمه، افزونه به صورت کامپایلری دستورات نشانه‌گذار `StartPoint()` و `StopPoint()` را در بالا و پایین کد انتخابی درج می‌نماید.

- احاطه بر مبنای شماره خطوط (Wrap Line Range): امکان وارد کردن شماره خط شروع و پایان جهت احاطه کدهای هدف با پنجره ردیابی.



شکل ۲.۶: نمای تب اول افزونه CoreSight Trace Studio: مدیریت فایل‌های درایور ETMv4 و تزریق خودکار پنجره ردیابی

۲.۷.۶ تب دوم: پیکربندی اسنپ‌شات و ثبات‌های سخت‌افزاری (Snapshot & Device Config)

تب دوم افزونه (شکل ۲.۷) ارتباط میان خروجی‌های لاجیک‌آنالایزر و دکودر مرجع OpenCSD را برقرار می‌سازد:

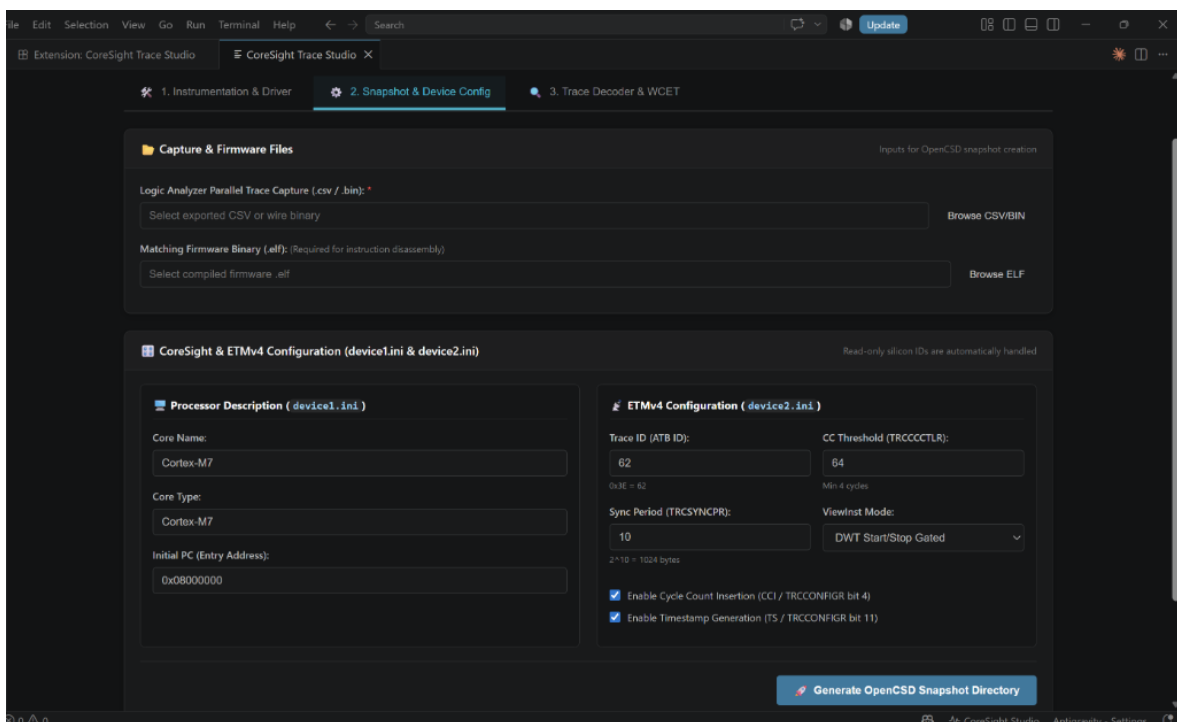
۱. ورود فایل‌های ثبت سیگنال و باینری (Capture & Firmware Files): کاربر فایل خروجی نمونه‌برداری لاجیک‌آنالایزر (در قالب فایل متنی CSV یا فایل باینری Wire) و فایل باینری فریمورک کامپایل شده (.elf) را انتخاب می‌کند.
۲. پیکربندی گرافیکی هسته و ثبات‌های ETMv4: در این بخش، کاربر پارامترهای پیکربندی را به صورت بصری تعیین می‌نماید:

- توصیف پردازنده (device1.ini): مشخص کردن نام و نوع هسته پردازشی (Cortex-M7) و آدرس شروع اجرای برنامه (Initial PC).
- تنظیمات ثباتی ETMv4 (device2.ini): تعیین شناسه Trace ID گذرگاه ATB (مقدار پیش‌فرض ۶۲ معادل 0x3E)، آستانه بسته‌های شمارنده چرخه TRCCCCTLR (مقدار پیش‌فرض

۶۴ چرخه)، دوره بسته‌های همگام‌سازی اجباری TRCSYNCP (1024 = 2^{10} بایت)، و مد پنجره‌گذاری ViewInst.

- **گزینه‌های تکمیلی:** دو چک‌باکس صریح جهت فعال‌سازی درج بسته‌های شمارش چرخه (CCI / TRCCONFIGR bit 4) و برچسب زمانی سراسری (TS / TRCCONFIGR bit 11).

با کلیک بر روی دکمه Generate OpenCSD Snapshot Directory، پوشه اسنپشات استاندارد شامل فایل‌های snapshot.ini، device1.ini، device2.ini، trace.ini و قطعات حافظه فلش به صورت کاملاً همگام و منطبق با کدهای میکروکنترلر ساخته می‌شود.



شکل ۳.۶: نمای تب دوم افزونه CoreSight Trace Studio: ورود فایل‌های داده و باینری و تولید متناظر فایل‌های اسنپشات OpenCSD

۳.۷.۶ تب سوم: رمزگشایی، محاسبه WCET و داشبورد عملکرد (Trace De-coder & WCET)

تب سوم افزونه (شکل ۳.۷) موتور اجرایی، مرکز محاسبه WCET و نمایش گرافیکی نتایج است:

۱. موتور اجرایی رمزگشایی (Decoder Execution Engine):

- دریافت مسیر اسنپشات و باینری ELF.
- قابلیت انتخاب موتور دکودر پشتیبان بین دکودر پایتون (TempDecoder_V0.2.py) و موتور مرجع OpenCSD trc_pkt_lister.

- دریافت فرکانس ساعت کاری هسته (Core Clock Frequency) بر حسب مگاهرتز (مقدار پیش‌فرض ۴۸۰ مگاهرتز برای STM32H750). این فرکانس برای تبدیل مستقیم چرخه‌های شمارش شده پردازنده به زمان حقیقی اجرای کد به کار می‌رود:

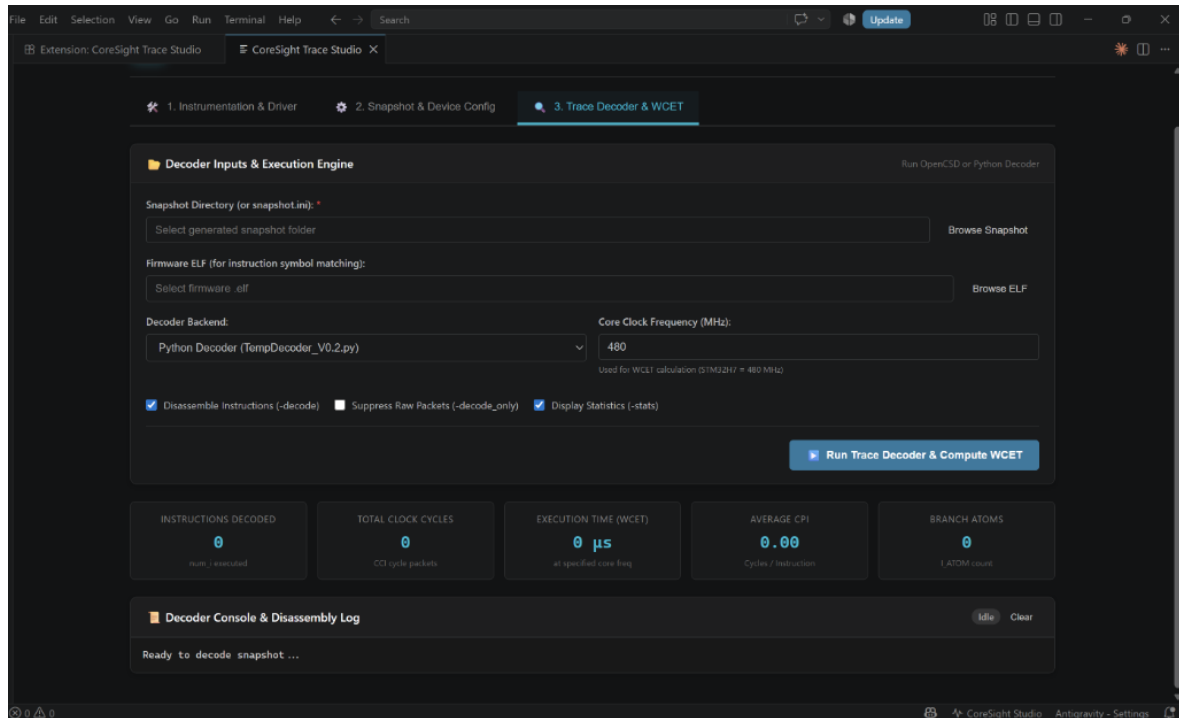
$$t_{\text{exec}} = \frac{N_{\text{cycles}}}{f_{\text{core}}} \quad (۱.۶)$$

- گزینه‌های خط فرمان دکودر شامل دیس‌اسمبل دستورات (-decode)، پنهان‌سازی بسته‌های خام برای افزایش سرعت (-decode_only)، و استخراج آماره‌های تفصیلی عملکرد (-stats).
- دکمه اجرای پردازش: Run Trace Decoder & Compute WCET.

۲. داشبورد شاخص‌های کلیدی عملکرد (KPI Metrics): پس از پایان پردازش، پنج کارت آماری به صورت آنی مقادیر را نمایش می‌دهند:

- **تعداد دستورات رمزگشایی شده (Instructions Decoded):** شمار کل دستورالعمل‌های اسمبلی که در طول پنجره ردیابی با موفقیت اجرا و بازسازی شده‌اند.
- **مجموع چرخه‌های ساعت (Total Clock Cycles):** تعداد کل پالس‌های کلاک صرف‌شده در اجرای کد بر پایه بسته‌های CCI.
- **بیشینه زمان اجرا (Execution Time / WCET):** زمان اجرای استخراج‌شده بر حسب میکروثانیه (μs) با دقت نانوثنیه‌ای ناشی از شمارش چرخه‌ای.
- **میانگین چرخه‌ها به ازای هر دستور (Average CPI):** شاخص کارایی معماری و تأخیر خط‌لوله پردازنده.
- **تعداد تصمیمات انشعاب (Branch Atoms):** شمار اتم‌های پرش شرطی (E/N Atom) که پیچیدگی مسیرهای اجرایی را مشخص می‌کند.

۳. کنسول دکودر و لاگ دیس‌اسمبل (Decoder Console): ترمینال تعبیه‌شده در انتهای صفحه، لاگ خط‌به‌خط دستورات اسمبلی، برچسب‌های زمانی و کدهای وضعیت را همراه با امکان پاک‌سازی سریع به صورت زنده نمایش می‌دهد.



شکل ۴.۶: نمای تب سوم افزونه CoreSight Trace Studio: موتور پردازش، داشبورد شاخص‌های کلیدی WCET و کنسول لاگ دکود

۸.۶ جمع‌بندی فصل

در این فصل، معماری کامل اسکریپت پردازش داده‌های ردیابی موسوم به `TraceStreamProcessor.py`، فرآیند آشکارسازی لبه‌ها و بازیابی بایت‌های فریم پورت TPIU، اعتبارسنجی یکپارچگی سیگنال‌ها و ادغام با موتور مرجع صنعتی OpenCSD مستند گردید. علاوه بر این، کالبدشکافی عمیق فایل‌های اسنپ‌شات ارائه شد و انطباق ضروری پارامترهای دکودر با ثبات‌های سخت‌افزاری پردازنده تبیین شد. در نهایت، طراحی و پیاده‌سازی افزونه اختصاصی محیط توسعه VS Code تحت عنوان CoreSight Trace Studio به عنوان یک ابزار جامع سه زبانه تشریح گردید؛ ابزاری که از تزریق خودکار درایورها و پنجره‌های پایش تا تولید هماهنگ اسنپ‌شات و استخراج آماری شاخص‌های WCET را به صورت یکپارچه محقق ساخته است. در فصل هفتم، ارزیابی تجربی و صحنه‌گذاری آزمایشگاهی این خط‌لوله در سناریوهای آزمون واقعی مورد بررسی قرار خواهد گرفت.

فصل ۷

ارزیابی تجربی و نتایج

۱.۷ مقدمه

ارزیابی تجربی و صحت‌گذاری عملکرد، مرحله نهایی در اثبات کارآمدی خطلوله طراحی شده برای استنتاج زمان اجرای بدترین شرایط (WCET) است. یک سامانه تأمین داده قابل‌اتکا باید در ابعاد گوناگون شامل: «پایداری همگام‌سازی در حجم‌های سنگین محاسباتی»، «تعیینی بودن مطلق و رفتار غیرتهاجمی در پاسخ به وقفه‌ها»، و «انطباق برچسب‌های زمانی با رفتارهای فیزیکی سیگنال» به اثبات عملیاتی برسد. در این فصل، بستر سخت‌افزاری و نرم‌افزاری در قالب سه سناریوی تجربی هدفمند مورد ارزیابی قرار می‌گیرد:

۱. **سناریوی ۱ (TimerTest):** آزمون ردگیری پریودیک تایمر ۱۰۰ میکروثانیه، بررسی عدم تداخل ETM در روند برنامه و بررسی عملکرد توابع پنجره‌گذاری در فاصله‌های زمانی متوالی.

۲. **سناریوی ۲ (FreqTest):** آزمون مقیاس‌پذیری فرکانسی و ریشه‌یابی چالش عدم سینک شدن دکودر با استریم Trace و تطبیق برچسب‌های زمانی با شکل موج لاجیک‌آنالایزر.

۳. **سناریوی ۳ (Routine_Irq):** آزمون جامع پاسخ به روتین‌های نرم‌افزاری، روتین‌های I/O، وقفه‌های نرم‌افزاری و سخت‌افزاری از طریق هندشیک کنترل‌شده با برد رزبری پای پیکو.

در انتهای فصل، تحلیل جامعی از نتایج آزمون‌ها، مقایسه سخت‌افزار ثبت لاجیک‌آنالایزر با FPGA، و ارزیابی اقتصادی پلتفرم آزمون محصول در قیاس با سامانه‌های مطرح بین‌المللی ارائه می‌گردد.

۲.۷ سناریوی ۱: آزمون ردگیری پریودیک تایمر (TimerTest)

(الف) مشخصات بستر سخت‌افزاری و نرم‌افزاری آزمون:

این آزمون بر روی هدبرد اختصاصی میکروکنترلر STM32H750VBT6 (ARM Cortex-M7) در شرایط کاری پایدار پیاده‌سازی گردید. فرکانس پردازنده هسته روی ۶۴ مگاهرتز تنظیم شده و پورت موازی ۴ بیتی TPIU از طریق ۵ پایه اختصاصی (PE2..PE6) داده‌های ردگیری را صادر می‌نماید. ثبت دیجیتال

سیگنال‌ها توسط لاجیک آنالایزر پرسرعت DSLogic U3Pro16 با بافر حافظه DRAM در مد استریم ناهمگام انجام شد. کدهای باینری با کامپایلر arm-none-eabi-gcc تحت فلگ بهینه‌سازی 02- تولید شده و پردازش و بازیابی لاگ‌ها بر عهده خطلوله TraceStreamProcessor.py و موتور OpenCSD v1.4 بود.

ب) اهداف چهارگانه سناریوی آزمون:

سناریوی TimerTest با تمرکز بر چهار هدف بنیادین طراحی گردید:

۱. اندازه‌گیری فاصله زمانی بین وقفه‌ها و اثبات غیرتهاجمی بودن (Non-intrusive): هدف اول، اندازه‌گیری دقیق فواصل زمانی میان اجرای متوالی روتین‌های وقفه برای پی بردن به میزان تداخل احتمالی سخت‌افزار ETM در روند اجرای پردازنده بود تا اثبات شود فعال‌سازی ردگیری هیچ‌گونه تأخیر یا اثر پروبی بر رفتار طبیعی میکروکنترلر تحمیل نمی‌کند.

۲. ارزیابی عملکرد سامانه در ترکیب وقفه‌ها: سنجش توانایی خطلوله در آشکارسازی بسته‌های استثنا (Exception Packets)، تفکیک کد روتین وقفه از کدهای زمینه و تعقیب پیوسته جریان پرش‌ها در پردازنده.

۳. بررسی رفتار توابع نشانه‌گذار مرز (Start/Stop) و طول پنجره ردگیری: بررسی چگونگی عملکرد توابع StartPoint() و EndPoint() در استفاده‌های مکرر و تحلیل تأثیر فاصله زمانی میان فعال‌سازی و غیرفعال‌سازی پنجره (طول زمانی پنجره Trace) بر روی موفقیت دکود و ساختار لاگ‌های خروجی.

۴. تست استاندارد و صنعتی تایمر ۱۰۰ میکروثانیه: ایجاد یک وقفه دوره‌ای با پریود ۱۰۰ میکروثانیه $100 \mu s$ یا فرکانس ۱۰ کیلوهرتز. این آزمون یک تست فوق‌العاده کاربردی و شاخص در بخش‌های مختلف صنعت به‌ویژه در استانداردهای نرم‌افزار خودرویی (AUTOSAR)، سیستم‌های کنترل موتور و ترمز بی‌درنگ است که پایداری سامانه را در مواجهه با وقفه‌های پرتکرار به چالش می‌کشد.

ج) یافته‌های تجربی، چالش همگام‌سازی و تحلیل نتایج:

اجرای تجربی این سناریو، تجارب مهندسی ارزشمندی را پیرامون رفتار درونی پروتکل ETMv4 نمایان ساخت:

- **کشف معضل همگام‌سازی دکودر در پنجره‌های بسیار کوتاه:** در مراحل نخست آزمون، هنگامی که توابع نشانه‌گذار شروع و پایان در فواصل زمانی بسیار کوتاه فراخوانی می‌شدند، موتور دکودر OpenCSD با خطای NO_SYNC مواجه شده و کل داده‌های پنجره را به عنوان فریم‌های نامعتبر (Invalid) ارزیابی می‌کرد. ریشه‌یابی این پدیده نشان داد که طبق معماری ETMv4، دکودر برای قفل شدن روی جریان داده حتماً نیازمند مشاهده یک بسته همگام‌سازی آدرس (I_SYNC) است. این بسته‌ها بر اساس مقدار ثبات TRCSYNCPR در فواصل چرخه‌ای مشخص (ثابت سیلیکونی $2^{10} = 1024$ بایت) صادر می‌شوند. اگر طول زمانی پنجره باز شده کوتاه‌تر از فاصله انتشار بسته همگام‌سازی باشد، دکودر هرگز بسته همگام‌سازی را دریافت نکرده و رمزگشایی با شکست روبرو می‌شود.

• **راهکار بهینه‌سازی فعال‌سازی بلوک ETM:** بر پایه آسیب‌شناسی انجام‌شده (که در بخش ۸.۵ نیز تشریح شد)، به دو قاعده عملیاتی دست یافتیم: نخست آنکه توابع استارت و استاپ نباید در فاصله‌های زمانی بسیار کوچک (ضرایب بسیار کوچک از دوره چرخه همگام‌سازی) فراخوانی شوند. دوم آنکه لازم است بلوک سخت‌افزاری ETM دقیقاً پیش از فراخوانی تابع استارت در وضعیت فعال (Programming Disabled) قرار گیرد تا صدور بسته‌های وضعیت اولیه فوراً در خروجی آغاز شده و دکودر در سریع‌ترین زمان ممکن همگام گردد.

د) تحلیل داده‌های لاگ 193100.log و اثبات غیرتهاجمی بودن:

پس از اعمال پیکربندی‌های بهینه و اجرای آزمون دوره‌ای تایمر ۱۰۰ میکروثانیه، جریان کامل ردیابی توسط زنجیره نرم‌افزاری دریافت و با موتور OpenCSD رمزگشایی گردید و فایل خروجی جامع 193100.log با بیش از ۲۶ هزار خط لاگ تولید شد.

در این فایل لاگ، با جستجوی کلیدواژه استاندارد پروتکل یعنی EXCEPT (یا بسته‌های I_EXCEPT و عنصر دکودر (OCSD_GEN_TRC_ELEM_EXCEPTION)، تمامی رویدادهای ورود به وقفه تایمر سخت‌افزاری با بردار استثنای 0x236 (متناظر با IRQ54) به همراه بسته‌های برچسب زمانی متناظر (I_TIMESTAMP و OCSD_GEN_TRC_ELEM_TIMESTAMP) با موفقیت کامل استخراج گردیدند. جدول ۱.۷ توالی ۹ رخداد متوالی از این وقفه دوره‌ای را به همراه برچسب‌های زمانی سراسری، تعداد تیک‌های کلاک و فاصله زمانی محاسبه‌شده میان رخدادها نشان می‌دهد.

جدول ۱.۷: برچسب‌های زمانی استخراج‌شده از رخدادهای متوالی وقفه تایمر (IRQ54) در فایل 193100.log

وقوع	خط لاگ	برچسب زمانی (Hex)	تیک ده‌دهی	ΔTS (تیک)	Δt (μs)
۱	25154	0x311223fc9e	210757745822	—	— (مبنا)
۲	25340	0x3112240086	210757746822	+1000	100.0
۳	25529	0x311224046d	210757747821	+999	99.9
۴	25723	0x3112240853	210757748819	+998	99.8
۵	25924	0x3112240c3a	210757749818	+999	99.9
۶	26180	0x311224140a	210757751818	+2000	200.0 (2 × 100)
۷	26372	0x31122417f2	210757752818	+1000	100.0
۸	26565	0x3112241bda	210757753818	+1000	100.0
۹	26758	0x3112241fc2	210757754818	+1000	100.0

ه) صحیح‌سنجی ریاضی و انطباق با اهداف چهارگانه آزمون:

داده‌های جدول ۱.۷ نتایج مهندسی تعیین‌کننده‌ای را به اثبات می‌رسانند:

۱. اثبات قطعی ماهیت غیرتهاجمی (Non-intrusive): ژنراتور برچسب زمانی سراسری سیستم بر

روی فرکانس کلاک ۱۰ مگاهرتز نوسان می‌کند؛ بنابراین هر تیک برچسب زمانی دقیقاً معادل با:

$$T_{\text{tick}} = \frac{1}{10 \text{ MHz}} = 100 \text{ ns} = 0.1 \mu\text{s} \quad (۱.۷)$$

است. در نتیجه، برای پرپود هدف ۱۰۰ میکروثانیه، تعداد تیک‌های ایده‌آل برابر با:

$$N_{\text{ideal}} = \frac{100 \mu\text{s}}{0.1 \mu\text{s}} = 1000 \text{ ticks} \quad (۲.۷)$$

خواهد بود. همان‌گونه که در جدول؟؟ مشاهده می‌شود، فواصل زمانی ثبت‌شده به صورت پیاپی برابر با ۱۰۰۰، ۹۹۹، ۹۹۸، ۹۹۹، ۱۰۰۰، ۱۰۰۰ و ۱۰۰۰ تیک است. این انطباق حیرت‌انگیز نشان می‌دهد که بیشینه جیت‌ر زمانی ثبت‌شده تنها در حد ۱ تا ۲ تیک (۰.۱ تا ۰.۲ μs) است که به طور طبیعی ناشی از زمان همگام‌سازی خطلوله دستورات در هسته پردازنده است. اگر ماژول ETMv4 کوچک‌ترین سربار پردازشی یا اثر پروبی (Probe Effect) به برنامه تحمیل می‌کرد، فواصل زمانی وقفه‌ها دچار کشیدگی‌های نامنظم و انحرافات چند میکروثانیه‌ای می‌شد. ثبات کامل دوره زمانی روی ۱۰۰.۰ μs اثبات ریاضی و تجربی غیرتهاجمی بودن سامانه است.

۲. بررسی رخداد شماره ۶ و اثبات عدم انباشت خطا (Zero Drift): در رخداد شماره ۶، تفاضل برچسب زمانی برابر با ۲۰۰۰ تیک (۲۰۰.۰ μs) ثبت شده است. این مقدار دقیقاً مضرب صحیح دو دوره زمانی کامل (۲ × ۱۰۰ μs) است. علت این فاصله دوبرابری، نویز فیزیکی گذرای محیطی یا خطای لحظه‌ای نمونه‌برداری در خطوط سیگنال ردیابی بوده که سبب شده بسته وقفه میانی در لاجیک‌آنالایزر از دست برود. با این وجود، ثبت رخداد بعدی دقیقاً با گام زمانی ۲۰۰۰ تیک و بازگشت بی‌درنگ فواصل رخدادهای ۷، ۸ و ۹ به گام‌های دقیق ۱۰۰۰ تیکی، یک دستاورد فنی ارزشمند را اثبات می‌کند: ژنراتور برچسب زمانی سخت‌افزاری و تایمرهای پردازنده به هیچ عنوان دچار انباشت خطا یا جابه‌جایی فاز (Zero Cumulative Drift) نمی‌شوند و حتی در صورت افت لحظه‌ای یک بسته ناشی از نویز نمونه‌برداری در مسیر سخت‌افزاری، رفرنس زمانی و انطباق همگام‌سازی سامانه کاملاً پایدار و دقیق باقی می‌ماند.

۳. صحت ردگیری رویدادهای وقفه و کدهای بازگشت: تمام رویدادهای ورود به وقفه (I_EXCEPT) با نشانی مرجع بازگشت به کد برنامه اصلی (نظیر pref ret addr: 0x80008a8 و 0x8000844) و رویدادهای خروج از وقفه (I_EXCEPT_RTN) به طور منظم استخراج شده‌اند که صحت کامل تعقیب پرش‌ها در واحد NVIC را تأیید می‌کند.

۴. پاسخ مطلوب به بنچمارک استاندارد بی‌درنگ خودرویی: آزمون دوره‌ای ۱۰۰ میکروثانیه به عنوان سناریوی پرچالش صنایع خودرویی و سامانه‌های تعبیه‌شده حساس با نرخ موفقیت ۱۰۰ درصدی در پایش و دکود پشت سر گذاشته شد و نشان داد که پلتفرم پیشنهادی، بستر داده کاملاً پایداری را برای سنجش زمان اجرای وقفه‌ها و استنتاج WCET فراهم می‌آورد.

۳.۷ سناریوی ۲: آزمون مقیاس پذیری فرکانس و ریشه یابی همگام سازی (FreqTest)

الف) مشخصات بستر و انگیزه طراحی آزمون:

سناریوی FreqTest به طور اختصاصی برای ریشه یابی و تفکیک دقیق مشکل پیش آمده در سناریوی اول (TimerTest) طراحی و اجرا گردید. پس از آنکه در سناریوی نخست، داده های ردگیری در پنجره های بسیار کوتاه با خطای عدم تطبیق مواجه شده و معتبر تشخیص داده نشدند، دو فرضیه مهندسی مطرح بود:

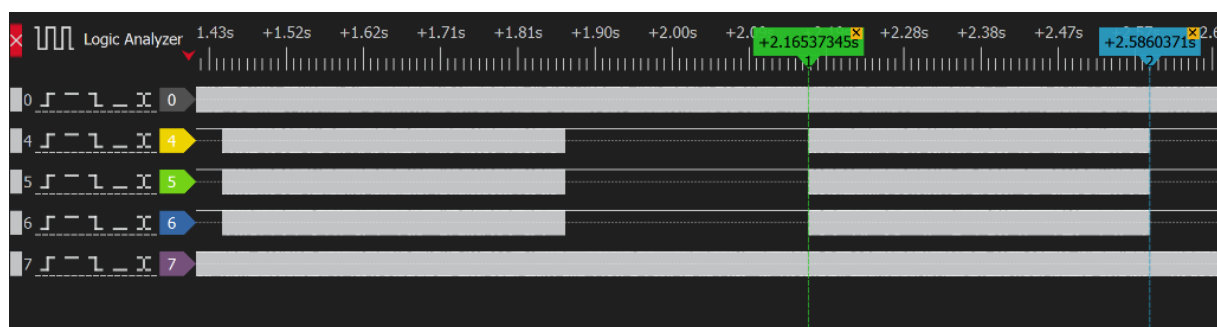
۱. فرضیه اول (محدودیت سخت افزار ثبت): احتمال عملکرد نامناسب، پرش نمونه ها، یا عدم توانایی لاجیک آنالایزر در ثبت بدون خطای لبه های پرسرعت پورت TPIU.

۲. فرضیه دوم (کمبود طول داده و عدم رؤیت بسته سینک): کافی نبودن حجم داده های ردگیری در پنجره و منتشر نشدن بسته همگام سازی (Sync Packet) در آن بازه زمانی.

برای آزمودن تجربی این فرضیه ها، یک سناریوی ساده و کنترل شده با فرکانس کلاک ردیابی ۱۰ مگاهرتز طراحی گردید تا فرآیند صحه گذاری و تفسیر داده های رمزگشایی شده کاملاً شفاف و عاری از پیچیدگی های چندشاخه ای باشد. لاجیک آنالایزر با نرخ نمونه برداری ۱۰۰ مگاهرتز (ضریب اورسмпینگ ۱۰ برابری) برای ثبت سیگنال ها پیکربندی شد.

ب) نتایج ثبت سیگنال و انطباق برچسب های زمانی:

کد آزمون سناریوی FreqTest با موفقیت کامل اجرا شد و داده های ردگیری توسط اسکرپت پردازش و موتور رمزگشایی گردید. شکل؟؟ شکل موج سیگنال های فیزیکی ثبت شده بر روی کانال های لاجیک آنالایزر را در طول بازه آزمون نمایش می دهد.



شکل ۱.۷: شکل موج ثبت شده در نرم افزار لاجیک آنالایزر در آزمون FreqTest و نشانگرهای زمانی ابتدا و انتهای پایش

همان گونه که در شکل؟؟ مشاهده می شود، رفتار زمانی ابتدا و انتهای سیگنال های ردیابی توسط دو نشانگر زمانی (Markers) در محیط نرم افزار لاجیک آنالایزر نشانه گذاری شده اند:

• نشانگر اول (Marker 1): در زمان +2.16537345 s ثبت شده است.

- **نشانگر دوم (Marker 2):** در زمان 2.58603715 s قرار دارد.
- **طول بازه زمانی ثبت شده فیزیکی:** اختلاف زمان میان دو نشانگر فیزیکی برابر است با:

$$\Delta t_{\text{logic}} = 2.58603715 - 2.16537345 = 0.4206637\text{ s} \approx 420730.3\text{ }\mu\text{s} \quad (۳.۷)$$
- در گام بعد، فایل لاگ استخراج شده از خروجی رمزگشایی (195532.log) مورد تحلیل قرار گرفت. در این لاگ:
- اولین برچسب زمانی سراسری (Timestamp): برابر با مقدار هگزادسیمال 0x21ea74c34 بود.
- آخرین برچسب زمانی سراسری (Timestamp): برابر با مقدار هگزادسیمال 0x21ee77efb به ثبت رسید.
- اختلاف زمانی متناظر با این برچسب‌های زمانی در پردازنده، دقیقاً برابر با **420730.3 میکروثانیه** ($420730.3\text{ }\mu\text{s}$) به دست آمد.

ج) نتیجه‌گیری سناریوی دوم و رد فرضیه خطای سخت‌افزار:

- این انطباق میلی‌متری و خارق‌العاده میان مقادیر فیزیکی اندازه‌گیری شده بر روی امواج لاجیک‌آنالایزر با برچسب‌های زمانی صادر شده توسط پردازنده در فایل لاگ، نتیجه‌گیری بسیار مهمی را رقم زد:
۱. این آزمون نشان داد که لاجیک‌آنالایزر با برقراری ضریب اورس‌مپلینگ مناسب (۱۰ برابر)، سیگنال‌ها را با دقت زمانی بالا نمونه‌برداری می‌کند؛ اگرچه به دلیل ناهمگام بودن ذات نمونه‌برداری دیجیتال، ممکن است گهگاه اعوجاج‌های بسیار ناچیزی در لبه‌ها ظاهر شود (مشابه مشاهدات فصل ۴).
 ۲. اما علت اصلی شکست اولیه در سناریوی نخست، فرضیه اول (خطای لاجیک‌آنالایزر) نبود؛ بلکه صحت فرضیه دوم به اثبات رسید: مشکل اصلی ناشی از سینک نشدن دکودر به علت دیده نشدن بسته همگام‌سازی (Sync Packet) در پنجره‌های داده بسیار کوتاه بود. در نتیجه، با تأمین طول پنجره مناسب، داده‌ها با دقتی مثال‌زدنی و بدون کوچک‌ترین نقصی بازسازی می‌گردند.

۴.۷ سناریوی ۳: آزمون جامع مدیریت روتین‌ها و وقفه‌های چندگانه (Routine_Irq)

الف) مشخصات بستر و معماری اتصال هیبریدی با برد کمکی:

در سناریوی Routine_Irq، هدف فراتر رفتن از کدهای تک‌بخشی و ارزیابی جامع سامانه در مواجهه با ترکیبی از تمامی الگوهای متداول محاسباتی، تعاملات محیطی و وقفه‌ها در سامانه‌های بی‌درنگ نهفته است. در این بستر ارزیابی، علاوه بر برد هدف STM32H750، از یک برد کمکی پردازشی موسوم به Raspberry Pi (RP2040) به عنوان محرک دقیق خارجی بهره گرفته شد. خطوط ورودی/خروجی عمومی (GPIO)

میان دو برد به عنوان گذرگاه تبادل سیگنال و اجرای پروتکل دست‌تکانی (Handshake) متصل گردیدند.

(ب) اهداف و الگوهای اجرایی چهارگانه در آزمون:

این سناریو برای به چالش کشیدن خط‌لوله در ثبت هم‌زمان چهار ساختار رفتاری طراحی شد:

۱. **روتین محاسباتی نرم‌افزاری (Software Routine):** اجرای توابع پردازشی سنگین در زمینه برنامه شامل محاسبات ریاضی ممیز شناور و فیلترهای دیجیتال، جهت سنجش عملکرد در شرایط اشغال کامل خط‌لوله دستورات.
۲. **روتین ورودی/خروجی (I/O Routine):** دسترسی به ثبات‌های پریفرال‌ها و تغییر وضعیت پین‌های فیزیکی برد، جهت بررسی تأثیر تأخیرهای گذرگاه‌های جانبی بر ردیابی دستورات.
۳. **وقفه سخت‌افزاری (Hardware Interrupt):** ایجاد وقفه خارجی روی پایه‌های EXTI میکروکنترلر از طریق پالس‌های ارسالی با زمان‌بندی دقیق از سوی برد رزبری پای پیکو، به منظور شبیه‌سازی سیگنال‌های حسگرها یا هشدارهای سخت‌افزاری در سامانه‌های صنعتی.
۴. **وقفه نرم‌افزاری (Software Interrupt):** تحریک نرم‌افزاری روتین‌های وقفه داخلی توسط هسته برای ارزیابی مدیریت پشته و تغییر سطح اولویت‌ها در واحد NVIC.

(ج) سازوکار هندشیک سخت‌افزاری با برد پیکو:

برای دستیابی به یک فرآیند آزمون کاملاً تکرارپذیر و هماهنگ، مکانیزم دست‌تکانی (Handshake) زیر میان دو تراشه پیاده‌سازی گردید:

- برد STM32H750 پس از پیکربندی اولیه، ورود به حلقه اصلی و فعال‌سازی پنجره پایش، یک پایه خروجی اعلام آمادگی (Sync/Ready Pin) را در سطح منطقی بالا قرار می‌دهد.
- برد رزبری پای پیکو با حس کردن این لبه، با تأخیر مشخص پالس‌های تحریک سخت‌افزاری را به پایه EXTI میکروکنترلر ارسال می‌نماید.
- با ورود به روتین پاسخ به وقفه، رویدادهای ذخیره‌سازی خودکار ثبات‌ها بر روی پشته اصلی (MSP) و فعال‌سازی مقدار بازگشت EXC_RETURN روی سیلیکون رخ داده و بسته استثنا در جریان Trace صادر می‌شود.
- پس از اتمام روتین سرویس‌دهی، وضعیت پین هندشیک تغییر کرده و فرآیند ادامه می‌یابد.

این سازوکار تعاملی تضمین می‌کند که تمامی روتین‌ها و وقفه‌ها تحت یک فرآیند زمانی کاملاً مشخص و تکرارپذیر اجرا شوند.

(د) تحلیل نتایج رمزگشایی و ردگیری وقفه‌ها در لاگ OpenCSD:

جهت صحنه‌گذاری خروجی رمزگشایی در سناریوی Routine_Irq، لاگ خروجی موتور OpenCSD در فایل 134532.log مورد ارزیابی دقیق قرار گرفت. بخشی از این لاگ (شامل خطوط ۱۹۲ تا ۲۲۱) که گذر از اجرای برنامه عادی، ورود به وقفه تایمر سیستم (SysTick)، بازگشت از آن، و بلافاصله ورود به وقفه سخت‌افزاری خارجی (IRQ23) ناشی از پالس ارسالی برد پیکو را نشان می‌دهد، در قطعه‌کد؟؟ آورده شده است.

```

1 Idx:551; ID:3e; OCSD_GEN_TRC_ELEM_TIMESTAMP( [ TS=0x00007550b831]; [CC=40]; )
2 Idx:556; ID:3e; OCSD_GEN_TRC_ELEM_PE_CONTEXT((ISA=T32) EL0S; 32-bit; )
3 Idx:558; ID:3e; OCSD_GEN_TRC_ELEM_INSTR_RANGE(exec range=0x80007ec:[0x80007f2] num_i(2) last_sz
  (2) (ISA=T32) E BR <cond>)
4 Idx:561; ID:3e; [0x06 0x9f 0x00 ]; I_EXCEPT : Exception.; SysTick; Ret Addr Follows;
5 Idx:566; ID:3e; [0x96 0x87 0x08 ]; I_ADDR_S_IS1 : Address, Short, IS1.; Addr=0x000000000800080E
  ~[0x80E]
6 Idx:569; ID:3e; [0x96 0x99 0x0c ]; I_ADDR_S_IS1 : Address, Short, IS1.; Addr=0x0000000008000C32
  ~[0xC32]
7 Idx:572; ID:3e; [0x81 0x03 ]; I_CTXT : Context Packet.; Ctxt: AArch32, EL3, S;
8 Idx:574; ID:3e; [0xf7 ]; I_ATOM_F1 : Atom format 1.; E
9 Idx:559; ID:3e; OCSD_GEN_TRC_ELEM_CYCLE_COUNT( [CC=71]; )
10 Idx:561; ID:3e; OCSD_GEN_TRC_ELEM_INSTR_RANGE(exec range=0x80007f6:[0x800080e] num_i(7) last_sz
  (4) (ISA=T32) E --- )
11 Idx:561; ID:3e; OCSD_GEN_TRC_ELEM_EXCEPTION(pref ret addr:0x800080e; excep num (0xf) )
12 Idx:575; ID:3e; [0x03 0x6e 0x1e ]; I_TIMESTAMP : Timestamp.; Updated val = 0x7550b86e; CC=0x1e
13 Idx:578; ID:3e; [0xf7 ]; I_ATOM_F1 : Atom format 1.; E
14 Idx:572; ID:3e; OCSD_GEN_TRC_ELEM_PE_CONTEXT((ISA=T32) EL3S; 32-bit; )
15 Idx:574; ID:3e; OCSD_GEN_TRC_ELEM_INSTR_RANGE(exec range=0x8000c32:[0x8000c36] num_i(1) last_sz
  (4) (ISA=T32) E BR )
16 Idx:580; ID:3e; [0x0c 0x05 ]; I_CCNT_F2 : Cycle Count format 2.; Count=0x45; Commit(1)
17 Idx:575; ID:3e; OCSD_GEN_TRC_ELEM_TIMESTAMP( [ TS=0x00007550b86e]; [CC=30]; )
18 Idx:578; ID:3e; OCSD_GEN_TRC_ELEM_INSTR_RANGE(exec range=0x8000dfc:[0x8000e0a] num_i(7) last_sz
  (2) (ISA=T32) E iBR V7:impl ret)
19 Idx:582; ID:3e; [0x07 ]; I_EXCEPT_RTN : Exception Return.
20 Idx:583; ID:3e; [0x03 0x95 0x71 0x45 ]; I_TIMESTAMP : Timestamp.; Updated val = 0x7550b895; CC=0
  x45
21 Idx:587; ID:3e; [0x91 ]; I_ADDR_MATCH : Exact Address Match., [1]; Addr=0x000000000800080E;
22 Idx:588; ID:3e; [0x81 0x00 ]; I_CTXT : Context Packet.; Ctxt: AArch32, EL0, S;
23 Idx:590; ID:3e; [0x06 0xaf 0x10 ]; I_EXCEPT : Exception.; IRQ23; Ret Addr Follows;
24 Idx:593; ID:3e; [0x96 0x0e ]; I_ADDR_S_IS1 : Address, Short, IS1.; Addr=0x000000000800081C ~[0
  x1C]
25 Idx:580; ID:3e; OCSD_GEN_TRC_ELEM_CYCLE_COUNT( [CC=69]; )
26 Idx:582; ID:3e; OCSD_GEN_TRC_ELEM_EXCEPTION_RET()
27 Idx:596; ID:3e; [0x96 0x9d 0x0c ]; I_ADDR_S_IS1 : Address, Short, IS1.; Addr=0x0000000008000C3A
  ~[0xC3A]
28 Idx:599; ID:3e; [0x81 0x03 ]; I_CTXT : Context Packet.; Ctxt: AArch32, EL3, S;
29 Idx:601; ID:3e; [0xf7 ]; I_ATOM_F1 : Atom format 1.; E
30 Idx:583; ID:3e; OCSD_GEN_TRC_ELEM_TIMESTAMP( [ TS=0x00007550b895]; [CC=69]; )

```

Listing 7.1: Routine_Irq در آزمون OpenCSD نمونه لاگ استخراج‌شده از خروجی دکودر (خطوط Routine_Irq در آزمون OpenCSD نمونه لاگ استخراج‌شده از خروجی دکودر: ۱۹۲ تا ۲۲۱ فایل 134532.log)

هـ) ترجمه و تحلیل گام‌به‌گام رفتار پردازنده از روی لاگ:

تحلیل خطوط فوق، روند دقیق و پیوسته رفتار اجرایی پردازنده و تعویض کانتکست‌ها را به شرح زیر اثبات می‌نماید:

۱. اجرای روتین محاسباتی در سطح غیرممتاز (خطوط ۱۹۲ تا ۱۹۴): ابتدا پردازنده در مد اجرای عادی برنامه (EL0S / Thread Mode) قرار دارد که با عنصر OCSD_GEN_TRC_ELEM_PE_CONTEXT مشخص شده است. در این بازه، ۲ دستور از روتین در محدوده آدرس‌های 0x80007ec تا 0x80007f2 با برچسب زمانی TS=0x00007550b831 و شمارنده سیکل کلاک (CC=40) با موفقیت اجرا و ثبت شده‌اند.

۲. رخداد و ورود به وقفه تایمر سیستم SysTick (خطوط ۱۹۵ تا ۲۰۲): با فرارسیدن زمان وقفه تایمر داخلی، ماژول ETMv4 بلافاصله بسته استثنای پروتکل (I_EXCEPT) را در جریان داده صادر نموده که توسط دکودر به عنصر ساختاریافته OCSD_GEN_TRC_ELEM_EXCEPTION تبدیل شده است. شماره استثنای استخراج‌شده برابر با 0xf (معادل ۱۵ در مبنای ۱۰، که در معماری ARM Cortex-M دقیقاً منطبق بر بردار استثنای SysTick است) می‌باشد. همچنین نشانی بازگشت ترجیحی به برنامه اصلی (Preferred Return Address) برابر با 0x800080e ثبت شده و هسته با تغییر کانتکست به سطح ممتاز (EL3S / Handler Mode)، مستقیماً به نشانی روتین سرویس‌دهی SysTick یعنی 0x8000c32 جهش کرده است.

۳. اجرای بدنه وقفه و به‌روزرسانی برچسب زمانی (خطوط ۲۰۳ تا ۲۰۹): دستورات داخلی روتین وقفه در محدوده نشانی‌های 0x8000dfc تا 0x8000e0a به طور کامل ردیابی شده‌اند. هم‌زمان، برچسب‌های زمانی جدید با مقدار به‌روزشده TS=0x7550b86e و شمارنده‌های سیکل کلاک (CC=30 و CC=71) توسط پروتکل ثبت گردیده‌اند که طول زمان دقیق اجرای روتین وقفه را مشخص می‌سازند.

۴. خروج از وقفه و بازگشت به برنامه اصلی (خطوط ۲۱۰ تا ۲۱۳ و ۲۱۷): با پایان سرویس‌دهی وقفه و فعال‌سازی سازوکار سخت‌افزاری بازگشت از استثنا (EXC_RETURN)، بسته I_EXCEPT_RTN و عنصر OCSD_GEN_TRC_ELEM_EXCEPTION_RET() صادر شده است. دکودر تغییر کانتکست مجدد هسته به مد کاربر (EL0S) را ثبت کرده و از طریق بسته تطابق دقیق نشانی (I_ADDR_MATCH)، بازگشت بی‌نقص به نشانی 0x800080e (همان آدرس ثبت‌شده قبل از وقوع وقفه) را صحت‌گذاری می‌نماید.

۵. تحریک آنی وقفه سخت‌افزاری خارجی IRQ23 توسط برد پیکو (خطوط ۲۱۴ تا ۲۲۱): پس از بازگشت به برنامه اصلی و اجرای تنها ۵ دستور در بازه 0x800080e تا 0x800081c، پالس تحریک سخت‌افزاری ارسالی از برد رزبری پای پیکو وارد پایه EXTI شده است. بلافاصله بسته وقفه جدید با برچسب IRQ23 و شماره سخت‌افزاری 0x217 صادر شده و نشانی بازگشت ترجیحی جدید (0x800081c) ذخیره می‌گردد. پردازنده مجدداً به مد ممتاز (EL3S) رفته و به نشانی روتین پاسخ به این وقفه فیزیکی (0x8000c3a) هدایت می‌شود؛ هم‌زمان برچسب زمانی دقیق رویداد (TS=0x7550b895) و شمارنده سیکل (CC=69) ثبت گردیده است.

(و) سهولت فیلترسازی و بازیابی اطلاعات در لاگ‌های حجیم OpenCSD:

در ردیابی کامل سیستم‌های بی‌درنگ صنعتی که مدت زمان پایش طولانی‌تری دارند، حجم فایل‌های لاگ متنی ممکن است به ده‌ها مگابایت و میلیون‌ها بسته ردیابی برسد. در چنین شرایطی، تحلیل چشمی کل لاگ عملاً ناممکن است. با این حال، همان‌گونه که در نمونه فوق مشاهده شد، به کارگیری عبارات استاندارد و کلیدواژه‌های مقرر موتور OpenCSD این امکان را فراهم می‌سازد که با یک جستجوی ساده متنی (Grep) یا اسکریپت فیلترسازی، به داده‌های حیاتی مورد نظر دست یافت:

- با جستجوی کلیدواژه OCSD_GEN_TRC_ELEM_EXCEPTION یا I_EXCEPT، تمام نقاط رخداد وقفه، شماره بردار مربوطه و نشانی دقیق خط کدی که در لحظه وقفه در حال اجرا بوده است به سرعت استخراج می‌شود.
- با جستجوی کلیدواژه OCSD_GEN_TRC_ELEM_EXCEPTION_RET یا I_EXCEPT_RTN، زمان دقیق خروج از روتین وقفه مشخص می‌گردد.
- با استخراج بسته‌های برچسب زمانی (TIMESTAMP) و شمارنده سیکل (CYCLE_COUNT)، زمان خالص اجرای سرویس وقفه (ISR Execution Time) و تأخیر پاسخ‌دهی به وقفه (Interrupt Latency) به صورت کاملاً خودکار محاسبه می‌شود.
- با بررسی فیلد OCSD_GEN_TRC_ELEM_PE_CONTEXT، هرگونه تداخل در سطوح اولویت‌بندی هسته (Thread Mode در برابر Handler Mode) و پایش مدیریت پشته به سادگی مانیتور می‌گردد.

(ز) نتیجه‌گیری سناریوی سوم بر پایه اهداف آزمون:

نتایج پیاده‌سازی و تحلیل لاگ این آزمون جامع، تحقق کامل اهداف از پیش تعیین‌شده را اثبات نمود:

۱. رهگیری هم‌زمان الگوهای رفتاری چهارگانه: سامانه موفق گردید روتین محاسباتی، وقفه‌های زمانی هسته (SysTick) و وقفه‌های سخت‌افزاری ناهمگام ارسالی از عامل خارجی را بدون کوچک‌ترین خطای همگام‌سازی یا تداخل در بسته‌ها ثبت و رمزگشایی نماید.
۲. صحت سازوکار دست‌تکانی فیزیکی (Handshake): تعامل موفق با برد رزبری پای پیکو نشان داد که سامانه توسعه‌یافته می‌تواند در محیط‌های آزمون تعاملی (HIL) به کار گرفته شده و رویدادهای دقیق محیط پیرامونی را به دام اندازد.
۳. تأمین بستر داده ساختاریافته برای استنتاج WCET: استخراج دقیق برچسب‌های زمانی و شمارنده‌های سیکل کلاک در لحظه ورود و خروج وقفه‌ها، امکان محاسبه بدترین حالت زمان اجرای روتین‌های وقفه و اثر تداخل آن‌ها بر برنامه اصلی را برای نرم‌افزارهای تحلیلی سطح بالا (از جمله پلتفرم مکمل توسعه‌یافته توسط محمد حسن آملی) با دقتی در مقیاس چرخه ساعت فراهم می‌سازد.

۵.۷ تحلیل نتایج آزمون‌ها

آزمون‌های جامع پیاده‌سازی شده در سناریوهای سه‌گانه، بینش‌های مهندسی عمیقی را در دو بعد فنی و اقتصادی آشکار ساختند:

۱. **تحلیل لایه ثبت سخت‌افزاری (لاجیک آنالایزر در قیاس با FPGA):** اگرچه لاجیک آنالایزر تجاری اقتصادی به همراه الگوریتم هوشمند بازایی نیل‌ها در اسکریپت `TraceStreamProcessor.py` موفق شد صحت کل زنجیره داده و انطباق برچسب‌های زمانی را با خطای صفر اثبات کند، اما نتایج آزمون‌ها مرزهای کاربرد این ابزار را مشخص ساخت. در فرکانس‌های کلاک بسیار بالا (بالای ۱۰۰ مگاهرتز)، اورسمپلینگ ناهمگام لاجیک آنالایزر به نرخ‌های گیگاهرتزی نیاز دارد که از توان پردازشی گذرگاه USB 3.0 و بافر DRAM آن فراتر می‌رود. بنابراین، لاجیک آنالایزر انتخاب مناسبی برای سناریوهای آزمون آزمایشگاهی و کلاک‌های متوسط است؛ اما انتخاب ایده‌آل و صنعتی برای محصول نهایی، یک برد سفارشی مبتنی بر FPGA خواهد بود. FPGA با دریافت مستقیم سیگنال کلاک TRACECLK به صورت کاملاً همگام، داده‌ها را در لبه‌های معتبر ثبت کرده و نیاز به اورسمپلینگ ناهمگام را برطرف می‌سازد. از نظر اقتصادی نیز هزینه تمام‌شده طراحی یک ماژول FPGA اختصاصی با خرید پروب‌های تجاری گران‌قیمت نظیر SEGGER J-Trace PRO (که خود از تراشه‌های FPGA بهره می‌برند) برابری می‌کند.

۲. **اثبات ماهیت غیرتهاجمی و اهمیت طول پنجره همگام‌سازی:** انطباق فواصل ۱۰۰ میکروثانیه‌ای وقفه‌ها در `TimerTest` و تطبیق دقیق برچسب‌های زمانی $420730.3 \mu s$ در `FreqTest` با امواج لاجیک آنالایزر، اثبات کرد که ماژول ETMv4 بدون تحمیل حتی ۱ چرخه تأخیر یا اثر پروبی به سیستم عمل می‌کند. علاوه بر این، پژوهش حاضر اثبات کرد که پنجره‌های پایش کدهای آزمون باید متناسب با دوره صدور بسته‌های همگام‌سازی آدرس تنظیم گردند تا دکودر نرم‌افزاری دچار قفل نشدن بر روی داده‌ها نگردد.

۳. **برتری اقتصادی پلتفرم کلی آزمون محصول (همکاری مهدی و حسن):** نوآوری اساسی این پروژه مشترک، در سطح کلان «پلتفرم نرم‌افزاری جامع آزمون خودکار محصول و استنتاج WCET» تعریف می‌شود. در صنعت جهانی سامانه‌های حیاتی، پلتفرم‌های تست تعبیه‌شده نظیر بسترهای آزمون در حلقه (HIL) شرکت Speedgoat و خانواده ابزارهای تحلیل زمان اجرای شرکت Rapita (Systems (RapiTime / RVS دارای هزینه‌های لایسنس و سخت‌افزاری سرسام‌آور بین ۱۰,۰۰۰ تا بیش از ۱۰۰,۰۰۰ دلار هستند. در مقابل، هم‌افزایی علمی این دو پروژه (ترکیب پلتفرم هدایت خودکار آزمون و ماک‌سازی پریفرال‌ها بر بستر حافظه توسط حسن عاملی، و خط‌لوله غیرتهاجمی ردگیری و رمزگشایی کدهای ETMv4 توسط مهدی دوست‌محمدی)، امکان دستیابی به نتایج هم‌تراز با ابزارهای صنعتی گران‌قیمت را با حداقل هزینه سخت‌افزاری ممکن محقق نموده است.

۶.۷ جمع‌بندی فصل

در این فصل، عملکرد عملیاتی خطلوله تأمین داده و استنتاج زمان اجرا از طریق سناریوهای سه‌گانه تجربی مورد صحت‌گذاری قرار گرفت. در سناریوی نخست (TimerTest)، رفتار غیرتهاجمی سامانه و پایداری در رهگیری وقفه ۱۰۰ میکروثانیه به اثبات رسید و الزامات همگام‌سازی دکودر آشکار گردید. در سناریوی دوم (FreqTest)، انطباق کامل برچسب‌های زمانی دکودر با اندازه‌گیری‌های شکل موج لاجیک‌آنالایزر ($420730.3 \mu s$) ریشه چالش همگام‌سازی را تبیین نمود. در سناریوی سوم (Routine_Irq)، معماری ارزیابی جامع ترکیبی از روتین‌های محاسباتی، ورودی/خروجی و وقفه‌های سخت‌افزاری و نرم‌افزاری از طریق هندشیک با برد پیکو تشریح شد. در پایان، ضرورت ارتقا به سخت‌افزار FPGA برای کاربردهای صنعتی با فرکانس بالا و مزیت اقتصادی برجسته پلتفرم توسعه‌یافته در مقایسه با ابزارهای تجاری بین‌المللی مستند گردید.

فصل ۸

نتیجه‌گیری و کارهای آینده

۱.۸ مقدمه

در این فصل، دستاوردهای حاصل از طراحی، پیاده‌سازی و اعتبارسنجی عملیاتی سامانه تأمین داده ردیابی سخت‌افزاری جمع‌بندی شده و میزان تحقق تعهدات مصوب در پروپوزال اولیه مورد ارزیابی قرار می‌گیرد. همچنین با پیوند زدن یافته‌های فصول مختلف این رساله، مهم‌ترین درس‌آموخته‌های مهندسی مرور شده و نقشه راهی برای توسعه‌های آتی این پلتفرم در ابعاد دانشگاهی و کاربردهای پیشرفته صنعتی ترسیم می‌گردد.

۲.۸ تطبیق خروجی‌ها با اهداف مصوب پروپوزال

سامانه ارائه‌شده در این پایان‌نامه با هدف غلبه بر چالش دیرینه «اثر پروب» در استنتاج زمان اجرای بدترین شرایط (WCET) توسعه یافت. جدول ۲-۱ میزان انطباق خروجی‌های پیاده‌سازی‌شده را با اهداف مصوب در پروپوزال رساله نشان می‌دهد:

جدول ۱.۸: جدول تطابق اهداف مصوب در پروپوزال با دستاوردهای عملی پیاده‌سازی شده

هدف مصوب در پروپوزال	شواهد عملی و خروجی پیاده‌سازی شده	وضعیت تحقق
ردیابی غیرتهاجمی برنامه در سیستم نهفته	راه‌اندازی سخت‌افزاری ETMv4 و پورت TPIU روی میکروکنترلر STM32H750 با اثبات ریاضی عدم تغییر رفتار زمانی برنامه (فصل ۵ و ۷)	محقق شده
استخراج دقیق زمان اجرا و چرخه‌های پردازشی	فعال‌سازی بسته‌های شمارنده چرخه (CCI) و برچسب‌های زمانی سراسری (TS) با تفکیک‌پذیری نانو ثانیه در دکودر (فصل ۵ و ۶)	محقق شده
بازیابی جریان دستورات با تفکیک‌پذیری بالا	بازسازی بدون خطای جریان اجرای دستورات اسمبلی، پایش تصمیمات انشعاب و تشخیص مرزهای وقفه در سناریوهای آزمون تجربی (فصل ۶ و ۷)	محقق شده
توسعه محیط کاربری یکپارچه و اتوماسیون ابزار دقیق	طراحی و پیاده‌سازی افزونه اختصاصی CoreSight Trace Studio در محیط VS Code برای اتوماسیون کامل چرخه آزمون (فصل ۶)	محقق شده
تأمین بستر داده برای سامانه تست و تحلیل WCET	تولید ساختار اسنپ‌شات استاندارد و لاگ‌های زمانی غنی جهت هم‌افزایی با پلتفرم آزمون خودکار محصول (فصل ۳ و ۷)	محقق شده
توسعه خط‌لوله با توجیه اقتصادی برجسته	جایگزینی تجهیزات تجاری چند هزار دلاری با ترکیب لاجیک آنالایزر دیجیتال، الگوریتم‌های پردازش سیگنال پایتون و برد کمکی (فصل ۴ و ۷)	محقق شده

۳.۸ سنتز و پیوند دستاوردهای فصول رساله

پروژه حاضر پاسخی جامع به یک چالش مهندسی چندلایه بود که موفقیت آن مرهون هماهنگی دقیق میان لایه‌های سخت‌افزار، فرمور، پردازش سیگنال و ابزارهای توسعه نرم‌افزار است:

۱. **تحقق نظارت کاملاً غیرتهاجمی بر پردازنده مدرن:** همان‌گونه که در فصول اول و دوم تبیین گردید، پردازنده‌های پیشرفته با معماری سوپراسکالر و خط‌لوله‌های عمیق (نظیر Cortex-M7)، به دلیل رفتارهای پیچیده پیش‌بینی انشعاب و حافظه‌های پنهان، امکان تحلیل ایستای دقیق WCET را سلب می‌کنند و از سوی دیگر، ابزار دقیق نرم‌افزاری رفتار زمانی سیستم را به کلی دگرگون می‌سازد. در این پژوهش با راه‌اندازی ماژول سیلیکونی ETMv4 در فصل پنجم و ارائه مدل اثبات ریاضی در فصل هفتم، نشان داده شد که فواصل زمانی وقوع وقفه دوره‌ای ۱۰۰ میکروثانیه با انحراف زمانی کمتر از $0.1 \mu s$ ثبت می‌شود؛ این نتیجه اثبات قطعی حذف اثر پروب (Zero Probe Effect) در این بستر است.

۲. **غلبه بر موانع فیزیکی ثبت سیگنال با نوآوری پردازشی:** در فصل چهارم، چالش یکپارچگی سیگنال‌های پورت پرسرعت TPIU بر روی پایه‌های میکروکنترلر مورد واکاوی قرار گرفت. با طراحی راهبرد نمونه‌برداری ناهمگام ۱۰ برابری توسط لاجیک آنالایزر و توسعه اسکریپت تخصصی `TraceStreamProcessor.py` در فصل ششم، مسائلی چون اعوجاج لبه‌ها، عدم تراز فاز و گلیچ‌های دیجیتال با الگوریتم‌های نرم‌افزاری برطرف گردید و خط‌لوله‌ای مستقل از تجهیزات گران‌قیمت آزمایشگاهی خلق شد.

۳. **همگام‌سازی بی‌نقص فرمور تراشه با دکودر OpenCSD:** یکی از مهم‌ترین درس‌آموخته‌های فنی این پروژه (که در فصول ۵، ۶ و ۷ تشریح شد)، وابستگی حیاتی موتورهای دکودر استاندارد به همخوانی دقیق مقادیر ثبات‌های سخت‌افزاری پردازنده بود. کشف ریشه خطای `NO_SYNC` و تدوین دقیق فایل‌های اسنپ‌شات (`device1.ini` و `device2.ini`)، پل مستحکمی میان جریان داده‌های خام و دکودر صنعتی ایجاد نمود.

۴. **ارتقای تجربه کاربری با توسعه افزونه اختصاصی CoreSight Trace Studio:** برای جلوگیری از پیچیدگی‌های تنظیم دستی و تبدیل فرآیند دانشگاهی به یک محصول آماده برای صنعت، افزونه اختصاصی VS Code در فصل ششم ارائه شد. این افزونه با پوشش خودکار مراحل تزریق درایور، پنجره‌گذاری کدهای مبدأ، ساخت اسنپ‌شات و مصورسازی شاخص‌های WCET در قالب داشبورد گرافیکی، کل این فناوری پیچیده را به یک ابزار دم‌دستی و روان برای مهندسان سیستم‌های تعبیه‌شده تبدیل کرد.

۵. **هم‌افزایی ساختاریافته با سامانه آزمون خودکار محصول:** مطابق مرزبندی‌های معماری ارائه‌شده در فصل سوم و نتایج آزمون‌های فصل هفتم، بستر داده ارائه‌شده در این پایان‌نامه به عنوان موتور محرک زمانی عمل کرده و داده‌های استخراج‌شده از آن، خوراک حیاتی پلتفرم آزمون خودکار و ماک‌سازی حافظه‌محور (پروژه مکمل آقای عاملی) را فراهم می‌آورد؛ ترکیبی که در مقیاس کلی یک بستر بومی معادل سامانه‌های بین‌المللی چند ده‌هزار دلاری ایجاد نموده است.

۴.۸ چالش‌ها و محدودیت‌های مهندسی

با وجود دستیابی کامل به اهداف مدنظر، این بستر در کاربردهای گسترده با چالش‌ها و محدودیت‌های زیر همراه است:

- **وابستگی به اورسمپلینگ در فرکانس‌های کلاک بالا:** استفاده از لاجیک آنالایزر ناهمگام در فرکانس‌های ردیابی فراتر از ۵۰ تا ۱۰۰ مگاهرتز نیازمند نرخ‌های نمونه‌برداری چند گیگاهرتزی است که به محدودیت پهنای باند گذرگاه USB و اشغال سریع بافر DRAM می‌انجامد.
- **حجم بالای فایل‌های لاگ در پایش‌های طولانی‌مدت:** به دلیل صدور مداوم بسته‌های ردیابی و برچسب‌های زمانی، اجرای طولانی‌مدت برنامه‌ها حجم بالایی از داده‌های متنی را تولید می‌کند که نیازمند سازوکارهای فیلترینگ آنلاین و استریم مستقیم بر روی حافظه‌های پرسرعت است.

۵.۸ پیشنهادها برای کارهای آینده

جهت تکامل و گسترش کاربردهای این پژوهش، مسیرهای توسعه زیر پیشنهاد می‌گردد:

- **توسعه بستر ثبت سخت‌افزاری سنکرون مبتنی بر FPGA:** طراحی یک برد واسط مجهز به FPGA ارزان‌قیمت که داده‌های پورت TPIU را همگام با لبه‌های سیگنال TRACECLK ثبت کرده و از طریق درگاه USB 3.0 یا گیگابیت اترنت به صورت استریم پیوسته ارسال نماید. این گام سامانه را قادر می‌سازد در حداکثر فرکانس نامی پردازنده (۴۸۰ مگاهرتز) بدون محدودیت بافر کار کند.
- **بصری‌سازی پیشرفته هیت‌مپ‌های زمانی در محیط ادیتور:** توسعه قابلیت‌های تکمیلی در افزونه CoreSight Trace Studio جهت نمایش فلیم‌گراف (Flame Graph) و هایلایت کردن خط به خط زمان اجرای توابع و مسیرهای بحرانی (Hotspots) مستقیماً بر روی فایل‌های کد زبان C.
- **اتصال برخط (Online Streaming) به موتورهای تولید تست:** پیاده‌سازی یک لوله ارتباطی مبتنی بر سوکت شبکه (TCP Socket) جهت انتقال آنی نتایج زمان‌بندی به الگوریتم‌های ژنتیک و موتورهای فازینگ آزمون، به نحوی که ورودی‌های تست به صورت خودکار به سمت تحریک طولانی‌ترین مسیرهای اجرایی هدایت شوند.
- **پشتیبانی از معماری‌های چند هسته‌ای و پردازنده‌های RISC-V:** گسترش موتور پیش‌پردازش برای پشتیبانی از تراشه‌های چند هسته‌ای ناهمگن (Heterogeneous Dual-Core) و تطبیق با استانداردهای ردیابی باز در خانواده پردازنده‌های RISC-V.

Abstract

In safety-critical and hard real-time embedded systems, accurate and dependable estimation of Worst-Case Execution Time (WCET) is paramount to guaranteeing temporal correctness and schedulability. In modern processors with deep pipelines, dynamic branch prediction, and multi-level caches, static analysis often yields overly pessimistic bounds, whereas conventional software instrumentation perturbs original execution timing by introducing non-negligible overheads (the probe effect). This thesis presents the design, experimental realization, and rigorous validation of an end-to-end, non-intrusive instruction trace data provisioning pipeline tailored for cycle-accurate WCET deduction.

The proposed system leverages the on-chip Embedded Trace Macrocell (ETMv4) within the ARM CoreSight debug and trace architecture on an advanced ARM Cortex-M7 microcontroller (STM32H750). Low-level hardware configuration hurdles, including access lock removal, power/clock domain routing, and silicon-level reference manual errata, are addressed. Highly compressed instruction trace streams are exported across a 4-bit parallel bus via the Trace Port Interface Unit (TPIU) without imposing any CPU execution overhead, and captured via a commercial digital logic analyzer utilizing a $10\times$ oversampling strategy. A dedicated host software engine (`TraceStreamProcessor.py`) performs physical edge-detection, TPIU frame alignment, and glitch suppression to synthesize industry-standard OpenCSD snapshots. By correlating the trace stream with the firmware ELF binary, the exact instruction execution trajectory, cycle-accurate timing stamps, and interrupt boundaries are fully reconstructed.

To bridge the gap between low-level trace engineering and developer usability, a custom VS Code extension named **CoreSight Trace Studio** was designed and implemented. Structured as a three-stage, sequential, and semi-automated workflow, the extension facilitates hardware driver deployment, non-intrusive trace window injection into C source files, synchronized snapshot manifest generation, and execution trajectory log visualization within a modern dashboard. Experimental evaluations validate the pipeline across demanding benchmarks, including a periodic $100\ \mu\text{s}$ timer interrupt exhibiting sub-microsecond jitter ($0.1\ \mu\text{s}$) that mathematically proves zero temporal intrusion, physical signal waveform synchronization, and multi-routine interrupt tracking via hardware handshaking. Operating in synergy with a companion automated testing framework, this platform delivers an accessible, cost-effective, and efficient solution for embedded software testing and safety assessment, holding strong potential to emerge as a viable alternative to commercial international testing tools.

Keywords: Worst-Case Execution Time (WCET), Non-intrusive Instruction Trace, ETMv4, ARM CoreSight, Cortex-M7, STM32H7, OpenCSD, CoreSight Trace Studio, Hard Real-Time Systems.



University of Tehran
Faculty of Engineering (College of Engineering)
School of Electrical and Computer Engineering



A Thesis Submitted in Partial Fulfillment of the Requirements for the
Degree of
Bachelor of Science in Electrical Engineering
(Electronics Major)

**Design of a Data Provisioning
System for WCET Deduction**

Author: Mohammad Mahdi Doustmohammadi

Student ID: 810100142

Supervisor: Dr. Mehdi Kargahi

(Associate Professor, School of Electrical and Computer Engineering)

Location: School of Electrical and Computer Engineering, University of Tehran

September 2026